



Prédiction de la gravité des accidents de la circulation routière en France

Rapport final - Format digital

Résumé

Dans le cadre de notre formation Data Analyst chez DataScientest, nous avons mené un projet visant à développer un modèle prédictif de la gravité des accidents de la circulation routière en France. À partir de données ouvertes fournies par le gouvernement français, notre objectif était d'identifier les facteurs influençant la gravité des accidents et de construire un modèle de machine learning permettant de prédire cette gravité (blessé léger, blessé grave, tué).

Membres de l'équipe

- Appolinaire ALLARASSEM
- Juliette MEUNIER
- Macha LAGUNE

Encadrant pédagogique

- Antoine TARDIVON
-

Lien vers le projet GitHub

<https://github.com/Projet-Accidents/Projet-Accidents>

Introduction	4
I. Nettoyage des données	5
I.1. Exploration initiale des données	5
I.2. Définition de l'objectif et sélection des données	6
I.3. Construction du Dataframe analytique unifié (2019 - 2023)	7
I.3.1. Stratégie de traitement et d'agrégation	7
I.3.2. Définition opérationnelle de la gravité de l'accident	9
I.3.3. Traitement spécifique par rubrique	10
I.3.3.a. Traitement détaillé de la table Usagers	10
I.3.3.b. Traitement détaillé de la table Véhicules	12
I.3.3.c. Traitement détaillé de la table Lieux	15
I.3.3.d. Traitement détaillé de la table Caractéristiques et finalisation du Dataframe	17
II. Analyse exploratoire de données	20
II.1. Distribution de la variable cible: gravité de l'accident	20
II.2. Contexte temporel de l'accident(Caractéristiques)	20
II.3. Facteurs géographiques et infrastructurels (Lieux)	25
II.3.2. Nombre de voies et gravité de l'accident	26
II.3.3. Catégorie de la route et gravité de l'accident	28
II.3.4. Localisation de l'accident	30
II.4. Profil et statut des usagers impliqués (usagers)	32
II.4.1. Répartition des accidents par sexe et gravité	32
II.4.2. Répartition des accidents par âge et gravité	34
II.4.3. Répartition des accidents par système de sécurité et gravité	35
II.5. Typologie et Points d'impact des véhicules impliqués (véhicules)	36
II.5.1. Catégorie de Véhicules et Gravité des accidents	36
II.5.2. Type de choc et Gravité des accidents	38
II.6. Matrices de corrélation	39
II.6.1. Corrélation selon les caractéristiques des usagers	39
II.6.2. Corrélations selon les conditions de routes	41
II.6.3. Corrélation selon les obstacles et types de chocs	42
III. Modélisation pour prédire la gravité d'un accident	43
III.1. Choix du modèle approprié de classification	44
III.2. Choix des métriques d'analyse et des classes prioritaires	45
III.3. Choix des stratégies de gestion du déséquilibre des classes	46
III.4. Modèle de classification à 3 classes: Blessé léger, Blessé hospitalisé et Tué	49
III.4.1 Analyse globale des résultats des modèles de classification à 3 classes	49
III.4.2. Modèle recommandé: Modèle Random Forest	51

III.4.3. Analyse et interprétation des résultats du Modèle Random Forest	52
III.5. Modèle de classification binaire: Urgent et Non urgent	56
III.5.1. Analyse globale des modèles de classification binaire	56
III.5.2. Analyse des résultats des différentes approches du XGBoost	60
III.6. Simulation de prédiction de la gravité d'un nouvel accident	62
III.6.1. Avec le Modèle Random Forest à 3 classes	62
III.6.2. Avec le Modèle binaire XGBoost	63
Conclusion et axes d'amélioration	64
IV. ANNEXES: Codes associés au projet	66
IV.1. Nettoyage et analyse des données	66
IV.2. Modélisation	80

Introduction

Face à l'urgence vitale d'un accident de la circulation, chaque second compte. L'efficacité de la chaîne de secours, depuis l'appel initial jusqu'à l'arrivée des équipes médicales, repose sur une évaluation rapide et précise de la situation. Or, comment, dans le tumulte des premiers instants, les centres d'appels d'urgence peuvent-ils déterminer avec fiabilité le niveau de gravité réel d'un sinistre pour dépêcher les ressources les plus adaptées – ni trop, ni trop peu. C'est précisément ce défi opérationnel et humain majeur que ce projet ambitionne de relever.

En s'appuyant sur la richesse des données de la Base Accidents Corporels (BAAC) pour la période 2019-2023, ce travail vise à développer un modèle prédictif robuste. **L'objectif : estimer, dès la réception d'une alerte, si un accident est potentiellement grave, afin de déclencher le nombre et le type de secours adaptés.**

Pour y parvenir, une démarche rigoureuse a été entreprise : d'abord, un nettoyage et une structuration méticuleuse des données hétérogènes issues des tables Caractéristiques, Lieux, Véhicules et Usagers ; ensuite, une analyse exploratoire approfondie pour déceler les facteurs temporels, géographiques, infra structurels et humains les plus déterminants dans la survenue et la sévérité des accidents ; enfin, une phase de modélisation comparative, évaluant différentes approches algorithmiques (Random Forest, Régression Logistique, XGBoost, KNN) et stratégies de gestion du déséquilibre des classes, pour identifier la solution la plus performante au regard des priorités opérationnelles – notamment la détection fiable des cas les plus graves.

Ce rapport détaille l'ensemble de ce processus, depuis les défis initiaux liés aux données jusqu'à la sélection et l'interprétation du modèle final retenu . Il met en lumière non seulement la faisabilité technique d'une telle prédiction, mais explore également les enseignements concrets tirés de l'analyse et les implications potentielles de ces travaux pour l'optimisation de la réponse aux urgences. In fine, il s'agit de transformer l'incertitude de l'appel initial en une action de secours plus ciblée, plus rapide, et potentiellement salvatrice.

I. Nettoyage des données

I.1. Exploration initiale des données

Une analyse préliminaire des jeux de données disponibles et de leur documentation a mis en lumière plusieurs caractéristiques et défis inhérents à ces sources :

- **Evolution temporelle des données**

Une discontinuité notable dans les méthodologies de collecte et de formatage des données a été identifiée à partir de l'année 2019. Cette transition se manifeste par des modifications dans la structure et la précision de certaines variables (par exemple, l'enregistrement de l'heure de l'accident passant d'intervalles de 30 minutes à une précision à la minute). Les données post-2019 apparaissent également plus complètes.

- **Clé d'identification commune**

L'identifiant unique de l'accident (Num_Acc) constitue le champ pivot, présent de manière cohérente dans les quatre catégories de fichiers (Caractéristiques, Lieux, Véhicules, Usagers), permettant ainsi le rapprochement des informations relatives à un même événement.

- **Volumétrie et granularité**

Le volume total des données sur l'ensemble de la période historique disponible (exemple : 1 247 773 enregistrements pour la catégorie 'Lieux' de 2005 à 2023) soulève des questions quant à la faisabilité et la pertinence de consolider l'intégralité des années et des catégories en une unique structure de données. De plus, des niveaux d'agrégation hétérogènes existent : les fichiers 'Véhicules' et 'Usagers' contiennent nativement plusieurs enregistrements pour un même accident (un par véhicule impliqué et un par usager impliqué, respectivement), nécessitant une agrégation pour une analyse au niveau de l'accident.

- **Dimensionnalité et complexité des variables**

L'ensemble des fichiers présente une dimensionnalité élevée, avec un nombre important de variables (56 colonnes mentionnées au total). Certaines de ces variables présentent une forte cardinalité (un grand nombre de valeurs uniques), ce qui peut complexifier les étapes de prétraitement et de modélisation.

- **Nécessité de définition de la variable cible**

L'exploration initiale souligne l'impératif de définir précisément la variable cible "gravité". Ce concept peut être interprété différemment (par exemple, en fonction du nombre de véhicules impliqués, du nombre total d'utilisateurs affectés, ou spécifiquement du nombre de décès). Une définition opérationnelle claire est donc requise pour orienter la modélisation.

I.2. Définition de l'objectif et sélection des données

L'objectif principal de ce projet est le développement d'un modèle prédictif destiné à assister les centres d'appels d'urgence. Ce modèle vise à évaluer le niveau de gravité d'un accident de la circulation dès sa signalisation, afin d'optimiser l'allocation et le type de ressources de secours à déployer. En conséquence, la variable cible pour la modélisation est la gravité de l'accident.

Dans cette perspective, deux décisions méthodologiques structurantes ont été prises :

- **Période d'analyse (2019 - 2023)**

L'analyse se concentrera sur les données collectées entre 2019 et 2023. Cette période a été retenue en raison de la disponibilité des données les plus récentes et de leur formatage homogène, suite aux évolutions de la base de données en 2019. Cette sélection permet la fusion cohérente des quatre sources de données (Caractéristiques, Lieux, Véhicules, Utilisateurs), essentielle pour relier l'ensemble des variables explicatives à la variable cible. De plus, elle fournit un volume de données substantiel (273.226 enregistrements) jugé adéquat pour l'entraînement d'un modèle d'apprentissage automatique robuste.

- **Réduction initiale des variables**

Une première sélection des variables a été opérée afin d'exclure celles jugées a priori non pertinentes pour la prédiction de la gravité de l'accident.

- **Traitement Préliminaire des Données**

Conformément à ces décisions, la première étape de préparation des données a consisté à agréger les jeux de données annuels. Les dataframes correspondant à chaque catégorie ('rubrique') ont été concaténés sur la période 2019-2023, en tirant parti de la structure de données constante au sein de chaque catégorie sur ces années.

I.3. Construction du Dataframe analytique unifié (2019 - 2023)

La création d'un Dataframe unique, intégrant les informations pertinentes des quatre sources de données (Caractéristiques, Lieux, Véhicules, Usagers) pour la période 2019-2023, constitue une étape cruciale pour l'analyse et la modélisation. L'hétérogénéité des niveaux d'agrégation entre ces tables a nécessité une approche méthodique de prétraitement et de fusion.

I.3.1. Stratégie de traitement et d'agrégation

La stratégie adoptée a consisté à traiter séquentiellement les tables en partant de la plus granulaire (Usagers), puis en intégrant progressivement les informations des autres tables (Véhicules, Lieux, Caractéristiques), tout en agrégeant les données pour obtenir une représentation unique par accident (Num_Acc).

Le processus général appliqué à chaque table source avant la fusion comprenait :

- **Analyse et sélection des variables**

Il s'agit d'un examen de chaque colonne, de ses modalités (valeurs uniques) et de sa pertinence vis-à-vis de l'objectif de prédiction de la gravité. Les colonnes jugées non pertinentes ont été écartées.

- **Regroupement des modalités**

Pour les variables catégorielles conservées, une analyse de la distribution des valeurs a conduit à des regroupements logiques afin de réduire la dimensionnalité tout en préservant l'information essentielle. Par exemple, les multiples états de la surface de la route ('surf') ont été regroupés en trois catégories : 'route_seche', 'route_mouillee_enneigee', et 'etat_route_autre'.

- **Transformation et encodage**

Les variables catégorielles prétraitées ont été transformées, souvent via des techniques d'encodage (type one-hot encoding implicite par agrégation), pour créer de nouvelles variables numériques représentant la présence ou le nombre d'occurrences de chaque modalité par accident.

L'objectif final de ces transformations était de passer d'une structure où plusieurs lignes pouvaient correspondre à un même accident (Exemple : une ligne par usager, une par véhicule) à une structure finale où chaque ligne représente un accident unique (Num_Acc), agrégeant l'ensemble des informations pertinentes sous forme de variables numériques.

Exemple illustratif de transformation par agrégation :

Une structure initiale telle que :

| NumAcc | Surface | Condition Atmosphérique | Type de véhicule | Catégorie Usager |

| :----- | :----- | :----- | :----- | :----- |

| 001 | mouillee | brouillard | Poids Lourd | conducteur |

| 001 | mouillee | brouillard | Moto | Conducteur |

Est transformée après traitement et agrégation par NumAcc en :

| NumAcc | Surf_Mouillee_Enneigee | Surf_sèche | Brouillard | Pluie | Poids_Lourds |
Zroues_3roues | Cat_conducteur | Cat_passager |

```
| :---- | :----- | :----- | :----- | :--- | :----- | :----- |
:----- | :----- |

| 001 | 1 | 0 | 1 | 0 | 1 | 1 | 2 | 0 |
```

I.3.2. Définition opérationnelle de la gravité de l'accident

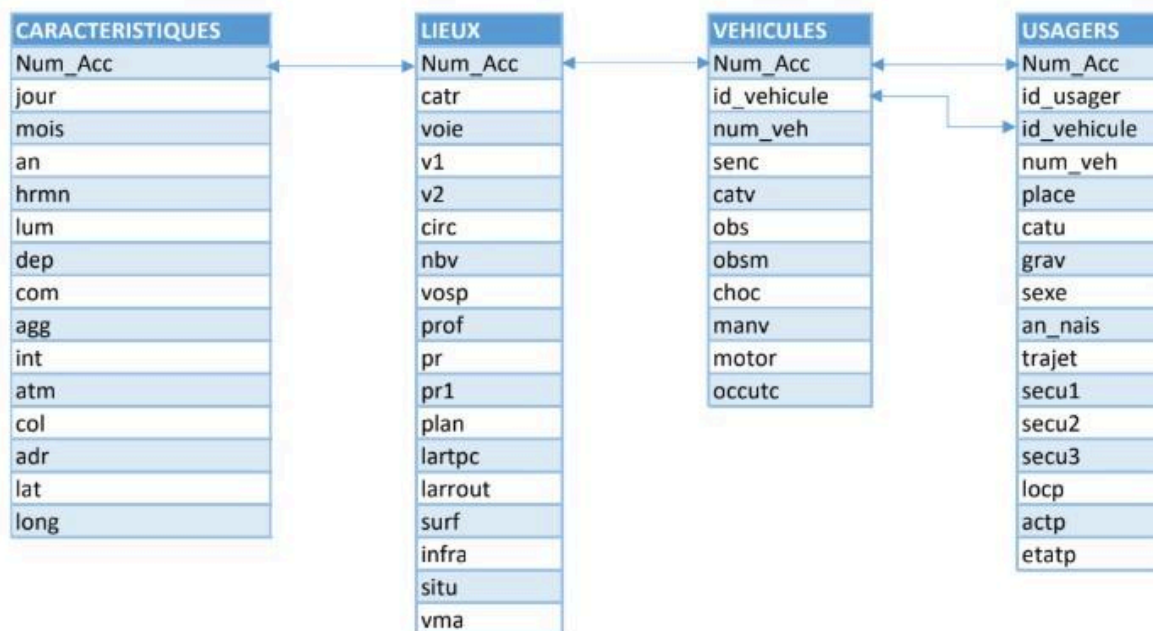
La variable cible, *gravité_accident*, a été définie au niveau de l'accident. Pour chaque accident, la gravité retenue correspond au niveau le plus élevé enregistré parmi tous les usagers impliqués. Les niveaux sont codifiés comme suit :

- 4 : Accident Mortel (au moins un usager tué)
- 3 : Accident avec Blessé(s) Hospitalisé(s) (au moins un usager hospitalisé, aucun tué)
- 2 : Accident avec Blessé(s) Léger(s) (au moins un usager blessé léger, aucun hospitalisé ou tué)

Note : Conformément à la description des données sources, les accidents ne comportant que des usagers indemnes ne sont pas recensés, car la base ne contient que les accidents ayant fait au moins une victime nécessitant des soins. Le niveau 1 "indemne" n'est donc pas utilisé comme catégorie de gravité pour un accident.

I.3.3. Traitement spécifique par rubrique

Dans le but de conserver un maximum de données, un travail est nécessaire sur chacune des tables avant de pouvoir merger le tout dans un dataset unique.



I.3.3.a. Traitement détaillé de la table Usagers

	Num_Acc	id_usager	id_vehicule	num_veh	place	catu	grav	sexe	an_nais	trajet	secu1	secu2	secu3	locp	actp	etatp
0	202200000001	1 099 700	813 952	A01	1	1	3	1	2008.0	5	2	8	-1	-1	-1	-1
1	202200000001	1 099 701	813 953	B01	1	1	1	1	1948.0	5	1	8	-1	-1	-1	-1
2	202200000002	1 099 698	813 950	B01	1	1	4	1	1988.0	9	1	0	-1	0	0	-1
3	202200000002	1 099 699	813 951	A01	1	1	1	1	1970.0	4	1	0	-1	0	0	-1
4	202200000003	1 099 696	813 948	A01	1	1	1	1	2002.0	0	1	0	-1	-1	-1	-1
5	202200000003	1 099 697	813 949	B01	1	1	4	2	1987.0	9	1	0	-1	-1	-1	-1
6	202200000004	1 099 694	813 947	A01	1	1	1	2	2000.0	0	1	8	-1	-1	-1	-1
7	202200000004	1 099 695	813 947	A01	10	3	4	2	1967.0	5	0	-1	-1	2	3	1
8	202200000005	1 099 692	813 945	B01	1	1	1	2	1975.0	0	1	0	-1	-1	-1	-1
9	202200000005	1 099 693	813 946	A01	1	1	4	2	1996.0	0	1	0	-1	-1	-1	-1

Le traitement initial s'est concentré sur la table Usagers, la plus granulaire avec ses 619 971 enregistrements initiaux (un par individu impliqué dans un accident). L'objectif était de nettoyer, structurer et enrichir les informations relatives à chaque usager avant de pouvoir les agréger au niveau de l'accident. Ce processus s'est déroulé comme suit.

- **Analyse de la catégorie d'utilisateur (catu)**

La variable `catu`, décrivant le rôle de l'utilisateur (conducteur, passager, piéton, etc.), a été l'une des premières à être traitée. Les différentes modalités ont été transformées, vraisemblablement par encodage (type one-hot), afin de créer des variables indicatrices distinctes pour chaque catégorie pertinente. Cette étape (aboutissant à `df_usage4`) a permis de quantifier la présence de chaque type d'utilisateur.

- **Consolidation des données de sécurité (secu1, secu2, secu3)**

Les informations sur les équipements de sécurité utilisés, initialement réparties sur trois colonnes (`secu1` à `secu3`), ont fait l'objet d'une attention particulière. Une logique de consolidation a été appliquée, impliquant potentiellement la somme des indicateurs pour des équipements identiques utilisés par un même individu (`df_secu`). Une étape clé a ensuite consisté à agréger ces informations consolidées de sécurité au niveau du véhicule (en groupant par `Num_Acc` et `id_vehicule`). Cette agrégation a ramené le nombre de lignes à 463 183, correspondant au nombre total de véhicules uniques impliqués, préparant ainsi le terrain pour la jointure avec la table `Vehicules`. Ce résultat (`df_secu_final`) a été fusionné avec les données de catégorie d'utilisateur (`df_usage4`) pour former `df_usag_5`.

- **Affinement de la place de l'utilisateur (place)**

La variable `place`, détaillant la position de l'utilisateur (siège conducteur, passager avant droit, etc.), a été simplifiée. Ses modalités ont été regroupées en cinq catégories principales plus générales : 'conducteur', 'passager avant', 'passager arrière', 'passager milieu', et 'piéton'. Ces informations de placement restructurées (`df_usag_place`) ont été intégrées. Une vérification a permis de constater une redondance entre les informations 'conducteur' et 'piéton' issues de `place` et celles déjà capturées via `catu`, menant à l'état `df_usag_6`.

- **Encodage du sexe (sexe)**

La variable `sexe` a été encodée pour créer des colonnes distinctes et mutuellement exclusives pour 'homme' et 'femme', tout en isolant potentiellement les cas 'non renseigné' (`df_sex`). Cette structuration a été ajoutée au Dataframe en cours (`df_usag6`), résultant en `df_usag7`.

- **Traitement de l'âge (dérivé d'an_nais)**

Calcul et Analyse : L'âge de chaque usager a été calculé à partir de son année de naissance (df_age). L'analyse de la distribution de cette nouvelle variable a révélé un âge médian de 35 ans.

Détection des Valeurs Aberrantes : Une investigation des valeurs extrêmes a été menée. Des graphiques et analyses statistiques ont mis en évidence la présence d'un nombre significatif d'enregistrements (2 060) correspondant à des âges très élevés (supérieurs à 95.5 ans, incluant des valeurs > 100 ans), considérés comme des outliers potentiels nécessitant un traitement ultérieur.

Catégorisation : Sur la base de la distribution et de l'analyse des outliers, l'âge a été catégorisé en tranches jugées pertinentes pour l'analyse : '0-17 ans', '18-60 ans', '61-95 ans', et une catégorie pour les outliers présumés ('>96 ans') (df_age_final).

Intégration : Ces catégories d'âge ont été fusionnées avec le Dataframe df_usag7.

Au terme de ces étapes de transformation spécifiques à la table Usagers, le Dataframe df_usag8 a été obtenu. Bien qu'encore au niveau de granularité de l'utilisateur à ce stade, il contenait désormais des informations nettoyées, structurées et catégorisées (type d'utilisateur, sécurité, place, sexe, tranche d'âge), prêtes pour l'étape suivante de fusion avec les données véhicules et l'agrégation finale par accident.

I.3.3.b. Traitement détaillé de la table Véhicules

	Num_Acc	id_vehicule	num_veh	senc	catv	obs	obsm	choc	manv	motor	occuttc	obstacle_fixe	obstacle_mobile
0	202100000001	201 764	B01	1	1	0	2	1	1	5	NaN	0	1
1	202100000001	201 765	A01	1	7	0	9	3	17	1	NaN	0	1
2	202100000002	201 762	A01	0	7	2	2	1	1	0	NaN	1	1
3	202100000002	201 763	B01	0	7	0	2	1	9	0	NaN	0	1
4	202100000003	201 761	A01	1	7	0	1	3	1	1	NaN	0	1
...	...	...	...	...	...	...	...	...	...	...	...	...	...
467164	201900058837	137 982 135	C01	1	7	0	2	1	2	1	NaN	0	1
467165	201900058838	137 982 132	A01	2	7	3	0	1	21	0	NaN	1	0
467166	201900058839	137 982 131	A01	2	33	0	0	7	1	1	NaN	0	0
467167	201900058840	137 982 129	B01	1	10	0	2	4	0	2	NaN	0	1
467168	201900058840	137 982 130	A01	1	10	0	2	1	0	1	NaN	0	1

467169 rows × 13 columns

Après la préparation des données usagers, l'étape suivante s'est concentrée sur la table Vehicules. Cette table, contenant initialement une ligne pour chaque véhicule impliqué dans un accident, a nécessité plusieurs transformations pour extraire et structurer les informations pertinentes en vue de l'agrégation finale par accident.

- **Analyse des obstacles heurtés (obsm, obs)**

Les colonnes obsm (obstacle mobile heurté) et obs (obstacle fixe heurté) ont été examinées. Une décision stratégique a été prise de simplifier cette information. Plutôt que de conserver la nature spécifique de l'obstacle, ces deux colonnes ont été transformées en variables binaires distinctes : obstacle_fixe et obstacle_mobile. Ces nouvelles variables prennent la valeur 1 si un obstacle (fixe ou mobile, respectivement) a été heurté, et 0 sinon. Ce choix a été motivé par l'hypothèse qu'un système embarqué (comme un boîtier de diagnostic envisagé pour l'application du modèle) serait plus susceptible de détecter la présence d'un choc avec un obstacle que d'en identifier précisément le type.

- **Regroupement des catégories de véhicules (catv)**

La variable catv, décrivant le type de véhicule, présentait une distribution dominée par certaines catégories (notamment les Véhicules Légers, catégorie 7) et suivie par les deux-roues motorisés et non motorisés. Afin de créer des groupes plus larges et potentiellement plus informatifs pour la modélisation, un regroupement a été effectué. Les nombreuses catégories initiales ont été consolidées en cinq groupes principaux, désignés par la nouvelle variable new_catv : 1 : VL (*Véhicule Léger*) et VU (*Véhicule Utilitaire*), 2 : Moto, scooter, 2 roues, 3 roues, quad, 3 : Poids Lourds (PL), 4: Bus/Car et 5 : Vélo/VAE/EDP (*Engin de Déplacement Personnel, exemple : trottinettes*) et Autres.

- **Simplification du point de choc (choc)**

La variable choc, indiquant le point d'impact initial sur le véhicule, a également été analysée et regroupée. En se basant sur la distribution des valeurs, les codes initiaux ont été reclassés en cinq catégories plus générales pour représenter la zone principale de l'impact : 0 : *Aucun choc apparent*, 1 : *Avant (codes initiaux 1, 2, 3)*, 2 : *Arrière (codes initiaux 4, 5, 6)*, 3 : *Côté (codes initiaux 7, 8)*, et 4 : *Tonneaux (code initial 9)*

Des variables indicatrices (*aucun_choc*, *choc_AV*, *choc_AR*, *choc_cote*, *choc_tonneaux*) ont été créées pour représenter ces catégories.

- **Sélection, encodage et préparation à la fusion**

Suite à ces transformations, une sélection a été opérée pour ne conserver que les colonnes jugées pertinentes : *Num_Acc*, *id_vehicule*, la catégorie regroupée *new_catv*, les indicateurs d'obstacles (*obstacle_fixe*, *obstacle_mobile*), et les indicateurs de points de choc.

La variable *new_catv* a ensuite été encodée (probablement via one-hot encoding) pour créer des colonnes distinctes pour chaque type de véhicule (exemple : *VL_VU*, *2roues_3roues_quad*, etc.), avec des noms plus explicites pour une meilleure lisibilité.

Une colonne *nbr_veh* a été ajoutée pour faciliter le comptage ultérieur du nombre total de véhicules par accident.

Le Dataframe résultant de ces étapes, nommé *df_veh1*, contenait les informations véhicules prétraitées, toujours au niveau du véhicule individuel.

- **Fusion avec les données Usagers et agrégation par accident**

L'étape cruciale suivante a été la fusion de ce Dataframe véhicule (*df_veh1*) avec le Dataframe usagers préalablement traité (*df_usag8*), en utilisant *Num_Acc* et *id_vehicule* comme clés de jointure.

Le Dataframe résultant (*df_merged_usag_veh*) contenait alors les informations combinées des usagers et des véhicules, mais toujours avec potentiellement plusieurs lignes par accident.

Pour atteindre l'objectif d'une ligne par accident, une opération d'agrégation (`groupby("Num_Acc").agg("sum")`) a été appliquée. En sommant les valeurs des colonnes numériques (qui étaient pour la plupart des indicateurs 0/1 ou des compteurs), cette opération a permis de calculer, pour chaque accident (*Num_Acc*), le nombre total d'usagers par catégorie de gravité, par place, par sexe, par tranche d'âge, ainsi que le nombre total de véhicules de chaque type, le nombre de véhicules ayant subi chaque type de choc, etc.

Cette séquence de traitement et d'agrégation a abouti au Dataframe `df_merged_usag_veh_final`. Ce dernier représente une synthèse riche des aspects humains et matériels de chaque accident, consolidée au niveau de l'événement unique (`Num_Acc`), et prête à être enrichie par les données de localisation et de caractéristiques générales.

I.3.3.c. Traitement détaillé de la table Lieux

	Num_Acc	catr		voie	v1	v2	circ	nbv	vosp	prof	pr	pr1	plan	lartpc	larrout	surf	infra	situ	vma
0	202000000001	4	HENRI BARBUSSE (AVENUE)	0.0	NaN	2	2	0	1	0	0	1	NaN	-1	1	0	1	50	
1	202000000002	4	MOUSSEAUX(CHEMIN)	0.0	NaN	2	2	0	1	0	100	3	NaN	-1	1	0	1	50	
2	202000000003	4	CARNOT(AVENUE)	0.0	NaN	-1	2	0	1	0	0	1	NaN	-1	1	0	1	50	
3	202000000004	4	VICTOR HUGO (AVENUE)	0.0	NaN	2	2	0	1	1	0	1	NaN	-1	1	0	1	30	
4	202000000005	3		35	0.0	NaN	1	1	0	1	4	674	2	NaN	-1	1	3	8	50
...	...	...		...	v1	...	...	...	...	...	...	...	...	...	...	...	...	...	...
289259	202200055298	3		71	-1.0	NaN	2	2	0	2	(1)	(1)	1	NaN	-1	1	0	8	80
289260	202200055299	3		973	-1.0	NaN	2	2	0	1	29	0	2	NaN	-1	1	0	3	80
289261	202200055300	3		22	0.0	D	2	2	0	1	39	553	2	NaN	-1	7	0	3	80
289262	202200055301	3		18	-1.0	D	2	2	0	1	30	125	1	NaN	-1	1	0	1	80
289263	202200055302	3		NaN	-1.0	NaN	3	4	0	1	(1)	(1)	1	NaN	4	1	0	1	70

289264 rows × 18 columns

L'étape suivante a porté sur l'intégration des informations contextuelles relatives au lieu de l'accident, issues de la table Lieux. L'objectif était d'extraire les caractéristiques pertinentes de l'environnement routier et de les préparer pour une fusion avec les données déjà agrégées des usagers et véhicules, tout en s'assurant de la cohérence au niveau de l'accident unique (`Num_Acc`).

- **Vérification préliminaire de cohérence**

Bien que le traitement principal concerne la table Lieux, une vérification contextuelle liée aux étapes précédentes a été mentionnée. Il a été noté une légère différence entre le nombre de lignes après l'agrégation Usagers/Véhicules (463 184, implicitement au niveau `id_vehicule` avant l'agrégation finale par `Num_Acc`) et le nombre total d'`id_vehicules` uniques dans la table Véhicules brute (467 169). Cette observation a conduit à une analyse comparative pour identifier les véhicules présents dans la table Véhicules mais potentiellement absents des enregistrements usagers, assurant ainsi une meilleure compréhension de la complétude des données avant les fusions finales. (Note : Cette vérification assure la qualité des données en amont de l'intégration des informations de Lieux).

- **Sélection des variables d'intérêt**

Pour la table Lieux spécifiquement, une sélection des variables jugées les plus pertinentes pour caractériser l'environnement de l'accident a été effectuée. Les colonnes retenues pour la suite du traitement furent : Num_Acc (la clé d'identification), catr (catégorie de route), circ (régime de circulation), nbv (nombre total de voies de circulation), vma (vitesse maximale autorisée), et surf (état de la surface de la chaussée).

- **Traitement de la catégorie route (catr)**

La variable catr a été simplifiée par regroupement en trois catégories principales pour une meilleure interprétabilité : 1 : *AutoRoute*, 2 : *Route Nationale, Départementale ou Communale (regroupant les codes initiaux 2, 3, 4)*, 3 : *Autre (incluant Hors réseau public, Parc de stationnement, Routes de métropole urbaine - codes initiaux 5, 6, 7, 9)*.

- **Simplification du régime de circulation (circ)**

De manière similaire, la variable circ a été regroupée en deux modalités pour distinguer les flux principaux : 1 : *À sens unique (correspondant au code initial 1)*, 2 : *Bidirectionnelle ou Autre (regroupant le code 2 et les éventuelles autres modalités)*

- **Traitement de l'état de la surface (surf)**

L'état de la surface, variable surf, a fait l'objet d'un regroupement basé sur l'analyse de sa distribution et de son impact potentiel sur la sécurité : 1 : *Normale (principalement surface sèche)*, 2 : *Mouillée/Enneigée*, 3 : *Autres (incluant boueuse, flaques, verglas, etc.)*. Une règle spécifique a été appliquée pour les cas où deux états de surface étaient renseignés (typiquement aux intersections) : l'état jugé le plus accidentogène (exemple : *mouillé plutôt que normal*) a été conservé. Les quelques valeurs manquantes (272 occurrences) ont été intégrées dans la catégorie 'Autre'.

- **Agrégation par accident et gestion de conflits**

L'étape finale pour cette table a consisté à agréger les données par Num_Acc pour obtenir une seule ligne par accident. Des règles spécifiques ont été définies pour gérer les variables pouvant présenter des valeurs multiples pour un même accident (par exemple, lors d'une intersection entre deux routes avec des caractéristiques différentes) :

- Pour le Nombre de Voies (nbv) : La valeur maximale a été conservée.
- Pour la Vitesse Maximale Autorisée (vma) : La valeur maximale a été conservée.

Le résultat de ce processus est le Dataframe `df_lieux_final`. Il contient 273 226 lignes, chacune correspondant à un accident unique, et inclut les caractéristiques retraitées et agrégées du lieu de l'accident. Ce Dataframe est désormais prêt à être fusionné avec les données caractéristiques générales de l'accident et les informations agrégées des usagers et véhicules.

I.3.3.d. Traitement détaillé de la table Caractéristiques et finalisation du Dataframe

	Num_Acc	jour	mois	an	hrmn	lum	dep	com	agg	int	atm	col	adr	lat	long
0	201900000001	30	11	2019	01:30	4	93	93053	1	1	1	2	AUTOROUTE A3	48,8962100	2,4701200
1	201900000002	30	11	2019	02:50	3	93	93066	1	1	1	6	AUTOROUTE A1	48,9307000	2,3688000
2	201900000003	28	11	2019	15:15	1	92	92036	1	1	1	4	AUTOROUTE A86	48,9358718	2,3191744
3	201900000004	30	11	2019	20:20	5	94	94069	1	1	1	4	A4	48,8173295	2,4281502
4	201900000005	30	11	2019	04:00	3	94	94028	1	1	1	2	A86 INT	48,7763620	2,4332540
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
273221	202300054818	26	10	2023	20:45	5	974	97416	2	1	1	6	LA FONTAINE (RUE JEAN DE)	-21,33828000	55,47771000
273222	202300054819	26	10	2023	19:10	3	974	97416	1	1	1	3	RN3 (ANCIENNE ROUTE)	-21,28865000	55,50994000
273223	202300054820	26	10	2023	09:40	1	974	97411	2	1	1	7	BAMBOU (CHEMIN DE LA RUELE)	-20,90129000	55,40598000
273224	202300054821	26	10	2023	17:20	1	973	97302	1	6	1	3	ROUTE NATIONALE 1	4,89713000	-52,32854000
273225	202300054822	20	10	2023	16:30	1	69	69387	2	1	6	3	Boulevard Yves Farge	45,73306000	4,82540000

273226 rows × 15 columns

La dernière étape de la construction du Dataframe analytique consistait à intégrer les informations générales sur les circonstances de l'accident, contenues dans la table Caractéristiques, et à finaliser la structure des données.

● Fusion avec les données de Lieux

Dans un premier temps, le Dataframe `df_lieux_final` (contenant les informations traitées sur le lieu de l'accident, avec une ligne par `Num_Acc`) a été fusionné avec la table Caractéristiques brute. Cette jointure, réalisée sur la clé commune `Num_Acc`, a permis d'associer les caractéristiques générales de l'accident (comme les conditions atmosphériques, la luminosité - bien que `lum` soit supprimée plus tard, le type de collision, etc.) aux informations géographiques et routières déjà traitées.

- **Fusion finale avec les données Usagers et Véhicules**

Ensuite, le Dataframe résultant de la fusion Lieux/Caractéristiques a été à son tour fusionné avec le Dataframe df_merged_usag_veh_final (qui contenait les informations agrégées des usagers et des véhicules, également à raison d'une ligne par Num_Acc). Cette fusion finale a consolidé l'ensemble des informations prétraitées issues des quatre tables sources en une unique structure de données, où chaque ligne représente exhaustivement un accident.

- **Amélioration de la lisibilité par renommage de certaines variables**

Pour une meilleure compréhension et interprétation des variables dans le Dataframe final, certaines colonnes issues de l'encodage des catégories de véhicules ont été renommées avec des libellés plus explicites. Par exemple :

- new_catv_1 est devenue VL_VU
- new_catv_2 est devenue 2roues_3roues
- new_catv_3 est devenue PL
- new_catv_4 est devenue bus_car
- new_catv_5 est devenue velo_tro (pour vélos, trottinettes/EDP)

- **Définition de la variable cible finale (gravité_accident)**

La variable clé pour la modélisation, représentant la gravité globale de l'accident, a été créée à ce stade. Conformément à la stratégie définie, la gravité retenue pour un accident correspond au niveau de gravité le plus élevé constaté parmi tous les usagers impliqués. Les niveaux ont été codifiés comme suit:

- 4 : Accident Mortel (au moins un usager tué)
- 3 : Accident Blessé(s) Hospitalisé(s) (au moins un usager hospitalisé, aucun tué)
- 2 : Accident Blessé(s) Léger(s) (au moins un usager blessé léger, aucun hospitalisé ou tué)

Rappel : Le niveau 1 'indemne' n'est pas utilisé pour qualifier la gravité globale de l'accident, car la base ne recense que les accidents ayant fait au moins une victime nécessitant des soins.

- **Nettoyage final par suppression de colonnes**

Une dernière étape de nettoyage a consisté à supprimer les colonnes jugées :

Inutiles pour l'étude : *adr* (adresse postale), *com* (code commune), *id_vehicule* (devenue inutile après agrégation par accident), *lum* (luminosité, potentiellement jugée moins pertinente ou redondante avec d'autres facteurs).

Redondantes : Les indicateurs *catu_conducteur*, *catu_passager*, *catu_pieton* faisaient double emploi avec les informations déjà capturées via la variable *place* retraitée.

Non Informatives : Les colonnes correspondant aux modalités "non renseigné" pour le sexe, ou aux informations non déterminées/non renseignées pour la sécurité, ainsi que les lignes où la gravité individuelle était non renseignée, ont été retirées car elles n'apportaient pas d'information exploitable pour la modélisation.

- **Exclusion des outliers d'âge**

Conformément à l'analyse effectuée lors du traitement de la table Usagers, les enregistrements (accidents) pour lesquels des âges d'usagers supérieurs à 96 ans avaient été identifiés comme outliers ont été exclus du jeu de données final.

Résultat : Dataframe df_total_final

	Num_Acc	jour	mois	an	hrmn	dep	agg	int	atm	col	...	choc_AR	choc_cote	choc_tonneaux	VL_VU	2roues_3roues_quad	PL	bus_car	velo_trott_edp	nbr_veh	gravité_accident	
0	201900000001	30	11	2019	01:30	93	1	1	1	2	...	1	0	0	1		0	0	0	1	2	2
1	201900000002	30	11	2019	02:50	93	1	1	1	6	...	0	0	0	1		0	0	0	0	1	2
2	201900000003	28	11	2019	15:15	92	1	1	1	4	...	2	0	0	3		0	0	0	0	3	2
3	201900000004	30	11	2019	20:20	94	1	1	1	4	...	2	0	0	3		0	0	0	0	3	2
4	201900000005	30	11	2019	04:00	94	1	1	1	2	...	2	0	0	2		0	0	0	0	2	2
...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...	...
273221	202300054818	26	10	2023	20:45	974	2	1	1	6	...	0	1	0	1		0	0	0	0	1	2
273222	202300054819	26	10	2023	19:10	974	1	1	1	3	...	0	1	0	1		1	0	0	0	2	3
273223	202300054820	26	10	2023	09:40	974	2	1	1	7	...	1	0	0	1		0	0	0	0	1	2
273224	202300054821	26	10	2023	17:20	973	1	6	1	3	...	0	0	0	1		1	0	0	0	2	2
273225	202300054822	20	10	2023	16:30	69	2	1	6	3	...	0	1	0	1		0	0	0	1	2	3

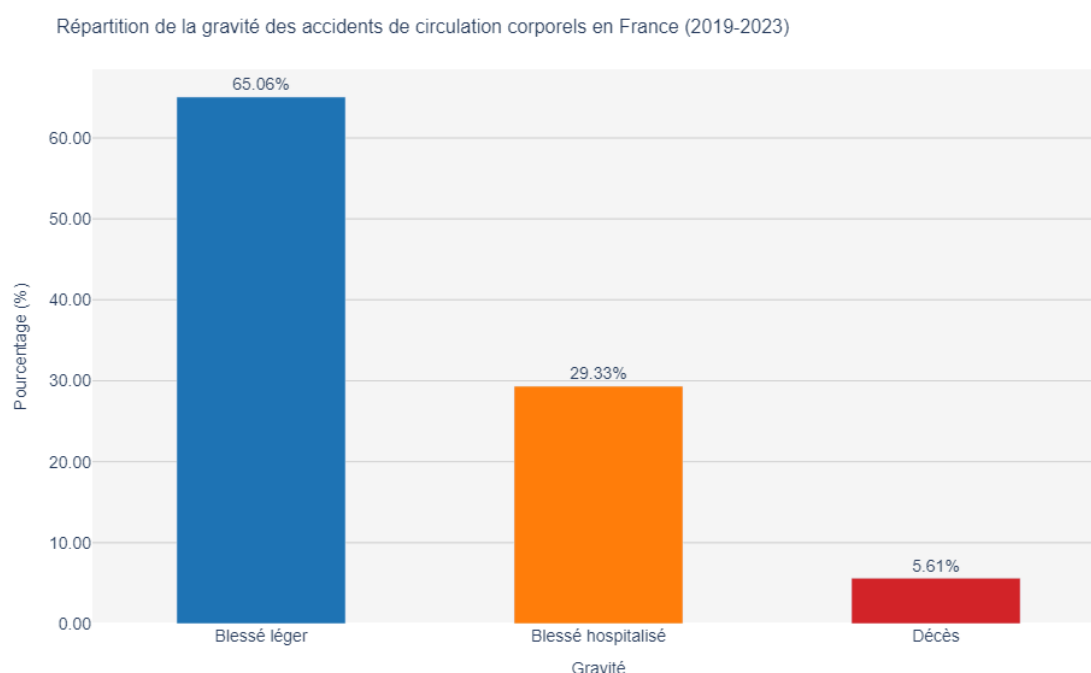
273226 rows × 60 columns

Au terme de l'ensemble de ce processus de traitement, de fusion, d'agrégation et de nettoyage, le Dataframe df_total_final a été obtenu et enregistré. Ce Dataframe constitue la

base de données analytique finale pour l'étude, avec une structure où chaque ligne correspond à un accident unique survenu entre 2019 et 2023, et les colonnes représentent les différentes caractéristiques de l'accident, des usagers et des véhicules impliqués, ainsi que la variable cible `gravité_accident`.

II. Analyse exploratoire de données

II.1. Distribution de la variable cible: gravité de l'accident



Nous observons un déséquilibre de la répartition des valeurs de la variable cible (gravité de l'accident), qu'il faudra garder en tête pour nos modèles de machine learning.

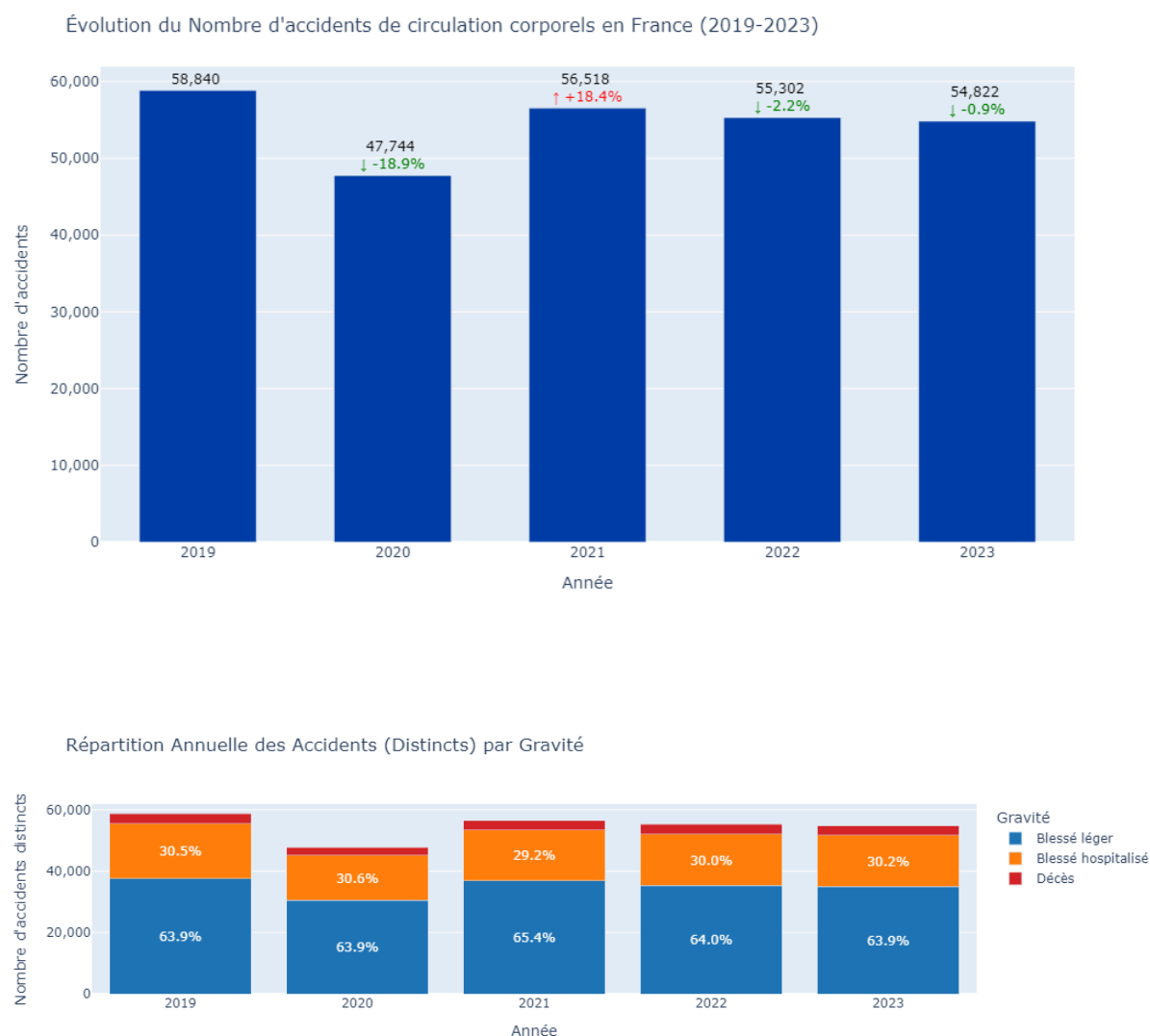
Nous confirmons également qu'il n'y a aucun accident avec la gravité 'indemne', ce qui signifie que pour chaque accident répertorié, il y a au moins un blessé.

II.2. Contexte temporel de l'accident(Caractéristiques)

L'analyse temporelle des accidents corporels en France entre 2019 et 2023 révèle des schémas récurrents et des points de vigilance spécifiques. Globalement, la répartition par gravité montre une nette prédominance des blessés légers (environ deux tiers des cas),

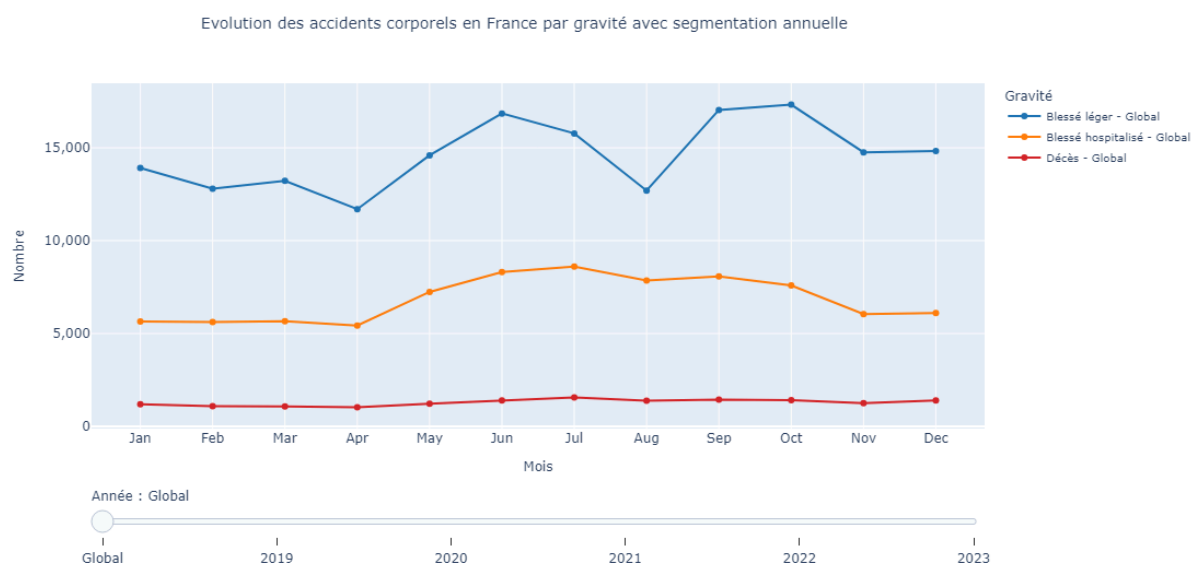
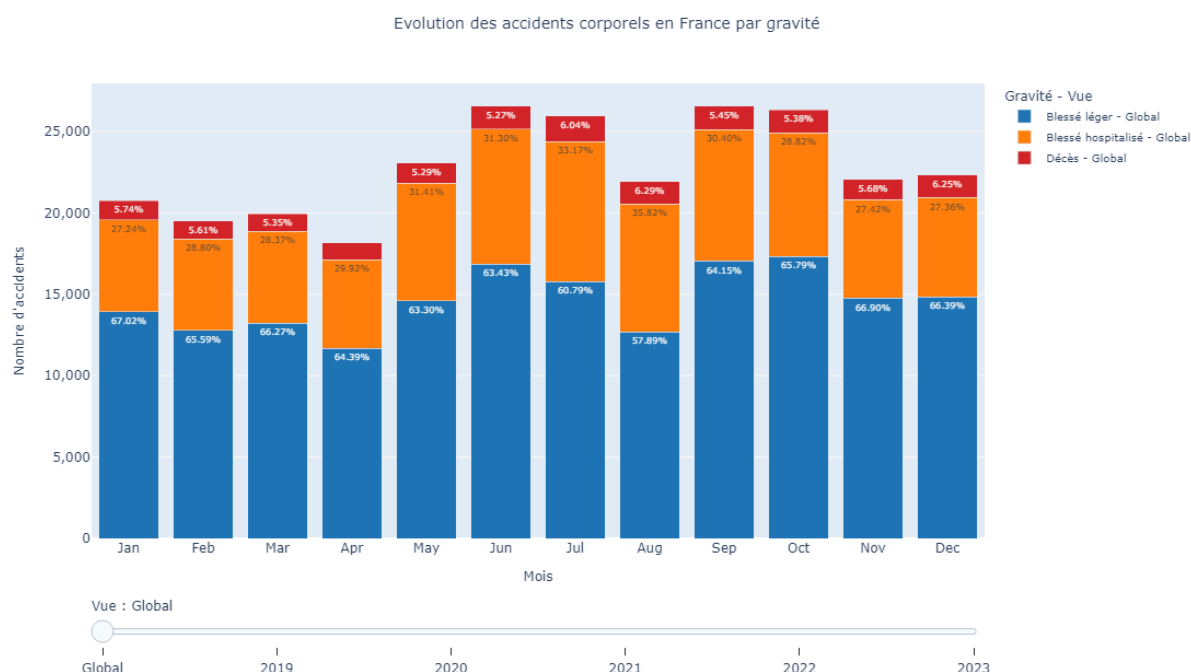
suivie des blessés hospitalisés (près d'un tiers), les accidents mortels représentant environ 5-6%. Si cette structure de gravité reste remarquablement stable sur la période, des variations temporelles significatives apparaissent à différentes échelles.

Tendances Annuelles :



L'année 2020 se distingue par une chute marquée du nombre total d'accidents (-18.9%), conséquence probable des restrictions liées à la pandémie de COVID-19. Les années suivantes montrent une reprise partielle puis une légère tendance à la baisse (-2.2% en 2022, -0.9% en 2023), sans toutefois retrouver le niveau de 2019. Un point notable est l'augmentation relative de la *proportion* d'accidents mortels en 2022 (atteignant 6.0%), suggérant une potentielle hausse de la sévérité cette année-là, avant un léger recul en 2023 (5.8%).

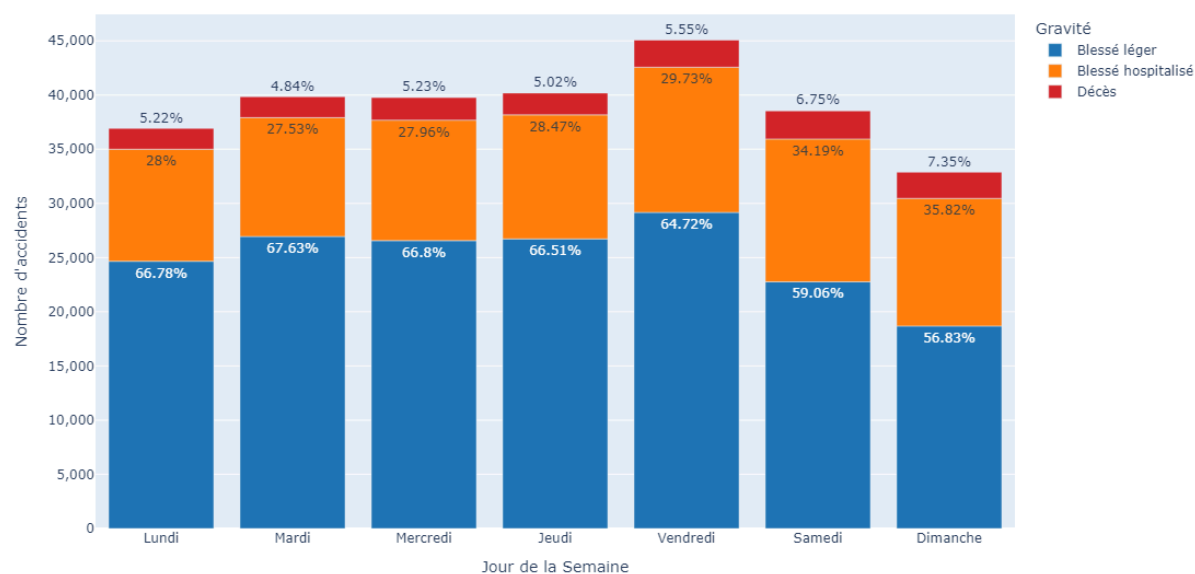
Variations Saisonnières Mensuelles



Une forte saisonnalité est observée, avec un pic d'accidents durant les mois d'été et d'automne (juin à octobre) et une diminution en hiver (notamment janvier et février). Cette tendance concerne toutes les catégories de gravité et coïncide vraisemblablement avec l'augmentation des déplacements durant les périodes de vacances et de climat plus favorable.

Rythme Hebdomadaire

Répartition Hebdomadaire des Accidents de Circulation par Gravité



La distribution hebdomadaire n'est pas uniforme, avec une concentration des accidents durant le week-end (vendredi, samedi, dimanche). Plus important encore, la *proportion* d'accidents graves (hospitalisations et décès) augmente significativement durant ces jours, particulièrement le dimanche où elle dépasse 43%, indiquant une dangerosité accrue en fin de semaine.

Fluctuations horaires par jour de la semaine



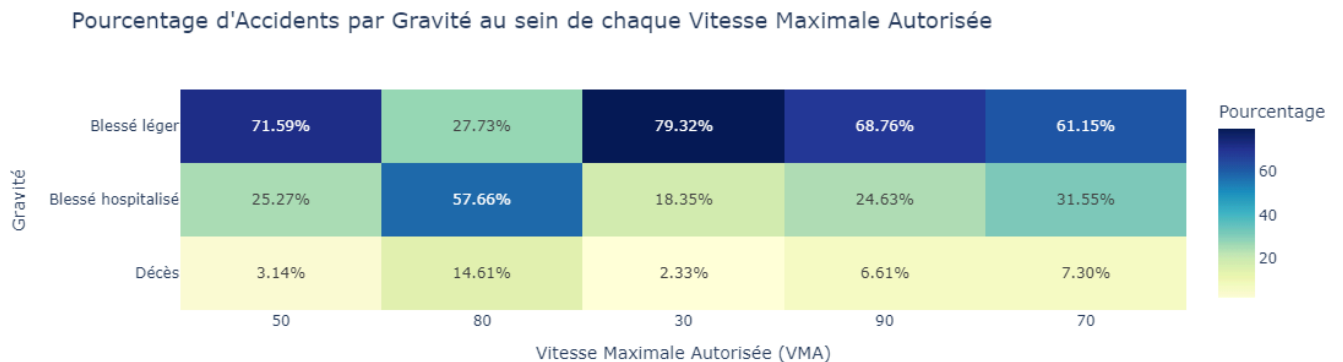
Deux pics d'accidents très nets se dessinent quotidiennement, correspondant aux heures de pointe : le matin entre 7h et 9h, et surtout le soir entre 17h et 19h. Ces créneaux, liés aux trajets domicile-travail, concentrent non seulement un volume élevé d'accidents mais voient également la gravité augmenter, la fin de journée étant particulièrement critique.

Conclusion partielle

L'analyse temporelle de l'accidentologie routière en France entre 2019 et 2023 met en évidence des fenêtres de risque clairement identifiées. Au-delà de l'impact conjoncturel de la pandémie sur le volume global en 2020, des schémas structurels persistent : une saisonnalité marquée avec un pic estival/automnal, une dangerosité accrue le week-end (tant en fréquence qu'en gravité relative), et une concentration critique des accidents aux heures de pointe quotidiennes (matin et surtout soir). Ces constats temporels soulignent que le risque routier n'est pas constant et que les efforts de prévention, de sensibilisation et de contrôle gagneraient à être intensifiés durant ces périodes spécifiques (été, week-ends, heures de pointe) pour réduire efficacement l'insécurité routière.

II.3. Facteurs géographiques et infrastructurels (Lieux)

II.3.1. Vitesse maximale autorisée et gravité de l'accident



Ce graphique révèle des disparités critiques dans la gravité des accidents selon la Vitesse Maximale Autorisée (VMA), offrant des leviers d'action clairs pour la sécurité routière.

- **Criticité exceptionnelle des zones à 80 km/h**

L'analyse montre une criticité marquée pour les accidents dans les zones à 80 km/h. Celles-ci présentent la proportion la plus élevée d'accidents aux conséquences graves ou fatales : plus de 72 % entraînent une hospitalisation (57.66 %) ou un décès (14.61 %). Inversement, la part des blessés légers y est la plus faible (27.73 %). Les zones à 80 km/h doivent être la priorité absolue pour les actions visant à réduire la gravité des accidents. Cibler ces zones est le moyen le plus efficace de diminuer substantiellement les décès et blessures graves.

- **Enjeux des zones à faible vitesse (30 et 50 km/h)**

Une dynamique différente concerne les zones à 30 et 50 km/h. Les accidents y sont moins graves, concentrant la plus forte proportion de blessés légers (72 %-79 %), avec une très faible part de décès (2 %-3 %). Bien que la sévérité individuelle soit moindre, le volume absolu de ces accidents pourrait être élevé. Les efforts doivent prioriser la prévention primaire (éviter l'accident) plutôt que la réduction d'une gravité déjà faible. Ceci est d'autant plus pertinent vu la présence fréquente d'utilisateurs vulnérables (piétons, cyclistes).

- **Risque intermédiaire mais non négligeable des zones à 70 et 90 km/h**

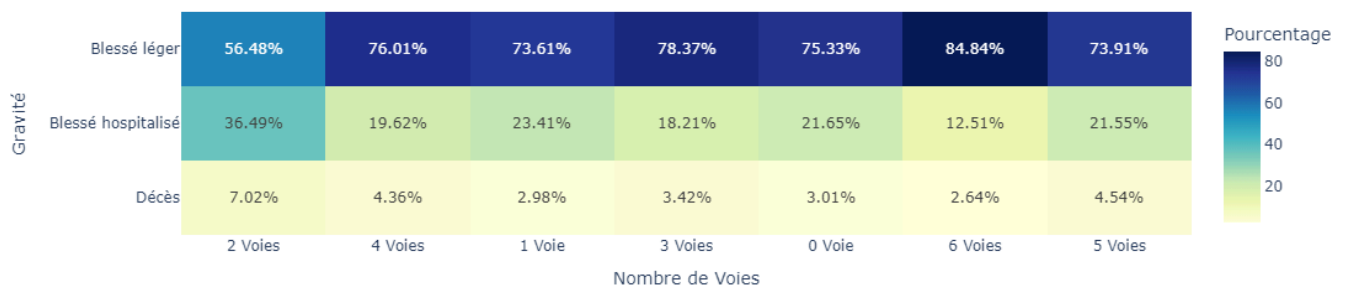
Les zones à 70 et 90 km/h présentent un risque de gravité intermédiaire mais réel. La proportion de décès (6 %-7 %) et d'hospitalisations (25 %-32 %) y est significativement plus élevée qu'en zones 30/50, mais inférieure à celle des zones 80. Les mesures classiques (contrôles de vitesse, améliorations infrastructurelles) restent pertinentes, mais constituent une priorité secondaire par rapport aux zones 80.

Conclusion partielle

L'analyse de la gravité par VMA impose une focalisation prioritaire sur les zones à 80 km/h, qui affichent une proportion alarmante d'accidents graves et mortels. Elles nécessitent un audit et des actions ciblées (contrôles renforcés, réévaluation VMA, infrastructure). Une stratégie différenciée s'impose : pour les zones 30/50, réduire la fréquence des accidents (sécurisation usagers vulnérables, modération trafic) ; pour les zones 70/90, maintenir les mesures classiques face à un risque intermédiaire. La communication doit aussi être adaptée : insister sur le risque vital à VMA élevée (70/80/90), sur la prévention générale à faible VMA. En résumé, l'allocation des ressources doit prioriser les interventions sur les tronçons à 80 km/h pour l'impact le plus significatif sur la réduction des décès et blessures graves.

II.3.2. Nombre de voies et gravité de l'accident

Pourcentage d'Accidents par Gravité selon le Nombre de Voies



- **Danger maximal sur les routes à 2 voies**

Le constat le plus alarmant de l'analyse concerne les routes à 2 voies. Celles-ci présentent, de manière très marquée, la proportion la plus élevée d'accidents aux conséquences graves ou mortelles. Le taux de décès y atteint un record de 7.02 %, ce qui est 1.5 à près de 3 fois supérieur aux autres types de routes. De plus, la proportion de blessés nécessitant une hospitalisation y est écrasante, s'élevant à 36.49 %, soit presque le double de la plupart des autres configurations. En combinant ces deux issues, plus de 43 % des accidents survenant sur les routes à 2 voies entraînent une hospitalisation ou un décès, constituant un signal d'alarme majeur quant à la dangerosité spécifique de ces infrastructures.

- **Moindre Gravité sur les Routes Multi-Voies**

À l'inverse de la situation critique des routes à 2 voies, l'analyse montre que l'augmentation du nombre de voies s'accompagne d'une diminution de la proportion d'accidents graves. Les routes à 6 voies illustrent parfaitement cette tendance, affichant le profil de gravité le moins sévère : elles enregistrent le taux de décès le plus bas (2.64 %), le taux d'hospitalisation le plus faible (12.51 %), et corrélativement la proportion la plus élevée de blessés légers (84.84 %). Les routes comportant 3, 4 ou 5 voies présentent des profils de gravité similaires, caractérisés par une large majorité de blessés légers (entre 74 % et 78 %) et des taux de décès et d'hospitalisation nettement inférieurs à ceux observés sur les routes à 2 voies.

- **Cas Particuliers : 1 Voie et 0 Voie**

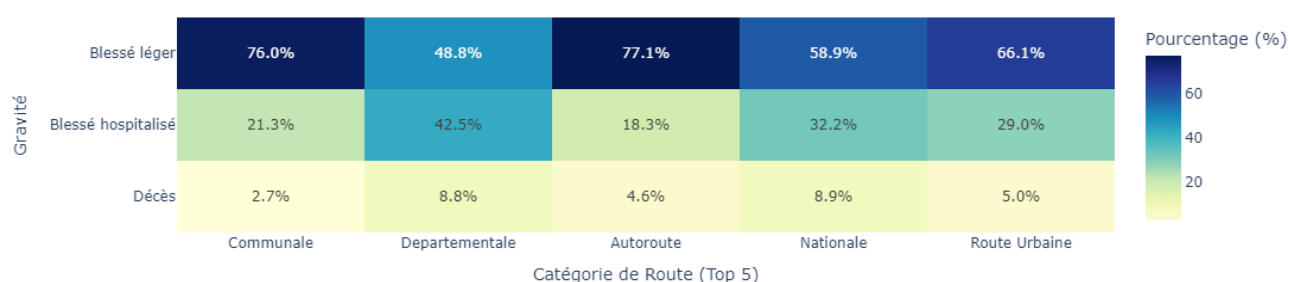
Enfin, deux cas méritent une mention particulière. Les routes à 1 voie se distinguent par un taux d'hospitalisation relativement élevé (23.41 %), qui contraste avec un faible taux de décès (2.98 %). La catégorie "0 voie", dont la nature exacte reste ambiguë (pouvant correspondre à des zones hors route, des parkings, ou potentiellement une classification erronée), semble néanmoins associée à un niveau de gravité moindre que celui constaté sur les routes à 2 voies, avec des taux de décès et d'hospitalisation modérés (respectivement 3.01 % et 21.65 %).

Conclusion partielle

L'analyse révèle une corrélation inverse très nette entre le nombre de voies et la gravité proportionnelle des accidents : les routes à 2 voies se distinguent comme les infrastructures les plus dangereuses, concentrant de loin les plus fortes proportions de décès (7.02 %) et d'hospitalisations (36.49 %), avec plus de 43 % d'accidents aux conséquences graves ou fatales. À l'inverse, les routes multivoies (3 à 6 voies) présentent une gravité significativement moindre, la proportion d'accidents sévères diminuant à mesure que le nombre de voies augmente, les routes à 6 voies étant les moins dangereuses proportionnellement. Les cas des routes à 1 voie (hospitalisations élevées, décès faibles) et "0 voie" (gravité modérée) complètent ce tableau, soulignant l'urgence d'interventions ciblées sur les routes à 2 voies pour réduire les conséquences les plus dramatiques des accidents routiers.

II.3.3. Catégorie de la route et gravité de l'accident

Répartition (en %) de la Gravité des Accidents pour le Top 5 des Catégories de Route



- **Zones de danger critique - Routes départementales et nationales**

L'analyse révèle que les routes Départementales et Nationales constituent des zones de danger critique où les accidents entraînent les conséquences les plus sévères. Les Départementales émergent particulièrement pour la gravité des blessures, affichant de très loin le taux d'hospitalisation le plus élevé (42.5 %), soit presque le double de celui des routes Urbaines ou Nationales, associé à un taux de décès également très préoccupant (8.8 %). Ainsi, près de la moitié des accidents survenant sur les routes départementales ont des issues graves (décès ou hospitalisation), ce qui est particulièrement alarmant.

Les routes Nationales, quant à elles, détiennent le triste record du taux de décès le plus élevé (8.9 %). Ce chiffre, combiné à un taux d'hospitalisation important (32.2 %), confirme leur statut de réseau à haut risque où les accidents ont fréquemment des conséquences dramatiques.

- **Gravité moindre - Autoroutes et Routes Communales**

À l'opposé des routes départementales et nationales, les autoroutes et les routes communales présentent un profil de gravité moindre. De manière contre-intuitive, et malgré les vitesses potentiellement élevées, les Autoroutes affichent la proportion la plus faible de blessés hospitalisés (18.3 %) et, corrélativement, la plus haute proportion de blessés légers (77.1 %). Leur taux de décès (4.6 %) reste modéré, indiquant que les accidents y sont *proportionnellement* moins graves que sur d'autres réseaux.

Les Routes Communales se caractérisent par le niveau de gravité le plus faible observé dans cette analyse, avec le taux de décès le plus bas (2.7 %) et une forte prédominance de blessés légers (76.0 %), suggérant des impacts moins violents.

- **Profil intermédiaire - Routes Urbaines**

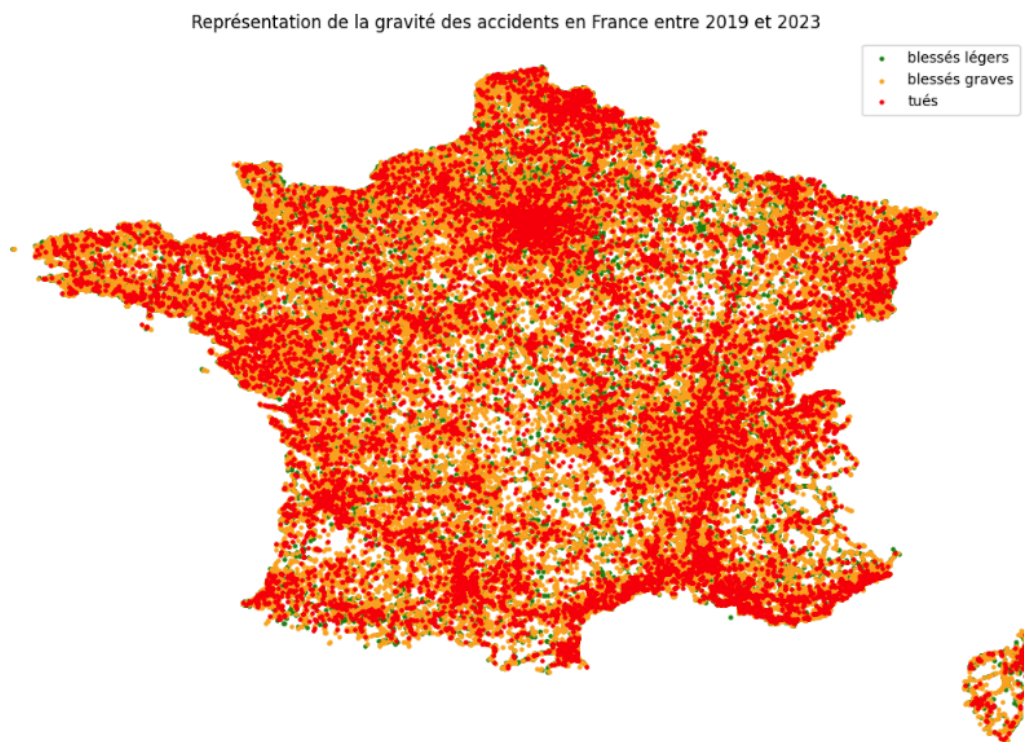
Enfin, les Routes Urbaines se situent dans une position médiane en termes de gravité des accidents. Leurs taux de décès (5.0 %) et d'hospitalisation (29.0 %) sont significatifs et témoignent d'un risque réel, bien qu'ils restent inférieurs aux niveaux observés sur les routes Départementales et Nationales. La majorité des accidents enregistrés sur les routes urbaines (66.1 %) résultent en blessures légères, plaçant ce type de route dans une catégorie de risque intermédiaire par rapport aux autres réseaux analysés.

Conclusion partielle

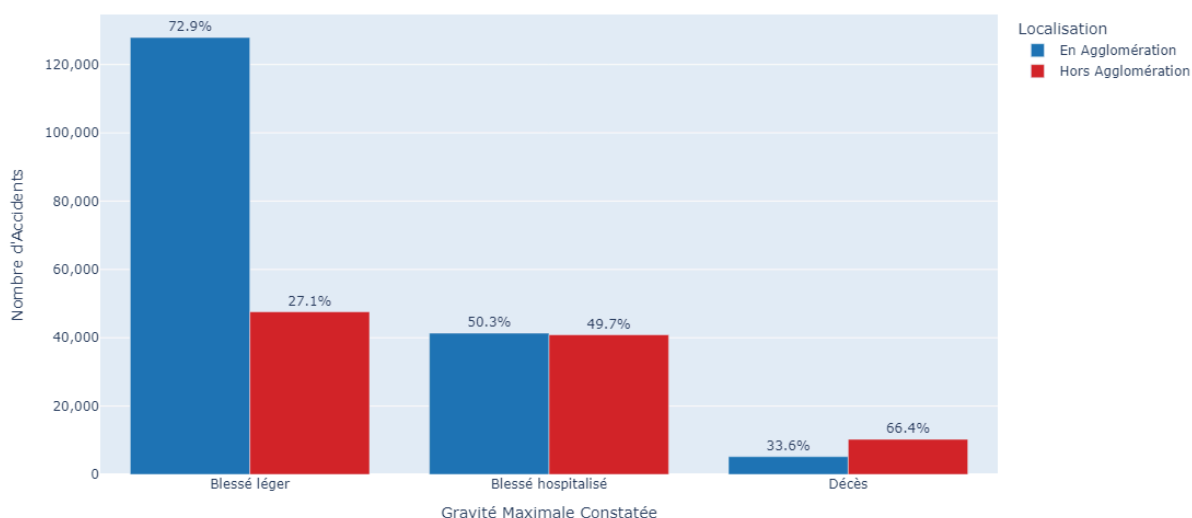
L'analyse de la gravité des accidents par catégorie de route met en évidence des disparités majeures : les routes Départementales et Nationales émergent comme les réseaux les plus dangereux, concentrant les taux les plus élevés d'hospitalisation (42.5 % sur Départementales) et de décès (8.9 % sur Nationales), indiquant des conséquences

particulièrement sévères lors d'accidents. À l'opposé, les Autoroutes et les Routes Communales présentent une gravité proportionnellement moindre, avec les taux de décès et d'hospitalisation les plus bas, malgré les vitesses élevées possibles sur autoroute. Les Routes Urbaines affichent un profil de risque intermédiaire. Cette hiérarchie souligne la nécessité de cibler prioritairement les routes Départementales et Nationales pour réduire les accidents graves et mortels.

II.3.4. Localisation de l'accident



Répartition des Accidents par Gravité et Localisation



- **Vue d'ensemble géographique**

On observe une **forte concentration d'accidents dans les zones densément peuplées**, notamment en Île-de-France, autour de Lyon, Marseille, Lille ou encore Toulouse. Cette densité est cohérente avec l'intensité du trafic routier dans les agglomérations.

Cependant, il est important de noter que des points rouges et orange sont également bien présents dans des **zones plus rurales**, témoignant d'accidents graves ou mortels sur des routes moins fréquentées mais potentiellement plus dangereuses (vitesse, visibilité, éloignement des secours...).

- **Gravité des accidents selon la localisation (agglomération / hors agglomération)**

Ce graphique met en lumière une dichotomie frappante du risque routier entre les zones urbaines (en agglomération) et rurales/périurbaines (hors agglomération) :

- Blessures Légères : Le Quotidien des Villes

Les accidents en agglomération représentent une écrasante majorité (72.9%) des blessures légères. Cela confirme que les environnements urbains, avec leur densité de trafic et leurs interactions complexes, génèrent un volume très élevé d'accrochages, heureusement le plus souvent sans conséquences graves. Les zones hors agglomération comptent pour seulement 27.1% de ces incidents bénins.

- Blessures avec Hospitalisation : Un Risque Partagé.

La probabilité qu'un accident entraîne une hospitalisation est remarquablement similaire, que l'accident survienne en agglomération (50.3%) ou hors agglomération (49.7%). Malgré des vitesses généralement plus faibles en ville, des facteurs comme la vulnérabilité des usagers (piétons, cyclistes) ou des types de chocs spécifiques peuvent expliquer ce quasi-équilibre.

- Décès : Le lourd tribut des routes hors agglomération

La tendance s'inverse tragiquement pour les accidents mortels. Les routes hors agglomération sont le théâtre de deux tiers (66.4%) des décès, contre seulement un tiers (33.6%) en ville. Cette surreprésentation dramatique est directement liée aux vitesses plus élevées pratiquées hors agglomération, qui transforment les accidents en événements aux conséquences souvent fatales.

Conclusion partielle

Cette analyse impose une stratégie de sécurité routière différenciée :

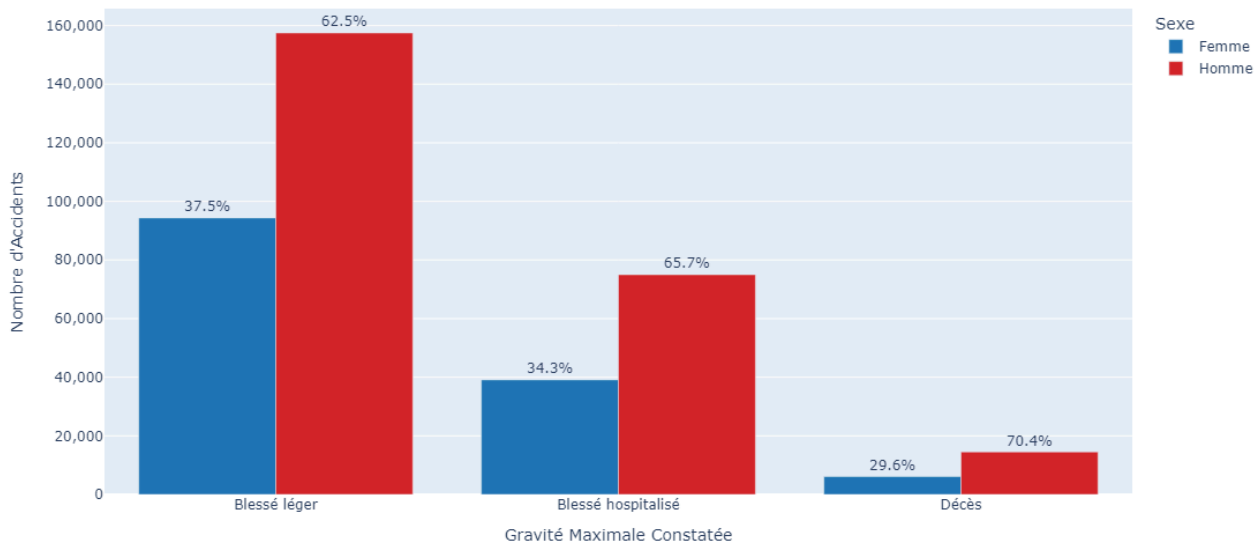
En agglomération : L'objectif prioritaire est la réduction du nombre total d'accidents. Les politiques doivent se concentrer sur l'apaisement de la circulation (zones 30, limitations de vitesse), la sécurisation des carrefours, la protection des usagers vulnérables (aménagements cyclables, cheminements piétons sécurisés), et la lutte contre l'inattention. Même bénins, ces accidents congestionnent, coûtent cher et peuvent générer de l'insécurité.

Hors agglomération : L'urgence est de réduire la gravité des accidents qui s'y produisent, et en premier lieu, d'éviter les accidents mortels. Les leviers d'action incluent le contrôle renforcé et la gestion de la vitesse (radars, aménagements modérateurs aux points dangereux), l'amélioration de l'infrastructure (routes qui pardonnent, suppression d'obstacles latéraux), la lutte contre l'alcoolémie et la fatigue au volant, et potentiellement l'optimisation des délais d'intervention des secours.

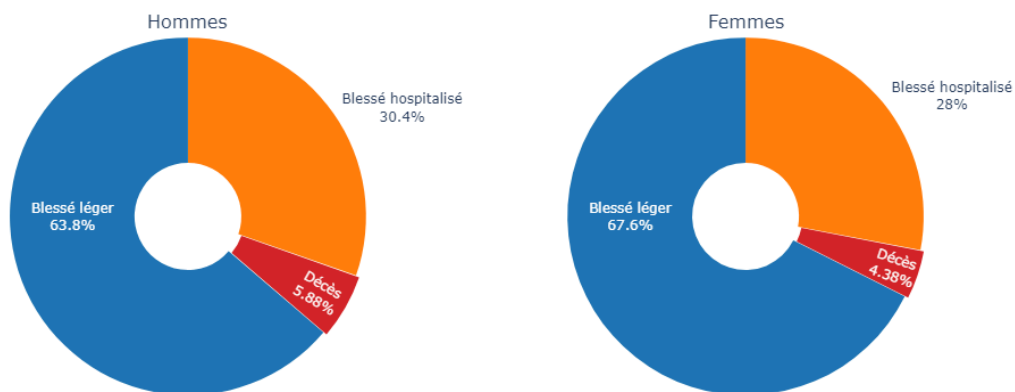
II.4. Profil et statut des usagers impliqués (usagers)

II.4.1. Répartition des accidents par sexe et gravité

Répartition des Accidents par Gravité et Sexe



Répartition Interne de la Gravité par Sexe



L'analyse des accidents par Sexe et Niveau de gravité révèle une disparité marquée au détriment des hommes, nettement surreprésentés tant en volume global qu'en gravité. Non seulement ils sont impliqués dans plus d'accidents à chaque niveau de sévérité, mais leur proportion augmente avec celle-ci, culminant à 70.4% des décès. Au-delà de ces chiffres, leur risque intrinsèque est aussi plus élevé : un homme accidenté a une probabilité

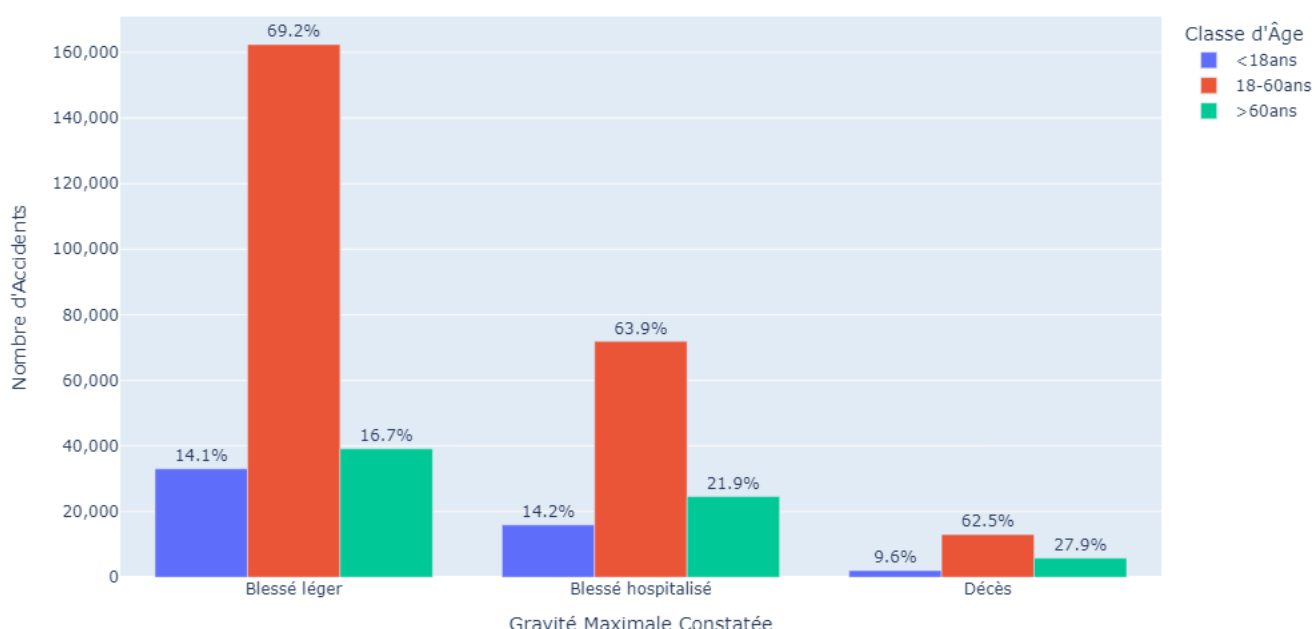
supérieure d'être hospitalisé (30.4% vs 28.0%) ou tué (5.88% vs 4.36%) qu'une femme dans la même situation. Cette double dimension – participation accrue aux accidents et risque relatif de conséquences graves plus important – souligne un enjeu majeur de sécurité routière pour la population masculine, nécessitant des stratégies de prévention ciblées.

Conclusion partielle

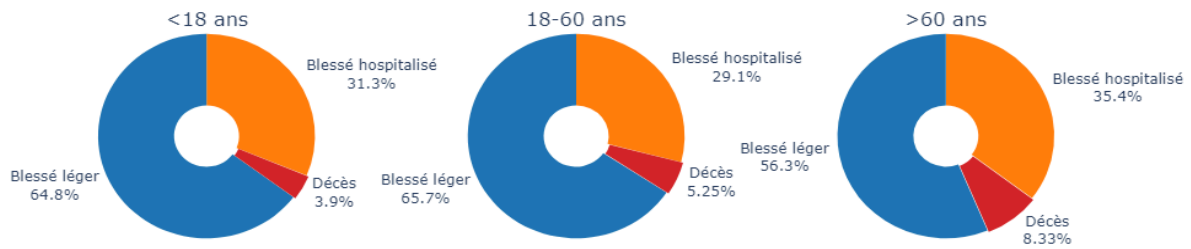
L'analyse révèle un enjeu majeur de sécurité routière spécifique à la population masculine. Les hommes sont non seulement surreprésentés dans les accidents de la route à tous les niveaux de gravité, mais cette disproportion s'accroît avec la sévérité, atteignant 70.4% des décès. De plus, même lorsqu'ils sont impliqués, les hommes ont un risque intrinsèque plus élevé que les femmes d'être hospitalisés (30.4% vs 28.0%) ou tués (5.88% vs 4.36%). Cette double dimension – participation accrue aux accidents et vulnérabilité supérieure aux conséquences graves – impose la nécessité de développer des stratégies de prévention spécifiquement ciblées sur les hommes.

II.4.2. Répartition des accidents par âge et gravité

Répartition des Accidents par Gravité et Classe d'Âge



Répartition Interne de la Gravité par Classe d'Âge



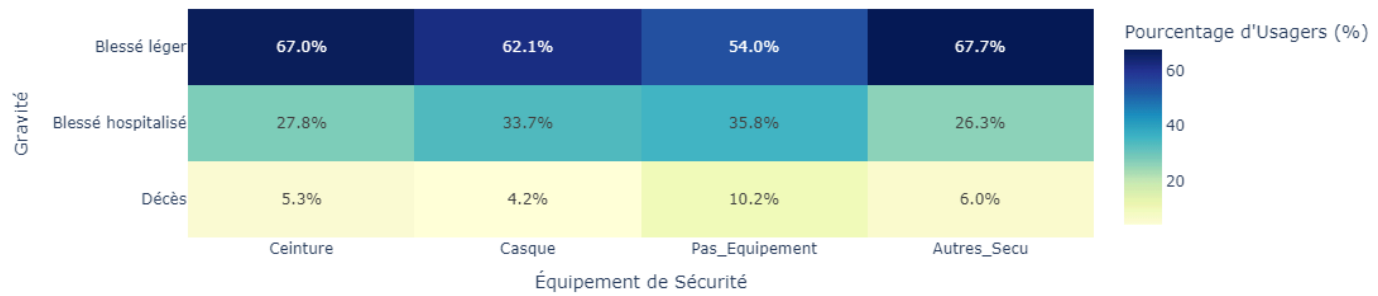
L'analyse révèle une double réalité claire du risque routier lié à l'âge : d'une part, les 18-60 ans dominent massivement le volume des accidents (représentant 60-70% des victimes). D'autre part, les seniors (>60 ans) affichent une vulnérabilité critique : ils sont surreprésentés parmi les victimes les plus graves (constituant 27.9% des décès) et, surtout, leur risque intrinsèque de décéder en cas d'accident est nettement plus élevé (8.33%). Cette distinction fondamentale entre volume et vulnérabilité impose des stratégies de prévention différenciées.

Conclusion partielle

En résumé, les implications pour les politiques publiques sont claires : il faut une approche différenciée selon l'âge. Pour la tranche majoritaire des 18-60 ans, l'accent doit rester sur la réduction du volume global d'accidents par des mesures générales de prévention et de contrôle. Pour les seniors (>60 ans), la priorité est de réduire la gravité des accidents en reconnaissant leur vulnérabilité accrue. Cela passe par des infrastructures plus sûres, la promotion de véhicules adaptés, des campagnes de sensibilisation et d'accompagnement spécifiques, et une planification à long terme qui anticipe le vieillissement de la population.

II.4.3. Répartition des accidents par système de sécurité et gravité

Répartition (en %) de la Gravité par Équipement de Sécurité (selon les Usagers)



L'absence d'équipement de sécurité multiplie le risque de conséquences dramatiques. Le constat le plus alarmant de ce graphique est la dangerosité extrême associée au non-port d'équipement de sécurité ('Pas_Equipement'). Dans cette catégorie, le taux de décès (10.2 %) est presque double de celui observé chez les usagers portant une ceinture (5.3 %) et près de 2.5 fois supérieur à celui des porteurs de casque (4.2 %). De plus, la proportion de blessés hospitalisés (35.8 %) y est également la plus élevée. Ainsi, près de la moitié (46%) des usagers sans équipement impliqués dans un accident subissent des conséquences graves (hospitalisation ou décès). À l'inverse, le port de la ceinture ou du casque démontre clairement son efficacité protectrice, réduisant significativement la proportion de décès et augmentant celle des blessures légères. Le casque se distingue par le taux de décès le plus faible (4.2 %), tandis que la ceinture est associée à la plus forte proportion de blessés légers (67.0 %) parmi les équipements principaux.

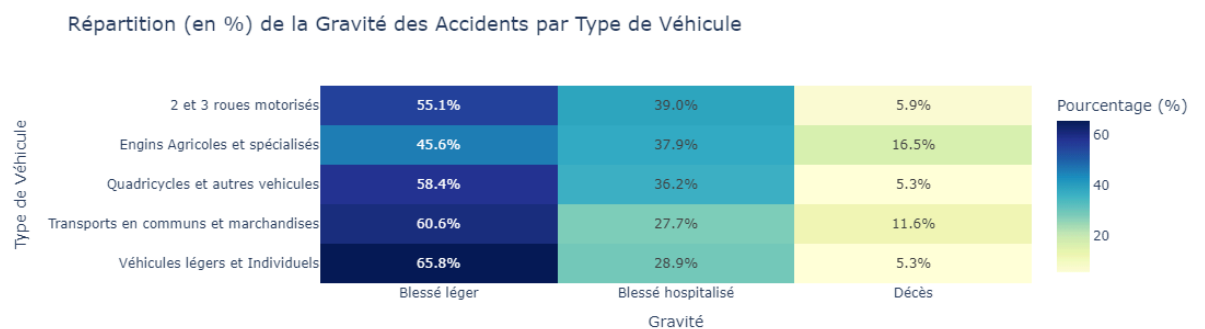
Conclusion partielle

En résumé, les politiques publiques de sécurité routière doivent prioritairement cibler le non-port des équipements de sécurité, responsable d'une augmentation dramatique du risque de décès et d'hospitalisation. Cela passe par un renforcement significatif des contrôles et sanctions, couplé à des campagnes de sensibilisation percutantes qui exposent crûment ces risques. Parallèlement, il faut continuer à valoriser l'efficacité protectrice prouvée de la ceinture et du casque. Une analyse plus fine de la catégorie "Autres Sécurités" est également nécessaire pour adapter les actions préventives. L'efficacité de ces

mesures repose sur une approche intégrée et continue, combinant répression, éducation et communication pour ancrer durablement les comportements sécuritaires.

II.5. Typologie et Points d'impact des véhicules impliqués (véhicules)

II.5.1. Catégorie de Véhicules et Gravité des accidents



Ce graphique met en lumière une réalité frappante : le type de véhicule impliqué dans un accident est un prédicteur significatif de la gravité des blessures subies. L'analyse révèle des profils de risque distincts qui exigent des stratégies de prévention ciblées.

- **Le paradoxe des deux roues motorisés : Blessures graves fréquentes, moins de décès relatifs**

Les conducteurs de 2 et 3 roues motorisés se distinguent par une vulnérabilité extrême aux blessures graves nécessitant une hospitalisation (39.0%). C'est la catégorie la plus à risque pour ce niveau de gravité. Bien que le taux de décès (5.9%) ne soit pas le plus élevé, la probabilité de finir à l'hôpital après un accident est alarmante pour ces usagers. Décision potentielle : Cela suggère un besoin urgent d'améliorer la protection passive (équipements) et active (formation, sensibilisation des autres usagers), ainsi que des infrastructures routières plus sûres pour cette catégorie.

- **Le danger mortel des engins spécifiques et lourds**

À l'opposé du spectre de gravité, les engins agricoles et spécialisés affichent le taux de décès le plus élevé (16.5%), et de loin. Les accidents impliquant ces véhicules sont disproportionnellement mortels. Les véhicules de transport en commun et marchandises

suivent avec un taux de décès également préoccupant (11.6%). Décision potentielle : La masse et la nature de ces véhicules rendent les collisions particulièrement dangereuses. Les efforts doivent se concentrer sur la prévention des accidents eux-mêmes (technologies anti-collision, régulation des heures de conduite, zones d'opération sécurisées) plutôt que sur la mitigation des conséquences post-accident pour les autres parties impliquées, car l'issue est souvent fatale.

- **Véhicules légers : Fréquence élevée, gravité moindre (en proportion)**

Les Véhicules légers et individuels, qui représentent probablement le volume le plus important sur les routes, montrent la plus forte proportion de blessés légers (65.8%) et un taux de décès relativement bas (5.3%). Bien que chaque accident soit moins susceptible d'être fatal en proportion, le nombre absolu de blessés graves ou de décès pourrait rester élevé en raison de la fréquence des accidents dans cette catégorie (ce que ce graphique seul ne montre pas). Décision potentielle : Les stratégies pour cette catégorie doivent viser la réduction globale du nombre d'accidents (sécurité routière générale, amélioration du comportement des conducteurs) et l'amélioration continue de la sécurité passive des véhicules pour limiter la gravité lorsque les accidents surviennent.

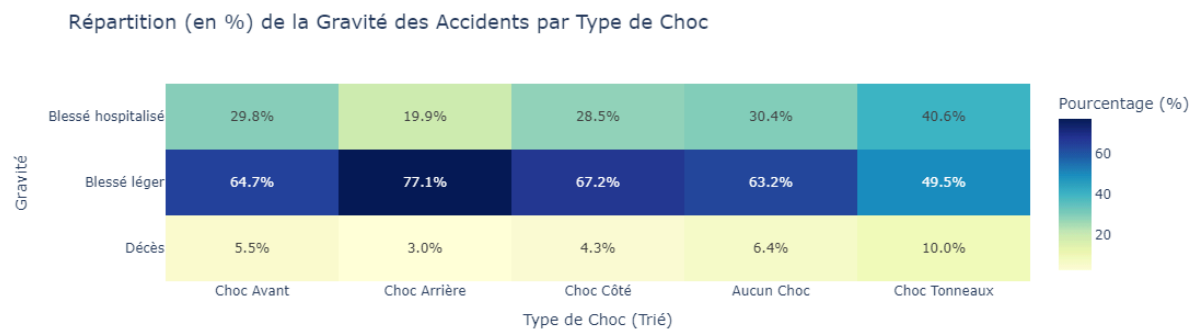
Conclusion partielle

L'analyse souligne que la sécurité routière ne peut être abordée de manière uniforme, tant les risques varient selon les types de véhicules. Il est donc indispensable de différencier les ressources et les actions : la protection des usagers les plus vulnérables, notamment les conducteurs de deux-roues, doit être une priorité pour réduire les hospitalisations.

Parallèlement, la prévention des collisions impliquant des véhicules lourds et spécialisés doit cibler la réduction de la mortalité. Enfin, il convient de poursuivre les efforts pour diminuer la fréquence des accidents et renforcer la sécurité passive des véhicules légers.

Cette segmentation fine des risques permet d'allouer les moyens de façon plus efficace et d'élaborer des politiques de sécurité routière et de prévention des accidents du travail plus adaptées et impactantes.

II.5.2. Type de choc et Gravité des accidents



L'analyse du graphique révèle une répartition contrastée de la gravité des accidents selon le type de choc. Les blessures légères prédominent largement dans tous les cas, avec des pourcentages particulièrement élevés pour les chocs arrière (77,1 %) et les chocs côtés (67,2 %), indiquant que ces types d'impact causent principalement des dommages moins graves.

En revanche, les chocs tonneaux se distinguent par une proportion significativement plus élevée de blessés hospitalisés (40,6 %) ainsi que le taux le plus important de décès (10,0 %). Ce constat met en lumière la dangerosité accrue des chocs impliquant des retournements. Par ailleurs, les chocs avant et latéraux engendrent également des taux substantiels de blessés hospitalisés, proches de 29 %, et des décès compris entre 4,3 % et 5,5 %, ce qui souligne leur impact sévère en termes de gravité.

Le cas « Aucun choc » présente une situation paradoxale avec un nombre relativement élevé de blessés hospitalisés (30,4 %) et un taux de décès de 6,4 %, ce qui pourrait s'expliquer par des mécanismes de blessure différents, tels que les chutes ou les sorties de route, nécessitant une investigation approfondie.

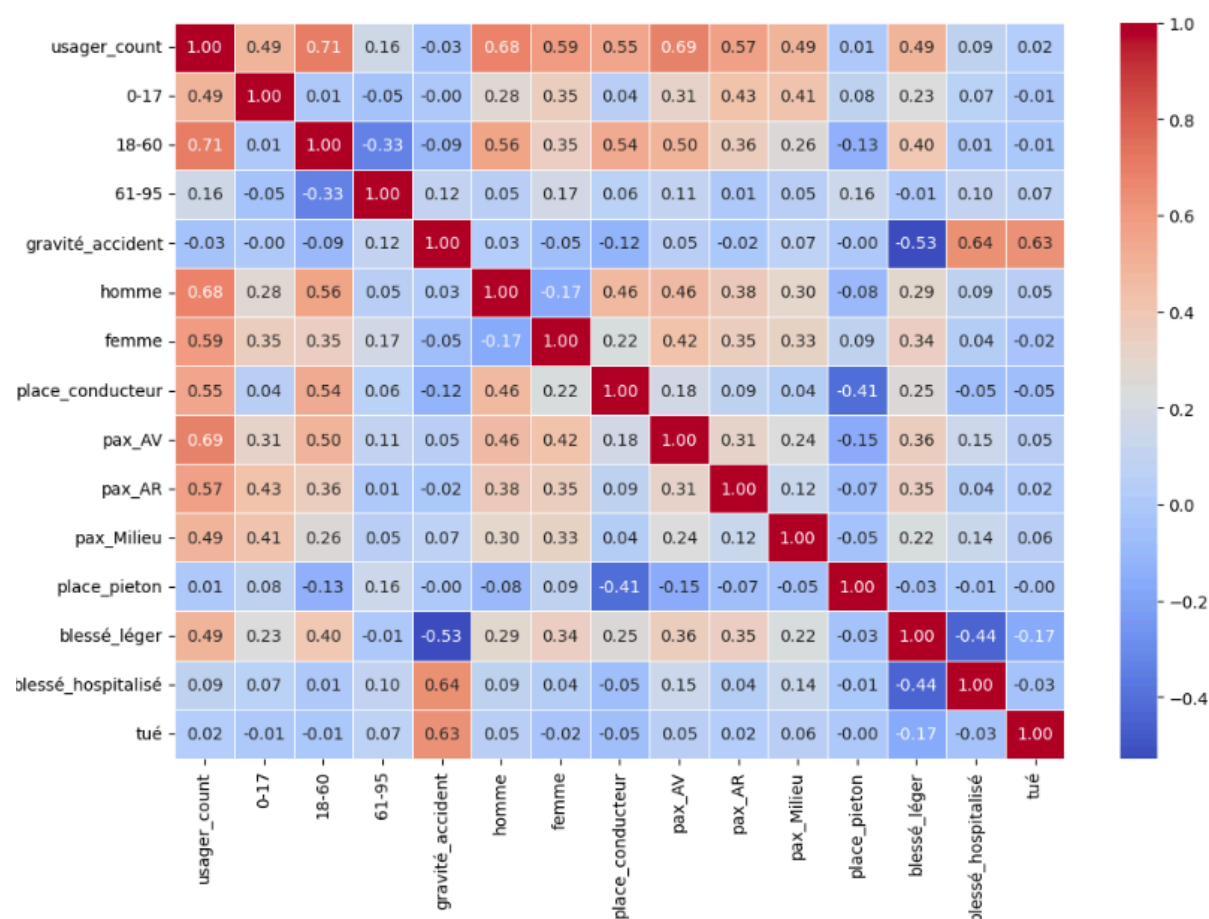
Conclusion partielle

Ces résultats montrent que la gravité des accidents varie fortement selon le type de choc, avec les chocs tonneaux qui requièrent une attention particulière en raison de leur forte mortalité et gravité. Les politiques de prévention doivent donc prioriser l'amélioration des dispositifs de sécurité visant à réduire les risques liés aux retournements, tout en maintenant des interventions ciblées sur les impacts avant et latéraux. Par ailleurs, il est essentiel d'examiner de près les situations sans choc apparent, afin d'élaborer des mesures

spécifiques adaptées aux mécanismes sous-jacents de blessure. Cette approche différenciée permettra de mieux protéger les usagers de la route et de réduire les conséquences les plus graves des accidents.

II.6. Matrices de corrélation

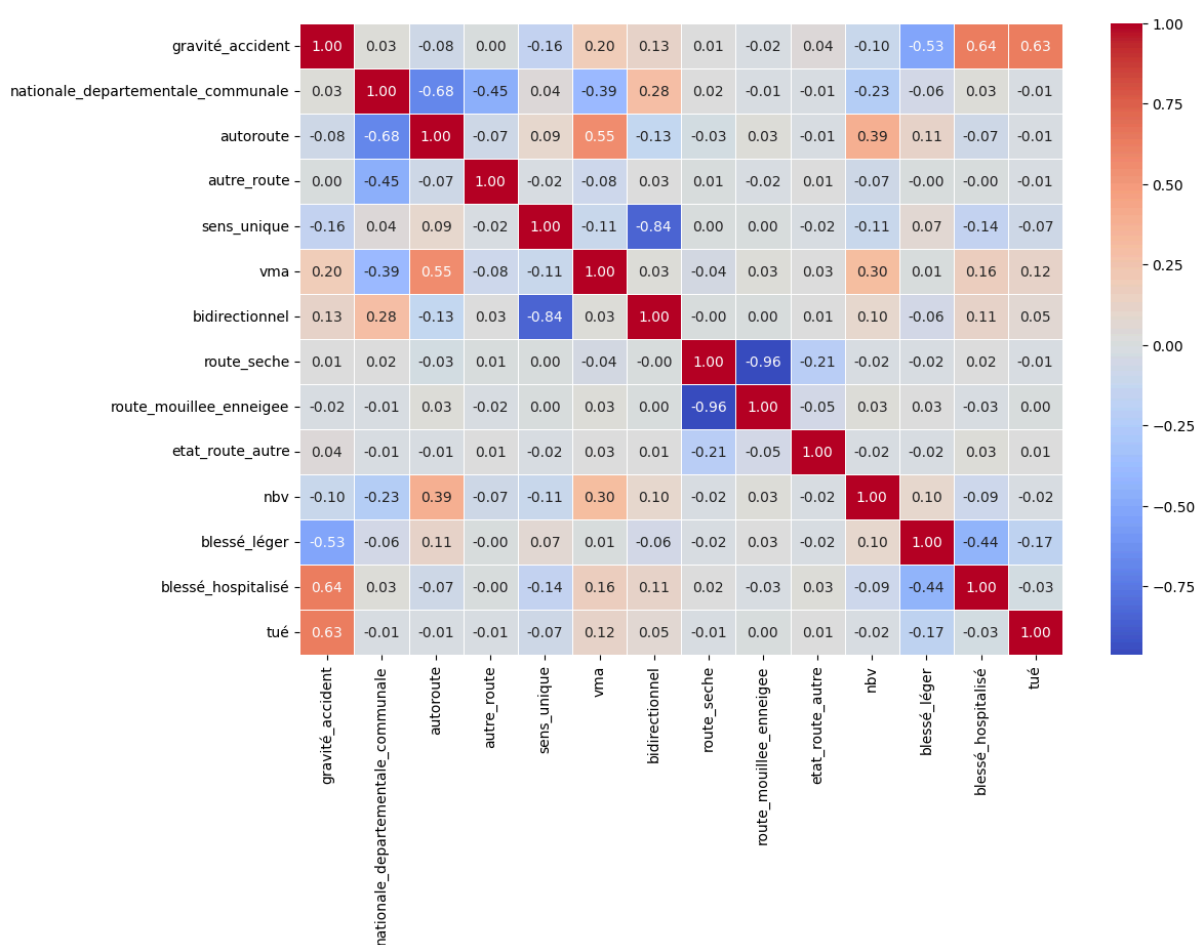
II.6.1. Corrélation selon les caractéristiques des usagers



L'examen des corrélations les plus intenses révèle des dynamiques clés dans les accidents. D'abord, la gravité objective de l'accident (gravité_accident) est indissociable des issues les plus graves : elle est fortement liée à la probabilité d'être tué (+0.63) ou hospitalisé (+0.64), tandis qu'elle exclut logiquement les blessures légères (-0.53). En pratique, cela confirme que la classification de gravité reflète directement le niveau de danger vital. Ensuite, le nombre d'usagers impliqués (usager_count) est fortement corrélé à la présence d'adultes (18-60 ans : +0.71), d'hommes (+0.68) et de passagers avant (pax_AV : +0.69), suggérant que les accidents impliquant plusieurs personnes concernent souvent ce profil

démographique dans les places avant. Par ailleurs, la forte corrélation négative (-0.41) entre être conducteur (place conducteur) et être piéton (place_pieton) illustre simplement des contextes d'implication mutuellement exclusifs. Enfin, la corrélation négative notable (-0.44) entre blessé léger et blessé hospitalisé renforce l'idée que ces issues appartiennent à des classes de sévérité distinctes et s'excluent mutuellement au niveau individuel dans un même accident. Ces liens forts dessinent un tableau où les accidents les plus graves impliquent fréquemment des adultes (notamment hommes) aux places avant, et où la classification de gravité prédit fidèlement le risque d'issues sévères plutôt que légères.

II.6.2. Corrélations selon les conditions de routes



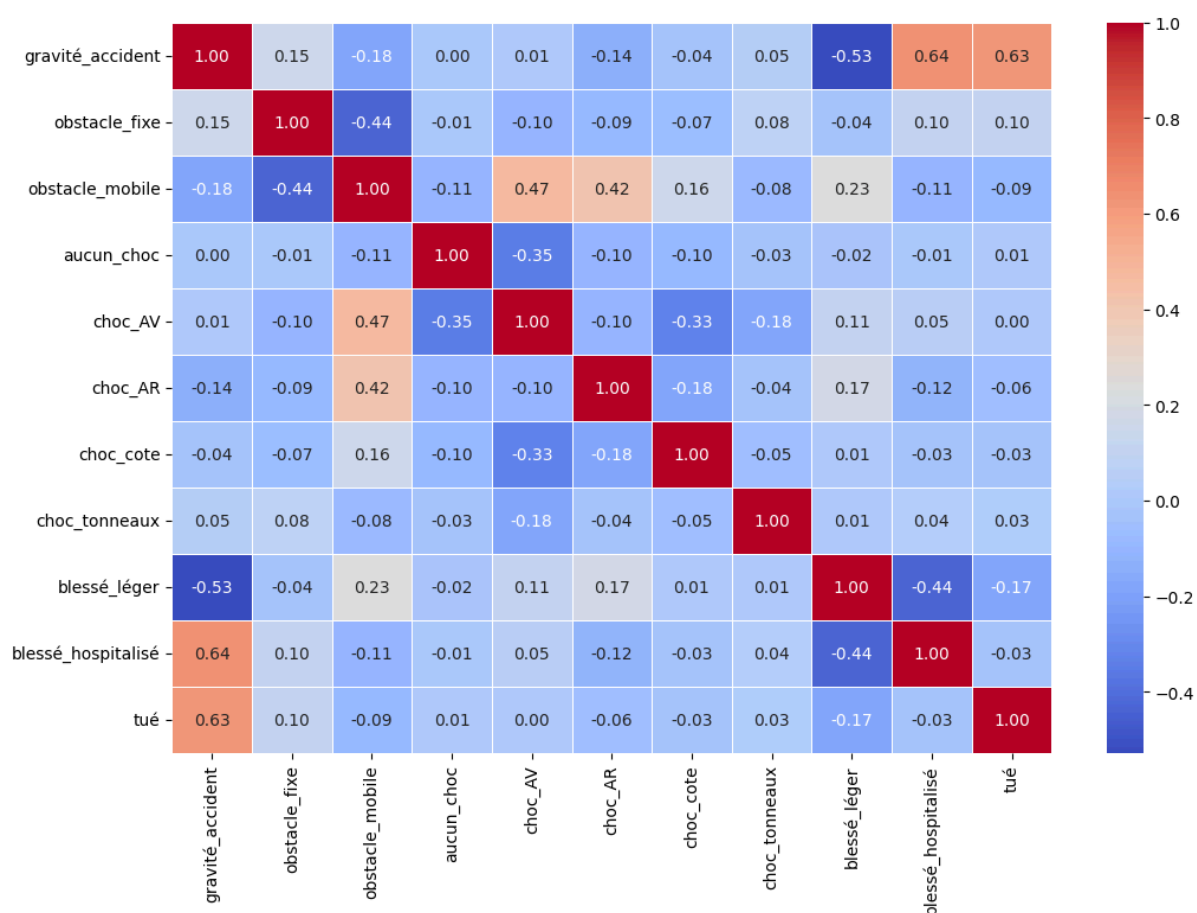
Cette matrice montre que si la gravité reste fortement liée aux blessures graves et décès (+0.64/+0.63) et opposée aux blessures légères (-0.53), les conditions routières spécifiques modulent ce risque de manière nuancée. Les autoroutes, bien qu'associées à des vitesses maximales autorisées (VMA) plus élevées (+0.55), ne montrent pas de corrélation positive forte avec la gravité (-0.08), suggérant que leur conception pourrait atténuer la sévérité des

accidents une fois survenus, contrairement aux autres réseaux

(nationale_départementale_communale) où le risque de gravité est légèrement plus présent (+0.03) malgré une VMA plus faible en moyenne (-0.39).

Le caractère bidirectionnel des routes présente une légère corrélation positive avec la gravité (+0.13), soulignant le risque inhérent aux croisements. Fait notable, ni route sèche (+0.01) ni route mouillée/enneigée (-0.02) ne sont fortement corrélées à la gravité finale, impliquant que si ces conditions influencent la survenue d'accidents, ce sont d'autres facteurs (comportement, vitesse d'impact) qui déterminent principalement la sévérité de l'issue. Les efforts doivent cibler la gestion des vitesses (VMA corrélée à la gravité +0.20), la sécurisation des routes bidirectionnelles hors autoroute, et l'adaptation du comportement des conducteurs indépendamment de l'état apparent de la chaussée (sèche/humide), car ce dernier n'est pas un prédicteur direct puissant de la gravité finale dans cette analyse.

II.6.3. Corrélation selon les obstacles et types de chocs



Cette matrice confirme l'équation gravité = conséquences sévères (+0.64 hospitalisés, +0.63 tués, -0.53 légers). L'insight clé réside dans les obstacles : heurter un obstacle fixe est légèrement corrélé à une gravité accrue (+0.15), tandis qu'un obstacle mobile l'est négativement (-0.18). Les chocs avant/arrière sont logiquement liés aux obstacles mobiles (+0.47/+0.42). Fait notable, aucun type de choc spécifique (avant, arrière, côté, tonneaux) n'est fortement corrélé positivement à la gravité seule.

L'impact contre un obstacle fixe est proportionnellement plus dangereux. La priorité est donc la sécurisation des abords routiers (dégagement, glissières). La sévérité finale dépend plus de l'énergie de l'impact (vitesse) et de la nature de l'obstacle (fixe > mobile en moyenne ici) que du simple angle de choc. La prévention doit cibler la réduction de l'énergie cinétique (vitesse) et la protection contre les obstacles rigides.

Conclusion partielle

L'analyse croisée de ces matrices révèle une image cohérente : la classification de gravité_accident reflète fidèlement le risque d'issues fatales ou d'hospitalisation. Un profil d'utilisateur se détache – les hommes conducteurs de 18 à 60 ans – comme étant particulièrement associé aux configurations accidentelles les plus sérieuses.

Sur le plan environnemental, si les autoroutes semblent relativement plus sûres en termes de gravité moyenne par accident, les routes hors autoroute, notamment bidirectionnelles, ainsi que la présence d'obstacles fixes, apparaissent comme des facteurs de risque aggravants.

De manière significative, ni l'état de la chaussée (sèche/humide) ni le type de choc initial ne semblent être les principaux déterminants directs de la gravité finale comparés à l'énergie de l'impact (liée à la vitesse) et à la nature de l'obstacle heurté (fixe étant potentiellement plus dangereux).

Ces constats suggèrent que les efforts de prévention doivent intégrer une approche ciblée sur les profils à risque, une gestion rigoureuse des vitesses, et une sécurisation accrue des infrastructures routières contre les obstacles les plus dangereux.

III. Modélisation pour prédire la gravité d'un accident

Nous entrons à présent dans la phase de modélisation prédictive à l'aide de techniques de machine learning. L'objectif est d'identifier les modèles les plus efficaces pour prédire la gravité d'un accident, afin de mieux mobiliser les secours en cas d'urgence.

La variable cible `gravité_accident` est divisée en trois classes : Blessé léger, Blessé hospitalisé, Tué. Il s'agira donc de tester des modèles de classification. Comme évoqué précédemment, la distribution de la variable gravité est très déséquilibrée: il faudra donc en tenir compte dans notre approche.

Notre démarche s'articule en plusieurs étapes :

- Feature engineering pour préparer un dataframe `df_machine_learning` avec les variables que l'on garde pour les modèles.
 - Conservation de l'heure de l'accident et suppression du reste de la date.
 - Nettoyage colonne nombre de voies (`nbv`), certaines valeurs n'étant pas numérique, et d'autres irréalistes (Les routes à 11 et 12 voies n'existent pas en France). On les remplace par le mode de la variable.
 - Nettoyage colonne atmosphère (`atm`): on les regroupe en 4 catégories: 'Temps normal', 'temps couvert', 'temps pluvieux', 'autre'
 - Enregistrement du `df_machine_learning`
- Tester les différents modèles de classification à 3 classes de manière simple.
- Tester les mêmes modèles avec les 5 variables reconnues comme les plus importantes par le modèle RandomForest.
- Tester les différents modèles avec deux approches de rééchantillonnage pour gérer le déséquilibre des classes: undersampling et de oversampling
- Tester les différents modèles avec des approches de pondération de classes pour optimiser les résultats selon les axes définis.
- Réitérer la démarche avec des modèles de classification binaire.

Remarque : Nous avons aussi exploré brièvement une approche en série temporelle visant à prédire le nombre de blessés hospitalisés par département et par mois. Cependant, les variations trop importantes du résidu ont limité la pertinence de cette méthode. Nous nous concentrerons donc sur les modèles de classification.

III.1. Choix du modèle approprié de classification

Suite à l'analyse exploratoire des données, l'objectif de cette phase d'analyse est de construire et d'évaluer différents modèles de classification en vue de prédire la gravité des accidents en mettant en exergue leurs niveaux de sévérité ("Blessé léger", "Blessé hospitalisé" et "Tué").

Pour cela, nous utilisons plusieurs approches de classification basées sur les principaux modèles (Random Forest, Logistic Regression, XGBoost, et KNN), en testant des stratégies variées pour gérer le déséquilibre des classes.

III.2. Choix des métriques d'analyse et des classes prioritaires

Dans le cadre de ce projet de prédiction de la gravité des accidents corporels de circulation, il est essentiel de comprendre les priorités métier pour orienter les choix méthodologiques. Nous justifierons dans ce paragraphe le choix de la classe prioritaire d'une part et le choix particulier de certaines métriques spécifiques d'autre part.

Dans le cadre de notre étude, la prédiction de la gravité d'un accident vise à améliorer la réactivité et l'adéquation des secours. Parmi les trois catégories de gravité (blessé léger, blessé hospitalisé, tué), c'est la classe des blessés hospitalisés qui représente le plus fort enjeu opérationnel pour les services d'urgence.

Blessés légers

Souvent autonomes, leur état ne nécessite pas toujours une prise en charge immédiate ou lourde. Ils peuvent parfois se déplacer par leurs propres moyens ou attendre une évacuation différée.

Tués

Malheureusement, lorsqu'une personne est décédée sur les lieux, l'urgence médicale n'existe plus. Bien sûr, la situation reste grave sur le plan humain, mais elle ne justifie pas une mobilisation médicale urgente (au sens de la logistique des secours).

Blessés hospitalisés : la classe critique

Ce sont les personnes en état grave mais encore en vie, qui nécessitent une prise en charge rapide, spécialisée, et potentiellement vitale : intervention de médecins ou pompiers spécialisés, acheminement rapide vers un hôpital adapté, mobilisation de moyens lourds (ambulance médicalisée, hélicoptère, équipe SMUR...).

Dans ce contexte, une prédiction fiable de la présence de blessés hospitalisés permettrait :

- D'anticiper les besoins en personnel et en matériel avant même l'arrivée sur les lieux.
- D'optimiser la répartition des ressources (notamment dans des zones rurales ou surchargées).
- De réduire le temps de prise en charge, un facteur directement lié au taux de survie.
- D'éviter le sous-dimensionnement (qui mettrait des vies en danger) comme le sur-dimensionnement (mobilisation inutile de secours ailleurs nécessaires).

En résumé, la bonne prédiction des blessés hospitalisés représente le point d'équilibre entre efficacité des secours, gain de temps critique, et rationalisation des ressources d'urgence.

Quant au choix des métriques, le choix du « *Recall* » garantit que nous détectons le maximum de vrais cas de blessés hospitalisés, même au prix d'une légère augmentation des faux positifs. Cette stratégie est essentielle pour ne rater aucun cas nécessitant une intervention urgente. Considérant que la « *Précision* » (réduction des faux positifs) reste importante dans un contexte de gestion de ressources limitées (hôpitaux, ambulances), l'équilibre *Recall/Précision* fourni par le F1-Score est un indicateur clé pour évaluer l'efficacité globale du modèle.

L'objectif est donc d'identifier le modèle offrant le meilleur équilibre entre précision et détection des cas critiques, tout en minimisant les erreurs de classification. Chaque modèle sera analysé individuellement pour mettre en lumière ses forces, ses limites et son adéquation avec les besoins.

III.3. Choix des stratégies de gestion du déséquilibre des classes

Dans le cadre de notre analyse, où l'objectif principal est de prédire la classe "Blessé hospitalisé", nous avons été confrontés à un déséquilibre important entre les différentes classes de notre jeu de données. Pour résoudre ce problème et améliorer la performance du modèle pour la classe sous-représentée, nous avons testé deux approches:

- **Ré-échantillonnage**

Nous avons exploré deux techniques couramment utilisées pour ré-échantillonner les données:

- **Oversampling**

L'oversampling consiste à augmenter artificiellement la quantité d'exemples des classes minoritaires. Dans notre cas, cela signifie augmenter le nombre d'exemples pour les classes "Blessé hospitalisé". et "Tué". Cette technique permet de rééquilibrer les classes en fournissant plus de données aux classes minoritaires, ce qui peut aider le modèle à mieux apprendre à prédire ces classes en particulier.

Pour cela, nous avons utilisé la technique SMOTE (Synthetic Minority Over-sampling Technique).

Grâce à l'oversampling, le modèle bénéficie d'un nombre plus élevé d'exemples pour la classe "Blessé hospitalisé", ce qui lui permet de mieux apprendre et de mieux reconnaître ces cas lors des prédictions. Les classes stratégiques, comme celle des "Blessés hospitalisés", sont mieux prises en compte par le modèle, ce qui améliore la précision de cette classe spécifique.

Mais en augmentant artificiellement le nombre d'exemples de la classe minoritaire, il existe un risque de sur-apprentissage (overfitting), où le modèle s'adapte trop aux exemples

répétés ou synthétiques, ce qui peut nuire à sa capacité de généralisation.

Cela augmente également la taille du jeu de données, ce qui peut allonger les temps d'entraînement.

- Undersampling

Il consiste à réduire le nombre d'exemples de la classe majoritaire, afin d'équilibrer les classes en supprimant une partie des données des classes les plus représentées. Dans notre cas, cela aurait impliqué de réduire le nombre d'exemples pour la classe "Blessé léger". Cela permet de rendre l'ensemble de données plus équilibré, en donnant un poids plus équitable à chaque classe.

Nous avons testé cette approche en éliminant aléatoirement une partie des exemples de la classe majoritaire, ce qui a permis de mieux équilibrer le jeu de données sans introduire de nouvelles instances. Bien que cette technique puisse accélérer l'entraînement et éviter certains biais, elle présente aussi des inconvénients importants.

L'undersampling permet une réduction de la taille du jeu de données, ce qui accélère l'entraînement du modèle. Le modèle peut être moins influencé par la classe majoritaire, ce qui l'aide à mieux apprendre à prédire la classe minoritaire.

Mais la suppression d'exemples peut entraîner une perte d'informations importantes provenant de la classe majoritaire, ce qui peut dégrader la capacité du modèle à généraliser sur les données de cette classe.

● Pondération de métriques

Afin de prendre en compte le déséquilibre des classes dans notre problème de classification, deux stratégies complémentaires ont été mises en œuvre :

- Pondération des classes pendant l'entraînement

Le paramètre `class_weight` du modèle a été utilisé pour accorder davantage d'importance à la classe "**Blessé hospitalisé**", souvent sous-représentée et particulièrement critique pour notre problématique. En ajustant les pondérations des classes (pondérations personnalisées ou `'class_weight = balanced'`), cela permet au modèle de mieux apprendre à

reconnaître ces cas, en augmentant la pénalité associée aux erreurs de classification pour cette classe.

Afin d'identifier la configuration optimale de pondération des classes, plusieurs jeux de poids ont été testés. Une **validation croisée à 5 plis** a été réalisée pour chaque configuration, avec une évaluation de la performance basée sur une **métrique de rappel pondéré** (ou **F1-score pondéré**), qui ajuste l'importance de chaque classe en fonction de son poids défini.

Cette méthodologie permet de comparer efficacement les différentes stratégies de pondération et de sélectionner celle qui maximise le **rappel pondéré global**, tout en mettant l'accent sur les classes stratégiques telles que **"Blessé hospitalisé"**. Une fois la meilleure configuration de poids identifiée, elle a été appliquée pour entraîner le modèle final, qui a ensuite été évalué sur l'ensemble test.

- Utilisation de métriques pondérées lors de l'évaluation

Afin de mieux refléter la performance globale du modèle, le **rappel** et le **F1-score** ont été calculés avec l'option 'average = weighted'. Cette approche pondère les scores de chaque classe en fonction de leur proportion dans l'ensemble de test. Ainsi, les classes plus fréquentes auront plus d'impact sur le score final, tandis que les classes moins fréquentes, mais stratégiques (comme **"Blessé hospitalisé"**), contribueront également de manière appropriée. Cela permet d'obtenir une évaluation plus équilibrée et représentative de la performance du modèle, particulièrement dans le cas de données déséquilibrées.

III.4. Modèle de classification à 3 classes: Blessé léger, Blessé hospitalisé et Tué

III.4.1 Analyse globale des résultats des modèles de classification à 3 classes

- **Modèle Random Forest**

RANDOM FOREST	Best parameters	Accuracy	Precision			Recall			F1_Score		
			Léger	Hospitalisé	Tué	Léger	Hospitalisé	Tué	Léger	Hospitalisé	Tué
Simple		0.69	0.76	0.53	0.27	0.85	0.46	0.09	0.80	0.49	0.14
Simple avec 6 variables les plus importantes		0.69	0.74	0.54	0.19	0.88	0.41	0.02	0.80	0.47	0.03
Oversampling		0.68	0.77	0.52	0.25	0.83	0.47	0.16	0.80	0.50	0.20
Undersampling		0.58	0.82	0.83	0.16	0.67	0.39	0.58	0.74	0.41	0.25
Grid search avec class_weight = balanced	{'classifier__class_weight': None, 'classifier__max_depth': 20, 'classifier__n_estimators': 100}	0.72	0.76	0.58	0.48	0.89	0.47	0.03	0.82	0.52	0.06
Métrique pondérée pour optimiser le Recall de la classe hospitalisé	{0: 1, 1: 2, 2: 1} n_estimator = 50, max_depth = 10	0.70	0.80	0.52	0.50	0.79	0.62	0.00	0.80	0.57	0.00
Métrique pondérée pour optimiser le F1_SCORE de la classe hospitalisés	{0: 1, 1: 2, 2: 1} n_estimator = 50, max_depth = 10	0.70	0.80	0.52	0.50	0.79	0.62	0.00	0.80	0.57	0.00
Métrique pondérée pour meilleur RECALL de la classe hospitalisés	{0: 1, 1: 2, 2: 1} n_estimator = 50, max_depth = 10	0.70	0.80	0.52	0.50	0.79	0.62	0.00	0.80	0.57	0.00

Les différentes approches de classification basées sur le modèle de Random Forest montrent une précision globale variant entre **0.58** et **0.72**. Le score de la métrique « Recall » pour la classe 'Blessé hospitalisé' atteint **0.62** pour les approches pondérées. De plus, la valeur du F1-Score atteint jusqu'à **0.57** grâce aux pondérations ciblées, montrant ainsi un excellent équilibre entre les métriques Précision et Rappel.

En somme, le modèle Random Forest présente une satisfaisante détection des cas « Blessé hospitalisé » grâce aux approches pondérées. Le niveau de F1-Score élevé prouve un compromis idéal entre capacité de détection (Recall) et précision.

- **Modèle Logistic Regression**

LOGISTIC REG	Best parameters	Accuracy	Precision			Recall			F1_Score		
			Léger	Hospitalisé	Tué	Léger	Hospitalisé	Tué	Léger	Hospitalisé	Tué
Simple		0.70	0.75	0.57	0.26	0.90	0.43	0.01	0.81	0.49	0.01
Oversampling		0.67	0.82	0.45	0.15	0.70	0.35	0.61	0.75	0.39	0.25
Undersampling		0.59	0.82	0.45	0.15	0.70	0.34	0.60	0.75	0.39	0.25
Grid search	{'classifier__C': 0.01, 'classifier__solver': 'liblinear'}	0.68	0.78	0.53	0.22	0.82	0.44	0.27	0.80	0.48	0.24

Dans ce deuxième modèle de classification, les différentes approches testées montrent un taux de prédictions correctes parmi l'ensemble des prédictions effectuées variant entre **0.59** et **0.70** mettant en évidence une robustesse globale. Le score maximum de détection des vrais « Blessés hospitalisés » s'élève à **0.44** avec validation par Grid Search mais demeurent inférieur au modèle Random Forest. Quant au F1-Score, il demeure stagnant à **0.48** dans les différentes approches limitant son équilibre entre Précision et Rappel.

En somme, bien que simple et rapide à l'entraînement, ce modèle affiche une sous performance marquée pour la détection des « Blessé hospitalisé » comparativement au modèle Random Forest. Le score moins performant du F1-Score démontre un déséquilibre entre détection et erreurs de classification.

- **Modèle XGBoost**

KNN	Best parameters	Accuracy	Precision			Recall			F1_Score		
			Léger	Hospitalisé	Tué	Léger	Hospitalisé	Tué	Léger	Hospitalisé	Tué
Grid search	{'classifier__n_neighbors': 9, 'classifier__weights': 'uniform'}	0.69	0.74	0.54	0.35	0.87	0.43	0.06	0.80	0.48	0.11
Grid search avec scoring personnalisé pour optimiser F1_score classe hospitalisés		0.68	0.75	0.52	0.28	0.85	0.44	0.06	0.79	0.48	0.10

Dans ce troisième modèle de classification, la proportion de prédictions correctes parmi l'ensemble des prédictions effectuées (variant entre **0.63** et **0.72**) est comparable au modèle Random Forest. Les différentes approches testées donnent un score maximum de détection correcte des vrais « Blessés hospitalisés » à **0.49**, inférieur au modèle Random Forest.. Le F1-score, qui équilibre les détections positives (recall) et les fausses alertes (précision), est de **0.53**, légèrement meilleur que dans le modèle Logistic Regression mais inférieur à celui du modèle Random Forest (**0.57**).

En somme, le modèle Xtreme Gradient Boosting (XGBoost) présente une efficacité comparable à Random Forest dans les approches basiques et une bonne robustesse globale sur l'Accuracy et la Précision. En revanche, ce modèle montre un score de Rappel sur les cas « Hospitalisé » inférieur à Random Forest.

- **Modèle KNN**

XGBOOST	Best parameters	Accuracy	Precision			Recall			F1_Score		
			Léger	Hospitalisé	Tué	Léger	Hospitalisé	Tué	Léger	Hospitalisé	Tué
Simple		0.63	0.84	0.50	0.19	0.73	0.43	0.61	0.78	0.46	0.29
Oversampling		0.72	0.77	0.59	0.39	0.89	0.46	0.14	0.82	0.52	0.2
Undersampling											
Grid search	{'classifier__max_depth': 6, 'classifier__n_estimators': 100, 'classifier__scale_pos_weight': 1}	0.72	0.77	0.58	0.46	0.89	0.49	0.07	0.83	0.53	0.12
Grid search avec scoring personnalisé pour optimiser F1_score classe hospitalisés		0.72	0.77	0.59	0.39	0.89	0.47	0.13	0.82	0.52	0.20

Dans ce dernier modèle de classification, le score global de prédictions correctes sur l'ensemble de prédictions réalisées varie entre **0.68** et **0.69**, inférieur aux modèles de Random Forest et XGBoost. Le taux maximum de détection des vrais cas de « Blessé hospitalisé » est de **0.44**, bien inférieur au modèle Random Forest. Le niveau maximal d'équilibre entre Rappel et Précision est de **0.48**. Cette performance est comparable à celle du modèle Logistic Regression mais les métriques Rappel et F1-Score demeurent médiocres en comparaison avec Random Forest et XGBoost.

III.4.2. Modèle recommandé: Modèle Random Forest

Le choix du modèle Random Forest se justifie ainsi:

- Meilleures performances sur la classe prioritaire "Blessé hospitalisé" : Avec un Recall de 0.62 et un F1-Score de 0.57, le modèle Random Forest est le plus performant pour détecter efficacement les blessés hospitalisés.
- Meilleur compromis Précision/Rappel : Le modèle Random Forest offre un excellent équilibre avec un F1-Score élevé, prouvant sa capacité à minimiser les erreurs tout en capturant suffisamment de cas.
- Robustesse globale : En Accuracy, le modèle Random Forest atteint jusqu'à 0.72, prouvant que le modèle prédit convenablement sur l'ensemble de la cible (les 3 classes).
- Flexibilité des approches : Les variations (Grid Search, métriques pondérées) permettent d'adapter le modèle en fonction des besoins métier spécifiques.

III.4.3. Analyse et interprétation des résultats du Modèle Random Forest

Après une analyse minutieuse des performances des quatre modèles testés (Random Forest, Logistic Regression, XGBoost, et KNN), le choix du modèle Random Forest s'impose clairement pour répondre aux priorités métier. Ce modèle offre les meilleures performances globales pour la classe critique "Blessé hospitalisé" (avec un Recall atteignant 0.62 et un F1-Score de 0.57) tout en maintenant une précision acceptable. De plus, sa flexibilité à travers différentes approches, telles que les techniques de ré-échantillonnage ou les pondérations, permet d'adapter ses résultats aux exigences spécifiques des accidents graves.

Nous allons maintenant nous concentrer sur une analyse détaillée des variations du modèle Random Forest, en explorant ses différentes approches : *modèle simple*, *oversampling*, *undersampling*, *optimisation par Grid Search* et *pondérations ciblées*. Cette analyse fine permettra de confirmer la robustesse de ce modèle en identifiant l'approche optimale qui répond aux priorités de détection des blessés hospitalisés et des cas critiques "Tué", avec un équilibre précis entre rappel et précision.

- **Approche 1 : Modèle Classique Simple**

Le modèle de base Random Forest a été entraîné avec des paramètres standards, sans aucune technique de rééquilibrage des classes. Cette configuration simple met en évidence une limitation majeure : la forte domination de la classe "Blessé léger" dans les données. En effet, la grande majorité des cas appartient à cette classe, ce qui biaise automatiquement les prédictions en sa faveur. Ainsi, bien que techniquement performant sur cette classe dominante, le modèle peine à répondre aux besoins métiers plus spécifiques concernant les cas critiques et urgents.

En termes de performances, l'Accuracy atteint 0.69, ce qui signifie que 69% des prédictions globales sont correctes. Ce résultat peut sembler satisfaisant à première vue, mais il est important de noter qu'il est largement influencé par la capacité du modèle à prédire la classe majoritaire ("Blessé léger"). Cette illusion de précision ne reflète pas l'efficacité sur

les classes prioritaires, en particulier "Blessé hospitalisé" et "Tué", qui nécessitent une attention particulière.

La précision pour la classe "Blessé hospitalisé" est relativement faible, avec un score de 0.53. Cela indique que parmi tous les cas prédits comme "Blessé hospitalisé", seuls 53% étaient effectivement corrects. Ce chiffre met en lumière les nombreuses confusions avec les autres classes, particulièrement avec "Blessé léger", ce qui peut conduire à une gestion inefficace des ressources médicales.

Concernant le Recall pour "Blessé hospitalisé", le modèle atteint seulement 0.46, ce qui signifie qu'il ne détecte que 46% des vrais cas hospitalisés, en échappant ainsi à 54% des cas prioritaires. Ce faible rappel représente une lacune importante pour les urgences médicales, où chaque minute compte et où la non-détection peut entraîner des conséquences graves pour la santé des patients concernés.

En poursuivant l'analyse, le F1-Score pour la classe "Blessé hospitalisé" est de 0.49, mettant en évidence un mauvais équilibre entre précision et rappel. En d'autres termes, le modèle n'est pas capable de trouver un compromis optimal pour détecter cette classe prioritaire, tout en minimisant les prédictions incorrectes.

En ce qui concerne la classe critique "Tué", les performances sont particulièrement préoccupantes. Avec un Recall de seulement 0.09, le modèle ne parvient à détecter que 9% des accidents mortels. Cela signifie que la quasi-totalité des cas critiques (91%) n'est pas identifiée, rendant ce modèle inadéquat pour les priorités de sécurité routière et de prévention des accidents graves.

Sur le plan opérationnel, ce modèle manque cruellement de fiabilité pour les cas urgents. Il ignore la majorité des blessés hospitalisés et des accidents mortels, ce qui compromet directement la gestion efficace des ressources médicales et des interventions d'urgence. Bien que simple et rapide à exécuter, ce modèle est inadapté pour des échéances métiers critiques, où la priorisation des cas graves tels que "Blessé hospitalisé" et "Tué" est essentielle.

- **Approche 2 : Modèle simple avec 5 variables les plus importantes**

Ce deuxième modèle apporte une simplification en se concentrant sur les 5 variables les plus importantes. L'objectif est d'obtenir des performances comparables au modèle de base tout en réduisant la complexité. Cependant, cette simplification semble avoir des contreparties.

Avec une Accuracy de 0.69, le modèle conserve une performance globale stable, mais il reste fortement influencé par la classe majoritaire "Blessé léger", créant un biais en faveur de cette catégorie. Du côté de la précision pour les "Blessés hospitalisés", une légère augmentation à 0.54 est observée, ce qui indique que 54% des cas prédits comme hospitalisés étaient corrects. Malgré cette amélioration, le Recall chute à 0.41, ne capturant que 41% des vrais blessés hospitalisés, ce qui souligne une faiblesse importante dans la détection des cas prioritaires.

Le compromis entre précision et rappel pour la classe hospitalisée reste limité, comme le montre le F1-Score de 0.47, en léger recul par rapport au modèle précédent.

En simplifiant le modèle, la détection des urgences prioritaires, en particulier des cas hospitalisés, devient encore moins précise. Ce modèle n'est donc pas adapté à notre objectif.

- **Approche 3 : Oversampling**

L'Accuracy diminue légèrement à 0.68, ce qui reflète un effort accru pour capturer les classes secondaires. Toutefois, la Précision pour les "Blessés hospitalisés" baisse à 0.52, soulignant un plus grand nombre de faux positifs, malgré une amélioration du Recall à 0.47, où presque la moitié des vrais blessés hospitalisés sont correctement identifiés.

Pour cette classe, le F1-Score atteint 0.50, indiquant un meilleur équilibre entre précision et rappel par rapport aux approches plus simplifiées. De manière positive, le Recall pour la classe "Tué" augmente à 0.16, ce qui marque une avancée significative dans la détection des cas critiques.

Bien que certaines faiblesses subsistent, l'approche d'oversampling offre une détection prometteuse des urgences graves, mais un effort supplémentaire est nécessaire pour optimiser la précision.

- **Approche 4 : Undersampling**

L'Accuracy chute à 0.58, un compromis attendu en raison de la réduction de la classe dominante. La Précision pour la classe "Blessé hospitalisé" reste stable à 0.52, mais le Recall tombe à 0.39, dénonçant une baisse dans la détection des vrais cas hospitalisés. Cela se traduit par un F1-Score faible à 0.41, marquant un déséquilibre évident.

- **Approche 5: Grid Search avec `class_weight = balanced`**

L'utilisation de la Grid Search avec des poids équilibrés entre les classes optimise les hyperparamètres tout en prenant en compte les déséquilibres. Dans ce scénario, l'Accuracy grimpe à 0.72, la meilleure parmi tous les modèles. La Précision pour les "Blessés hospitalisés" atteint 0.58, marquant une nette amélioration. Cependant, le Recall stagne à 0.47, limitant la capacité à détecter efficacement les vrais cas hospitaliers. Le F1-Score de 0.52 reste satisfaisant sans être exceptionnel.

- **Approche 6 : Pondéré pour Recall sur "Blessé hospitalisé"**

Ce modèle met l'accent sur la classe "Blessé hospitalisé", au détriment d'une précision moindre. La précision pour la classe hospitalisée diminue à 0.42, mais le Recall bondit à 0.62, ce qui permet de capturer 62% des individus hospitalisés, une nette avancée.

Ce compromis se reflète dans un F1-Score de 0.51, bien que des erreurs subsistent.

- **Approche 7 : Pondéré pour F1-score sur "Blessé hospitalisé"**

Cette approche privilégie un équilibre entre Précision **et** Recall pour la classe hospitalisée, avec pour objectif d'optimiser le F1-Score. Le modèle respecte cet objectif en atteignant un F1-Score de 0.57, le plus élevé de toutes les configurations. Avec un Recall de 0.62 et une Précision de 0.52, il offre un équilibre optimal entre détection et réduction des faux positifs. En somme, c'est le meilleur compromis pour gérer les hospitalisés.

- **Approche 8 : Pondéré pour Recall maximal (global)**

Ce dernier modèle maximise le rappel pour toutes les classes, attribuant un poids significatif aux classes moins représentées. Les résultats pour les "Blessés hospitalisés" sont similaires au modèle précédent, avec un Recall de 0.62 et un F1-Score de 0.57, témoignant d'un très bon équilibre global.

Conclusion partielle du modèle de Random Forest à 3 classes

Chaque modèle testé offre des forces distinctes, mais également des failles importantes. Si la priorité métier reste les "Blessés hospitalisés", les approches pondérées pour le Recall ou le F1-Score (modèles 6 et 7) sont des choix logiques.

III.5. Modèle de classification binaire: Urgent et Non urgent

Après avoir analysé les performances des modèles sur les trois classes initiales de gravité ("Blessé léger", "Blessé hospitalisé", et "Tué"), une seconde approche est proposée en regroupant ces classes en deux catégories : "Urgent" (Blessé hospitalisé) et "Non Urgent" (Blessé léger et Tué). Ce regroupement présente deux avantages: une simplicité opérationnelle, dans la mesure où notre objectif est de savoir s'il faut mobiliser des secours urgents ou non, et non prédire précisément le nombre de tués ou de blessés légers.

De plus, les modèles de classification binaire sont souvent plus performants que les modèles à 3 classes, et nous espérons donc une meilleure performance.

Nous allons donc reprendre l'analyse des différents modèles pour évaluer leur capacité à identifier efficacement cette nouvelle structure binaire, en étudiant leurs performances sur des métriques clés telles que le Recall et le F1-Score de la classe "Urgent", tout en maintenant un équilibre global acceptable pour les prédictions.

III.5.1. Analyse globale des modèles de classification binaire

Dans le cadre de notre objectif prioritaire de distinguer efficacement les cas urgents ("Blessé hospitalisé") des cas non urgents ("Blessé léger" et "Tué"), nous avons testé différents modèles de classification. L'analyse comparative permet de déterminer le modèle le plus performant, capable de répondre aux besoins opérationnels tout en maintenant une

qualité prédictive robuste. Cet exercice repose sur des métriques clés telles que l'Accuracy, la Précision, le Recall et le F1-Score, en se concentrant particulièrement sur la classe prioritaire "Urgent".

- **Modèle Random Forest**

2_CLASSES RANDOM_FOREST	Best parameters	Accuracy	Precision		Recall		F1_Score	
			Non urgent (léger/tué)	Urgent (hospitalisé)	Non urgent (léger/tué)	Urgent (hospitalisé)	Non urgent (léger/tué)	Urgent (hospitalisé)
Simple		0.72	0.77	0.54	0.86	0.39	0.81	0.45
Oversampling		0.71	0.78	0.53	0.83	0.44	0.80	0.48
Undersampling		0.67	0.83	0.47	0.67	0.67	0.74	0.55
Grid search	{'classifier__class_weight': 'balanced', 'classifier__max_depth': 20, 'classifier__n_estimators': 200}	0.72	0.82	0.53	0.77	0.60	0.80	0.56
Grid search avec scoring personnalisé pour optimiser recall classe Urgent		0.70	0.83	0.51	0.73	0.65	0.78	0.57
Métrique pondérée pour meilleur recall de la classe urgent	{0: 1, 1: 2}	0.72	0.82	0.53	0.77	0.60	0.79	0.56

Le modèle Random Forest se démarque par la solution équilibrée et robustes. Les performances dudit modèle démontrent une maîtrise de l'équilibre entre précision et rappel, notamment grâce à l'optimisation par Grid Search. Avec une accuracy constante entre 0.67 et 0.72, il garantit une fiabilité des prédictions globales tout en se concentrant sur les priorités métiers.

Sa précision pour la classe "Urgent" oscille entre 0.47 et 0.54, **ce** qui est honnête pour un modèle équilibré. Mais son véritable atout réside dans sa capacité à atteindre un Recall maximal de 0.67, signifiant une meilleure détection des blessés hospitalisés, cible essentielle de notre analyse.

Enfin, son F1-Score de 0.57 sur la classe "Urgent", positionne Random Forest comme une des solutions stratégiques et techniques idéales pour allier efficacité et pertinence.

- **Modèle Logistic Regression**

2_classes LOGISTIC_REG	Best parameters	Accuracy	Precision		Recall		F1_Score	
			Non urgent (léger/tué)	Urgent (hospitalisé)	Non urgent (léger/tué)	Urgent (hospitalisé)	Non urgent (léger/tué)	Urgent (hospitalisé)
Simple		0.73	0.76	0.60	0.91	0.32	0.83	0.41
Oversampling		0.68	0.83	0.47	0.68	0.68	0.75	0.56
Undersampling		0.68	0.83	0.48	0.68	0.67	0.75	0.56

La régression logistique se révèle être une solution simple et lisible, appréciée pour ses performances globales. Elle montre des performances solides, mais limitées dans la spécialisation. Ce modèle brille notamment par son accuracy stable entre 0.68 et 0.73, offrant des résultats fiables et facilement interprétables. Cependant, sa précision pour la classe "Urgent", bien qu'atteignant 0.60 sur un modèle simple, tend à diminuer lorsqu'elle est combinée avec des techniques d'échantillonnage. Cela reflète un manque de flexibilité dans les scénarios optimisés.

Son Recall, culminant à 0.68 dans l'approche oversampling, démontre une excellente capacité de détection des vrais blessés hospitalisés, mais au détriment d'autres métriques. Malgré cela, son F1-Score de 0.56, bien qu'équilibré, reste légèrement inférieur à celui de Random Forest, suggérant une adaptabilité technique plus faible pour certains besoins spécifiques.

- **Modèle XGBoost**

2_classes XGBOOST	Best parameters	Accuracy	Precision		Recall		F1_Score	
			Non urgent (léger/tué)	Urgent (hospitalisé)	Non urgent (léger/tué)	Urgent (hospitalisé)	Non urgent (léger/tué)	Urgent (hospitalisé)
Simple		0.70	0.84	0.50	0.70	0.70	0.77	0.59
Oversampling		0.74	0.79	0.58	0.85	0.47	0.82	0.52
Grid search avec scoring personnalisé pour optimiser Recall classe hospitalisés		0.70	0.77	0.49	0.81	0.42	0.79	0.45

Le modèle XGBoost, largement connu pour sa capacité à gérer des tâches complexes, se positionne comme une alternative proche de Random Forest. Cependant, malgré ses atouts, il peine à dépasser les performances obtenues par ce dernier dans des scénarios plus spécifiques.

Ce modèle affiche une accuracy globale entre 0.70 et 0.74, une plage relativement meilleure que celle de Random Forest. Sa précision pour la classe "Urgent" atteint un maximum de 0.58 dans l'approche oversampling, dépassant encore celle du Random Forest.

Le Recall pour la classe "Urgent (hospitalisé)" est de 0.70, indiquant une supériorité par rapport à Random Forest. Cette capacité améliorée à identifier correctement les blessés hospitalisés souligne l'importance du modèle XGBoost dans notre contexte, où la détection rapide est essentielle.

Le F1-Score pour XGBoost est de 0.59, ce qui témoigne d'une meilleure synergie entre précision et Recall par rapport à Random Forest. Un F1-Score de 0.59, combiné avec un Recall de 0.70, met en exergue ses atouts en termes de détection efficace des cas urgents

- **K-Nearest Neighbors (KNN)**

2_classes	KNN	Best parameters	Accuracy	Precision		Recall		F1_Score	
				Non urgent (léger/tué)	Urgent (hospitalisé)	Non urgent (léger/tué)	Urgent (hospitalisé)	Non urgent (léger/tué)	Urgent (hospitalisé)
Grid search avec scoring personnalisé pour optimiser Recall classe hospitalisés			0.70	0.77	0.49	0.81	0.42	0.79	0.45

KNN, bien qu'il ait ses vertus dans des contextes spécifiques de classification, montre ici une inadéquation manifeste pour répondre aux attentes stratégiques.

Avec une accuracy stagnante à 0.70, il n'apporte pas de réelle valeur ajoutée dans les prédictions globales. Sa précision pour la classe "Urgent", plafonnée à 0.49, souligne d'importants problèmes de faux positifs, ce qui complique la fiabilité des recommandations. Son Recall pour "Urgent", limité à 0.42, illustre également ses lacunes dans la détection des vrais blessés hospitalisés.

Avec un F1-Score de seulement 0.45, le plus faible parmi tous les modèles testés, KNN s'avère inadapté pour gérer les impératifs critiques liés aux cas urgents, rendant son utilisation peu viable dans ce contexte.

Conclusion partielle

Dans cette analyse comparative des modèles de classification, nous avons exploré les performances de quatre algorithmes : Random Forest, XGBoost, K-Nearest Neighbors (KNN), et la régression logistique. Random Forest se distingue par une accuracy solide, mais sa capacité à détecter les cas urgents (Recall et F1-Score) est surpassée par XGBoost, qui émerge comme le modèle le plus performant, atteignant un Recall de 0.70 et un F1-Score de 0.59, montrant ainsi une efficacité accrue pour identifier les urgences. En revanche, KNN, malgré son accuracy de 0.70, présente de sévères limitations dans la détection des cas urgents, avec un F1-Score de seulement 0.45, ce qui le rend inadapté à nos besoins critiques. La régression logistique, bien qu'offrant une solution simple et interprétable, souffre d'une flexibilité limitée, avec des performances globales comparables, mais inférieures à celles de Random Forest et XGBoost lorsqu'il s'agit de détection spécialisée. En somme, XGBoost constitue le meilleur choix pour notre projet, tandis que KNN et la régression logistique nécessitent des améliorations pour répondre aux exigences de détection des cas urgents.

III.5.2. Analyse des résultats des différentes approches du XGBoost

Nous avons trois approches pour notre modèle XGBoost : l'approche simple, l'oversampling et l'utilisation d'un grid search avec scoring personnalisé. Dans cette section, nous évaluons chacune d'elles à travers des métriques clés pour identifier l'approche optimale.

- **Approche Simple**

L'approche simple de XGBoost montre une **accuracy** de **0.70**, ce qui indique une bonne capacité globale de classification. En ce qui concerne la **précision**, le modèle atteint **0.84** pour les cas non urgents, mais se heurte à un score de seulement **0.50** pour les cas urgents. Le Recall pour la classe "Urgents" est de **0.70**, indiquant une capacité raisonnable à détecter les urgences. Avec un F1-Score de **0.59**, cette approche équilibre relativement bien précisions et rappels, mais le score de précision pour les cas urgents suggère un potentiel d'amélioration.

- **Approche Oversampling**

Avec l'oversampling, l'accuracy augmente légèrement à **0.74**. Cependant, les performances pour la classe urgentes soulignent des défis. La précision pour les urgentes passe à **0.58**, mais le Recall chute à **0.42**, ce qui signifie que le modèle ne détecte correctement qu'une petite fraction des urgences. Le F1-Score s'établit à **0.45**, indiquant un équilibre moins favorable entre précisions et rappels pour cette classe. Bien que l'oversampling puisse améliorer la performance globale du modèle, il semble compromettre sérieusement la détection des urgences.

- **Approche Grid Search avec Scoring Personnalisé**

La dernière approche, utilisant le grid search avec un scoring personnalisé, a pour objectif d'optimiser spécifiquement le Recall pour la classe des hospitalisés. L'accuracy est de **0.70**, comparable à l'approche simple. La précision pour les cas urgents est de **0.49**, tandis que le Recall est limité à **0.42**. Cette approche montre un F1-Score de **0.45**, qui n'est pas satisfaisant pour les cas urgents.

Conclusion partielle

À travers l'analyse des performances des trois approches, il apparaît clairement que l'approche simple s'avère être la plus efficace pour détecter les cas urgents. Avec un Recall de **0.70** et un F1-Score de **0.59**, elle démontre une bien meilleure capacité de détection des urgences que l'oversampling ou le grid search. Le manque de précision dans les cas urgents pour les deux autres approches rend leur utilisation moins appropriée, en particulier dans des situations où la détection précoce est critique.

Ainsi, pour notre projet, l'approche simple de XGBoost doit être privilégiée, tout en continuant à rechercher des ajustements et des améliorations pour maximiser la détection des urgences .

On pouvait s'attendre à ce que le passage à un modèle binaire facilite la prédiction des cas urgents. Pourtant, les performances obtenues se révèlent assez proches de celles du modèle à trois classes, ce qui montre que la simplification du problème n'a pas suffi à améliorer significativement les résultats.

III.6. Simulation de prédiction de la gravité d'un nouvel accident

Nous envisageons que les véhicules de demain puissent être équipés d'un boîtier, similaire à la boîte noire d'un avion, capable d'enregistrer un ensemble précis de paramètres (via la connexion GPS et les interactions avec le conducteur). En cas d'accident, ce boîtier pourrait alors déclencher une alerte aux secours.

Les boîtiers des véhicules accidentés pourraient fusionner leurs données, appliquer un modèle de machine learning, et transmettre au centre de secours une estimation du nombre de blessés hospitalisés.

Il semble réaliste que ce boîtier puisse collecter les informations suivantes :

'heure', 'agg', 'nbv', 'vma', 'homme', 'femme', '0-17', '18-60', '61-95', 'PL', 'bus_car', 'VL_VU', '2roues_3roues_quad'

En nous basant sur les meilleurs modèles obtenus — Random Forest 3 classes et XGBoost 2 classes — nous testons ces configurations avec différentes variables et paramètres, dans le but de rapprocher au maximum les résultats de ceux obtenus précédemment et présentés dans la section précédente.

Pour le modèle Random Forest 3 classes, nous retenons ces variables: 'heure', 'agg', 'nbv', 'vma', 'homme', 'femme', '0-17', '18-60', '61-95'

III.6.1. Avec le Modèle Random Forest à 3 classes

Les scores obtenus avec la sélection de variables évoquée ci-dessus sont les suivants:

RANDOM FOREST	Best parameters	Accuracy	Precision			Recall			F1_Score		
			Léger	Hospitalisé	Tué	Léger	Hospitalisé	Tué	Léger	Hospitalisé	Tué
Modèle avec best paramètres et pondération, et uniquement les variables facilement accessibles par un boîtier (	{0: 1, 1: 2, 2: 1} n_estimator = 50, max_depth = 10	0.70	0.80	0.52	0.50	0.79	0.59	0	0.80	0.53	0

Pour rappel, avec la totalité des variables et ce même modèle, le Recall obtenu pour la classe blessé hospitalisé était 0.62 et le F1_score s'élevait à 0.57.

III.6.2. Avec le Modèle binaire XGBoost

Les scores obtenus avec la sélection de variables évoquée ci-dessus sont les suivants:

2_classes XGBOOST	Best parameters	Accuracy	Precision		Recall		F1_Score	
			Non urgent (léger/tué)	Urgent (hospitalisé)	Non urgent (léger/tué)	Urgent (hospitalisé)	Non urgent (léger/tué)	Urgent (hospitalisé)
Simple		0.70	0.83	0.49	0.70	0.65	0.76	0.54

Pour rappel, avec la totalité des variables et ce même modèle, le Recall obtenu pour la classe "urgent" était 0.7 et le F1_score s'élevait à 0.59.

Bien que les performances de ces deux modèles soient légèrement dégradées avec cette sélection restreinte de variables, cette approche reste pertinente car elle s'inscrit dans un scénario réaliste : celui où les données disponibles proviendraient d'un boîtier embarqué dans le véhicule. Dans cette optique, nous avons choisi de simuler une situation concrète de prédiction à l'aide de notre application Streamlit, qui sera présentée lors de la soutenance du projet.

Conclusion et axes d'amélioration

Analyse des résultats et perspectives

Au cours de ce projet, nous avons testé plusieurs modèles de machine learning et constaté que les résultats pouvaient varier de manière significative, non seulement selon le type de modèle utilisé, mais également en fonction des paramètres choisis et des variables sélectionnées.

Une application concrète à fort potentiel

Dans une mise en œuvre réelle, un tel modèle pourrait représenter un véritable atout pour les services de secours, notamment via une intégration dans un boîtier embarqué. Ce dispositif permettrait d'évaluer instantanément la gravité d'un accident et de déclencher le bon niveau d'intervention, optimisant ainsi la réactivité des secours.

Cependant, pour qu'un tel outil soit déployé efficacement, une amélioration des performances actuelles serait indispensable. Plusieurs axes d'amélioration sont envisageables :

- Retravailler les données

Un modèle performant repose avant tout sur des données de qualité. Il serait pertinent d'explorer certaines variables mises de côté jusqu'ici, ou encore de créer de nouvelles variables dérivées afin d'enrichir le jeu de données et d'apporter davantage de valeur prédictive.

- Élargir la période d'analyse

Si nous avons volontairement restreint notre étude aux années 2019 à 2023, il pourrait être intéressant d'inclure des années supplémentaires afin d'observer l'impact sur les performances des modèles.

- Optimiser l'utilisation en conditions réelles

Une orientation plus ciblée sur les variables pouvant être collectées par un boîtier permettrait de tester des modèles et configurations spécifiques à ce cas d'usage. L'objectif serait d'obtenir des scores de Recall et de F1-score significativement plus élevés, en garantissant une réelle utilité opérationnelle.

- Aller plus loin dans la prédiction

Aujourd'hui, nos modèles se concentrent sur la gravité maximale d'un accident. Une évolution possible consisterait à prédire le nombre de blessés par niveau de gravité, apportant ainsi une information plus fine pour la prise de décision des secours.

Une première mise en pratique enrichissante

Ce projet a été pour nous l'occasion de mettre en œuvre l'ensemble des compétences acquises au cours de notre formation :

- Analyse, compréhension et préparation d'un jeu de données
- Mise en place et entraînement de modèles de machine learning
- Interprétation critique des résultats

C'est une expérience riche qui nous a permis de lier théorie et application concrète et de travailler en équipe.

IV. ANNEXES: Codes associés au projet

IV.1. Nettoyage et analyse des données

```
##### Montage du drive #####

from google.colab import drive
import pandas as pd
import os
from io import StringIO
# Monter Google Drive
drive.mount('/content/drive', force_remount= True) #force_remount = True permet d'avoir un fichier mis
à jour si modifs sur google drive

##### Création all_data_vehicules (2019-2023) #####

# Chemin vers le dossier contenant les fichiers CSV
folder_path = '/content/drive/My Drive/Datascientest/Dataset_perso/Vehicules/Vehicules_par_annee'

# Lister tous les fichiers dans le dossier
all_files = os.listdir(folder_path)

# Liste pour stocker les DataFrames
dataframes_vehicules = []

# Itérer à travers chaque fichier
for file in all_files:
    file_path = os.path.join(folder_path, file)

    if file.endswith('.csv'):
        # Lire le fichier comme texte
        with open(file_path, 'r') as f:
            content = f.read()

        # Remplacer plusieurs types de séparateurs par un séparateur unique (par exemple, une
        tabulation)
        content = content.replace(',', '\t').replace('; ', '\t').replace('|', '\t') # Ajoutez d'autres
        séparateurs si nécessaire

        # Utiliser pd.read_csv avec le séparateur commun
        df_vehicules = pd.read_csv(StringIO(content), sep='\t', low_memory=False) # permet de mieux
        gérer les données de divers types pour une même variable

        # Ajouter le DataFrame à la liste
        dataframes_vehicules.append(df_vehicules)

# Concaténer tous les DataFrames
all_data_vehicules_2019_2023 = pd.concat(dataframes_vehicules, ignore_index=True)
#enregistrer dans le fichier concaténé dans le drive

all_data_vehicules_2019_2023.to_csv('/content/drive/My
Drive/Datascientest/Dataset_perso/all_data_vehicules_2019_2023.csv', index=False)
```

```
##### Création on du fichier all_data_usagers_2019_2023 #####

# Chemin vers le dossier contenant les fichiers CSV
folder_path = '/content/drive/My Drive/Datascientest/Dataset_perso/Usagers'

# Lister tous les fichiers dans le dossier
all_files = os.listdir(folder_path)

# Liste pour stocker les DataFrames
dataframes_usagers = []

# Itérer à travers chaque fichier
for file in all_files:
    file_path = os.path.join(folder_path, file)

    if file.endswith('.csv'):
        # Lire le fichier comme texte
        with open(file_path, 'r') as f:
            content = f.read()

        # Remplacer plusieurs types de séparateurs par un séparateur unique (par exemple, une
        # tabulation)
        content = content.replace(',', '\t').replace('; ', '\t').replace('|', '\t') # Ajoutez d'autres
        # séparateurs si nécessaire

        # Utiliser pd.read_csv avec le séparateur commun
        df_usagers = pd.read_csv(StringIO(content), sep='\t', low_memory=False) # permet de mieux gérer
        # les données de divers types pour une même variable

        # Ajouter le DataFrame à la liste
        dataframes_usagers.append(df_usagers)

# Concaténer tous les DataFrames
all_data_usagers_2019_2023 = pd.concat(dataframes_usagers, ignore_index=True)

all_data_usagers_2019_2023.to_csv('/content/drive/My
Drive/Datascientest/Dataset_perso/all_data_usagers_2019_2023.csv', index=False)
```



```
##### Création all_data_carac_2019_2023 #####

# Chemin vers le dossier contenant les fichiers CSV
folder_path = '/content/drive/My Drive/Datascientest/Dataset_perso/Caractéristiques'

# Lister tous les fichiers dans le dossier
all_files = os.listdir(folder_path)

# Liste pour stocker les DataFrames
dataframes_carac = []

# Itérer à travers chaque fichier
for file in all_files:
    file_path = os.path.join(folder_path, file)

    if file.endswith('.csv'):
        # Lire le fichier comme texte
        with open(file_path, 'r') as f:
            content = f.read()

        # Remplacer plusieurs types de séparateurs par un séparateur unique (par exemple, une
        # tabulation)
        content = content.replace(',', '\t').replace('; ', '\t').replace('|', '\t') # Ajoutez d'autres
        # séparateurs si nécessaire

        # Utiliser pd.read_csv avec le séparateur commun
        df_carac = pd.read_csv(StringIO(content), sep='\t', low_memory=False) # permet de mieux gérer
        # les données de divers types pour une même variable

        # Ajouter le DataFrame à la liste
        dataframes_carac.append(df_carac)

# Concaténer tous les DataFrames
all_data_carac_2019_2023 = pd.concat(dataframes_carac, ignore_index=True)

all_data_carac_2019_2023.to_csv('/content/drive/My
Drive/Datascientest/Dataset_perso/all_data_carac_2019_2023.csv', index=False)
```

```
##### Création all_data_lieux_2019_2023 #####

# Chemin vers le dossier contenant les fichiers CSV
folder_path = '/content/drive/My Drive/Datascientest/Dataset_perso/Lieux'

# Lister tous les fichiers dans le dossier
all_files = os.listdir(folder_path)

# Liste pour stocker les DataFrames
dataframes_lieux = []

# Itérer à travers chaque fichier
for file in all_files:
    file_path = os.path.join(folder_path, file)

    if file.endswith('.csv'):
        # Lire le fichier comme texte
        with open(file_path, 'r') as f:
            content = f.read()

        # Remplacer plusieurs types de séparateurs par un séparateur unique (par exemple, une
        # tabulation)
        content = content.replace(',', '\t').replace('; ', '\t').replace('|', '\t') # Ajoutez d'autres
        # séparateurs si nécessaire

        # Utiliser pd.read_csv avec le séparateur commun
        df_lieux = pd.read_csv(StringIO(content), sep='\t', low_memory=False) # permet de mieux gérer
        # les données de divers types pour une même variable

        # Ajouter le DataFrame à la liste
        dataframes_lieux.append(df_lieux)

# Concaténer tous les DataFrames
all_data_lieux_2019_2023 = pd.concat(dataframes_lieux, ignore_index=True)
all_data_lieux_2019_2023.to_csv('/content/drive/My
Drive/Datascientest/Dataset_perso/all_data_lieux_2019_2023.csv', index=False)
```

```

##### Travail table véhicules #####

## Importation de la table Véhicules - 1 ligne par véhicule impliqué
import pandas as pd
df_veh=pd.read_csv("/content/drive/MyDrive/Datascientest/Projet_accidents/Dataset/2019_2023/all_data_vehicules_2019_2023.csv")
df_veh.info() # 467169 lignes

## Pour les colonnes Obsm et Obs, on regarde la distribution des valeurs
print(df_veh["obsm"].value_counts()) # la modalité 2 signifie qu'un autre véhicule a été heurté, 0 signifie aucun obstacle heurté et 1 signifie un piéton
print(df_veh["obs"].value_counts()) # 0 signifie aucun obstacle fixe heurté, 1 signifie un autre véhicule en stationnement

# On choisit de transformer ces 2 colonnes en binaire: 0 sans obs ; 1 si obstacle. Dans l'idée d'un modèle qui prédirait la gravité de l'accident grâce à un boîtier intégré dans le véhicule, on juge que le type d'obstacle percuté n'est pas une information que le boîtier pourrait connaître, en revanche il serait probablement capable de déterminer s'il y a eu un choc avec un obstacle ou non, d'où le choix de garder une variable binaire
df_veh["obstacle_fixe"]=0
df_veh["obstacle_fixe"] = df_veh["obs"].apply(lambda x: 0 if x == 0 else 1)

df_veh
df_veh["obstacle_mobile"]=0
df_veh["obstacle_mobile"] = df_veh["obsm"].apply(lambda x: 0 if x == 0 else 1)

df_veh

##### Catégorie Véhicule #####
# Nous regardons la distribution par type de véhicule avant de procéder à un regroupement:
# La catégorie 7 est largement représentée( Véhicule Léger ), viennent ensuite les 2 roues (moto, scooter et bicyclette)

#Regroupement des catégories vehicules :
# 1 VL et V utilitaire
#2 Moto, scooter, 2 roues, 3 roues, quad
#3 Poids lourds
#4 Bus/car
#5 Velo/Vae/Edp et autres(trottinettes)

df_veh["catv"].value_counts()
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(10,10))
sns.countplot(data=df_veh, x='catv',palette="viridis")

def replace_catv(catv):
    if catv in [7,10]:
        return 1
    if catv in [2,30,31,32,33,34,35,36,41,42,43]:
        return 2
    if catv in [13,14,15]:
        return 3
    if catv in [37,38]:
        return 4
    else:
        return 5

df_veh['new_catv'] = df_veh['catv'].apply(replace_catv)
df_veh=df_veh.drop("catv",axis=1)

palette=["red","blue","yellow","green","black","purple"]
sns.countplot(data=df_veh, x='new_catv',palette=palette)

plt.xticks(ticks=[0,1, 2, 3, 4], labels=["VL/VU", "2-3roues&quad", "PL", "Bus/car", "Velo/EDP/autre"],rotation=45);
plt.title("Distribution du nombre d'accident par type de véhicule regroupés en 5 catégories");

plt.show()

```

```

#Pour la colonne CHOC , on choisit de regrouper de la manière suivante , après visualisation de la
distribution
#0 pas de choc
# 1-2-3 Avant
#4-5-6 Arriere
#7-8 coté
#9 tonneaux

df_veh["choc"].value_counts()

df_veh["aucun_choc"]=0
df_veh["choc_AV"]=0
df_veh["choc_AR"]=0
df_veh["choc_cote"]=0
df_veh["choc_tonneaux"]=0
def replace_choc(row):
    if row["choc"] in [1,2,3]:
        row["choc_AV"]+=1
    if row["choc"] in [4,5,6]:
        row["choc_AR"]+=1
    if row["choc"] in [7,8]:
        row["choc_cote"]+=1
    if row["choc"] in [9]:
        row["choc_tonneaux"]+=1
    if row["choc"] in [0]:
        row["aucun_choc"]+=1
    return row

df_veh = df_veh.apply(replace_choc, axis=1)

# On filtre le df en conservant les variables qui nous intéressent uniquement
df_veh_filtre=df_veh[["Num_Acc","id_vehicule","new_catv","obstacle_fixe","obstacle_mobile","aucun_choc",
"choc_AV","choc_AR","choc_cote","choc_tonneaux"]]

# On transforme la colonne nouvelle_cat_vehicule(new_catv) en 1 colonne par type de véhicule et les
renomme pour une meilleure lisibilité
df_veh1=pd.get_dummies(df_veh_filtre,columns=["new_catv"],dtype="int")
df_veh1=df_veh1.rename({"new_catv_1":"VL_VU","new_catv_2":"2roues_3roues_quad","new_catv_3":"PL","new_c
atv_4":"bus_car","new_catv_5":"velo_trott_edp","pieton_x":"catu_pieton","pieton_y":"place_pieton"},axis
=1)

# On rajoute une colonne qui comptabilise le nombre de véhicule

df_veh1["nbr_veh"]=0
nbr_veh_counts = df_veh1['id_vehicule'].value_counts().to_dict()
df_veh1['nbr_veh'] = df_veh1['id_vehicule'].map(nbr_veh_counts)
df_veh1

#On merge le df_veh1 avec df usagers final (df_usag8)

df_merged_usag_veh=df_usag8.merge(df_veh1,on=["Num_Acc","id_vehicule"],how="left")

df_merged_usag_veh

# On utilise le groupby par numéro d'accident en utilisant la fonction "sum" , par exemple pour les
lignes d'index 0 et 1 du df merged précédent qui correspondent au même accident, on regroupe à 1 ligne
par accident en sommant les valeurs pour les autres colonnes (ex: usager_count devient 3)
# ainsi pour chaque colonne on connait le nombre de personnes concernées ( combien d'indemne, de tués,
de passager, de piéton, de véhicule léger, de moto, etc, etc pour chaque accident )

df_merged_usag_veh_final=df_merged_usag_veh.groupby("Num_Acc").agg("sum").reset_in'dex()
df_merged_usag_veh_final.drop("id_vehicule",axis=1)

#On enregistre df_merged_usag_veh final
df_merged_usag_veh_final.to_csv('/content/drive/MyDrive/Datascientest/Projet_accidents/Dataset/2019_202
3/df_merged_usag_veh_final.csv', index=False)

#A ce stade Vehicules et usagers sont mergés dans un df qui contient 273226 lignes

```

```

##### Travail table lieux #####
import pandas as pd
df_lieux=pd.read_csv("/content/drive/MyDrive/Datascientest/Projet_accidents/Dataset/2019_2023/all_data_lieux_2019_2023.csv")
df_lieux

# On constate dans un premier temps que le df_usag8 précédent, qui contient désormais 1 ligne par
véhicule, compte 463184 lignes, alors que la table véhicule compte 467169 valeurs unique d'id_véhicules
print('nombre de valeurs id_véhicules uniques pour df_usag8',len(df_usag8["id_véhicule"].unique()))
print('nombre de valeurs id_véhicules uniques pour df_veh',len(df_veh["id_véhicule"].unique()))

# On crée 2 df pour me permettre de comparer et identifier les véhicules qui n'apparaissent pas dans la
table usagers

id_veh=df_veh[["Num_Acc","id_véhicule"]]
id_veh
id_veh_usagers=df_usag8[["Num_Acc","id_véhicule"]]

id_veh_usagers["compare"]=1
id_veh_usagers
df_compare=id_veh.merge(id_veh_usagers,on=["Num_Acc","id_véhicule"],how="left")
df_compare.isna().sum()
id_diff=df_compare[df_compare["compare"].isna()]
print(id_diff) # ce sont tous les numéros d'accidents pour lesquels ce numéro de véhicule n'est pas
dans ma table usagers: 3985 lignes . On peut supposer que ce sont des véhicules pour lesquels il n'y a
pas d'usagers ? par exemple des véhicules en stationnement ?

df_lieux_filtre=df_lieux[["Num_Acc","catr","circ","nbv","vma","surf"]]

df_lieux_filtre

df_lieux_filtre["catr"].value_counts() # 4 représente route communale, 3 route départementale , 1
autoroute et 2 nationale

##### CATR #####

##### NEW CATR : on choisit de regrouper les nationales/dep/communales d'un côté, les
autoroute de l'autre , les autres routes correspondent à – Hors réseau public – Parc de stationnement
ouvert à la circulation publique – Routes de métropole urbaine
# 1 Autoroute
#2 nationale, départementale et communale 2,3,4
#3 Autre 5,6,7,9

df_lieux_filtre["autoroute"]=0
df_lieux_filtre["nationale_départementale_communale"]=0
df_lieux_filtre["autre_route"]=0
def replace_catr(row):
    if row["catr"] in [1]:
        row["autoroute"] +=1
    if row["catr"] in [2,3,4]:
        row["nationale_départementale_communale"] +=1
    if row["catr"] in [5,6,7,9]:
        row["autre_route"] +=1
    return row

df_lieux_filtre=df_lieux_filtre.apply(replace_catr,axis=1)

```

```

##### Sens de circulation " CIRC" qu'on regroupe en 2 modalités #####

df_lieux_filtre["circ"].value_counts()

# 1 - A sens unique (initialement modalité 1)
# 2 - Bidirectionnelle (2 et autres)

df_lieux_filtre["sens_unique"]=0
df_lieux_filtre["bidirectionnel"]=0

def replace_circ(row):
    if row["circ"] in [1]:
        row["sens_unique"] +=1
    else:
        row["bidirectionnel"] +=1
    return row

df_lieux_filtre=df_lieux_filtre.apply(replace_circ,axis=1)

##### SURFACES #####

df_lieux["surf"].value_counts() # 1 route normale, 2 route mouillée

# au vu de la distribution, on regroupe comme suit :
#1 normale
#2 mouillée/enneigée
#3 autre

# lorsqu'il y a 2 types de surfaces renseignés ( ce qui arrive notamment lorsque l'accident est survenu
# au niveau d'une intersection entre 2 routes, l'état de chacune des routes est parfois différent), on
# choisit de conserver l'état le plus accidentogène ( mouillé plutôt que normal)
# 272 valeurs non renseignées, on les garde ainsi en les incluant dans "autres"

df_lieux_filtre["route_seche"]=0
df_lieux_filtre["route_mouillee_enneigee"]=0
df_lieux_filtre["etat_route_autre"]=0
def replace_surf(row):
    if row["surf"] in [1]:
        row["route_seche"] +=1
    if row["surf"] in [2,3,4,5]:
        row["route_mouillee_enneigee"] +=1
    if row["surf"] in [6,7,8,9]:
        row["etat_route_autre"]+=1
    return row

df_lieux_filtre=df_lieux_filtre.apply(replace_surf,axis=1)
df_lieux_filtre

## on fait ensuite un groupby par numéro d'accident , pour le nombre de voies(nbv), lorsqu'il y a 2
# valeurs différentes ( dans le cas d'accidents survenue à une intersection ou un croisement par
# exemple), on conserve pour le moment le plus élevé, pour la VMA on choisit de conserver la plus élevée

df_lieux_final=df_lieux_filtre.groupby("Num_Acc").agg({"nbv":"max","vma":"max","nationale_departemental
e_communale":"sum","autoroute":"sum","autre_route":"sum","sens_unique":"sum","bidirectionnel":"sum","ro
ute_seche":"max","route_mouillee_enneigee":"max","etat_route_autre":"max"}).reset_index()

# illustration des accidents où le Nombre de voies (nbv) est différents

grouped_df_nbv = df_lieux.groupby('Num_Acc')['nbv'].nunique().reset_index() # 8570 accidents
concernés
diff_nbv_groups = grouped_df_nbv[grouped_df_nbv['nbv'] > 1]['Num_Acc']

print(diff_nbv_groups)

# par ailleurs, on constate qu'il y a dans la table lieux initiale, 54 lignes pour lesquelles la
# colonne NBV est #valeurmulti, et 1 ligne où #ERREUR. On ne traite pas ces lignes qui sont en très
# faible nombre pour le moment, nous pourrions les remplacer si nécessaire par la suite ou les laisser
# telles quel, ceci n'impactera pas notre modèle)
print(len(df_lieux.loc[df_lieux["nbv"]=="#VALEURMULTI"]))
print(len(df_lieux.loc[df_lieux["nbv"]=="#ERREUR"]))
print(df_lieux.value_counts("nbv"))

# Il y a également 4404 valeurs non renseignées, et des valeurs aberrantes (9, 10, 11, 12 voies, peu
# probable sur les routes françaises.

```



```
##### illustration des accidents où la vitesse max (vma) est différente #####

grouped_df_vma = df_lieux.groupby('Num_Acc')['vma'].nunique().reset_index()    # 5169 accidents
concernés
diff_vma_groups = grouped_df_vma[grouped_df_vma['vma'] > 1]['Num_Acc']

print(diff_vma_groups)
print(df_lieux_final["vma"].unique())    # il y a quelques valeurs aberrantes ( ex. 300 900 700 800)
print(df_lieux_final.loc[df_lieux_final["vma"]>130])    # ces valeurs aberrantes ( supérieures à 130 qui
est la vitesse max autorisée, tout type de route confondu en France), représente 66 lignes
df_lieux_final.info()

##### Sauvegarde le df_lieux_final dans le drive partagé , il contient désormais 273226 lignes , 1
ligne par accident

df_lieux_final.to_csv('/content/drive/MyDrive/Datascientest/Projet_accidents/Dataset/2019_2023/df_lieux
_final', index=False)
```

```

##### Travail table caractéristiques #####

# importation du df_caractéristiques
df_carac=pd.read_csv("/content/drive/MyDrive/Datascientest/Projet_accidents/Dataset/2019_2023/all_data_carac_2019_2023.csv")
df_carac

# Merge de la table lieux précédente avec la table caractéristiques
df_carac_lieux=df_carac.merge(df_lieux_final,on="Num_Acc",how="left")

# Importation de la table précédemment mergée entre usagers et vehicules, puis merge des 2 tables afin d'obtenir le df_final complet et agrégé à 1 ligne apr accident
df_merged_usag_veh_final=pd.read_csv("/content/drive/MyDrive/Datascientest/Projet_accidents/Dataset/2019_2023/df_merged_usag_veh_final.csv")
df_total_final=df_carac_lieux.merge(df_merged_usag_veh_final,on="Num_Acc",how="left")

# On renomme quelques colonnes pour une meilleure compréhension

df_total_final=df_total_final.rename({"new_catv_1":"VL_VU","new_catv_2":"2roues_3roues","new_catv_3":"P L","new_catv_4":"bus_car","new_catv_5":"velo_trott_edp","pieton_x":"catu_pieton","pieton_y":"place_pieton"},axis=1)

# Ajout d'une colonne qui determine la gravité de l'accident, nous décidons de conserver la gravité la + importante lorsque plusieurs usagers ont des gravités différentes ( par exemple si un accident compte un blessé leger et un tué, nous considérons cet accident comme mortel )

df_total_final["gravité_accident"]=0

#4 accident mortel
#3 accident avec blessés hospitalisés
#2 accident avec blessés legers
#1 accident avec des indemnes

def determine_gravite(row):
    if row["tué"]>0:
        return 4
    if row["tué"]==0 and row["blessé_hospitalisé"]>0:
        return 3
    if row["tué"]==0 and row["blessé_hospitalisé"]==0 and row["blessé_léger"]>0:
        return 2
    if row["tué"]==0 and row["blessé_hospitalisé"]==0 and row["blessé_léger"]==0 and row["indemne"]>0:
        return 1

df_total_final['gravité_accident'] = df_total_final.apply(determine_gravite, axis=1)

# On supprime les colonnes inutiles pour notre étude (adr, com id_vehicule, lum)
# On supprime les colonnes redondantes (atu_conducteur, passager et catu_pieton sont redondantes avec la place de l'utilisateur)
# On supprime les colonnes non renseignées, qui n'apportent aucune information (sexe non renseigné, total_secu_non_enseigné, total_secu-non_déterminé, grav_non_enseignée)
# ON exclut les outliers de la variable age (>96)
df_total_final=df_total_final.drop(["adr","com","id_vehicule","lum","catu_pieton","catu_conducteur","passager","sex_non_enseigné","total_secu_non_enseigné","total_non_deter","grav_non_enseigné",">96"],axis=1)

# On enregistre le df_total_final dans le drive partagé
df_total_final.to_csv('/content/drive/MyDrive/Datascientest/Projet_accidents/Dataset/2019_2023/df_total_final.csv', index=False)

```




```
##### Usagers #####
##### Variable sexe #####

sex = df.groupby('gravité_accident').agg({'homme':'sum', 'femme': 'sum'}).reset_index()
display(sex)
fig = px.bar(sex, x='gravité_accident', y=['homme', 'femme'], title="Gravité selon le sexe",
barmode='group')
fig.update_xaxes(tickmode='array', tickvals=[2,3,4],ticktext=['blessé_léger', 'blessé_grave', 'tué'])
fig.show();

grav_homme = sex[['gravité_accident', 'homme']]
grav_femme = sex[['gravité_accident','femme']]

labels = ['Blessés légers', 'Blessés hospitalisés', 'Tués']
fig,axs=plt.subplots(1,2,figsize=(20,8))

axs[0].pie(grav_homme['homme'], labels = grav_homme['gravité_accident'],colors=
["green","orange","red"],autopct="%1.1f%%")
axs[0].set_title("Hommes")

axs[1].pie(grav_femme['femme'],labels = grav_femme['gravité_accident'], colors=
["green","orange","red"],autopct="%1.1f%%")
axs[1].set_title("Femmes")

fig.suptitle("Répartition des accidents par gravité chez les hommes et chez les femmes")
fig.legend(labels, loc='upper left')

##### Age #####

age = df.groupby('gravité_accident').agg({'0-17': 'sum', '18-60':'sum', '61-95':'sum'}).reset_index()
display(age)
fig = px.bar(age, x='gravité_accident', y=['0-17', '18-60', '61-95'], title="Répartition du nombre
d'usagers par classe d'âge et par gravité", barmode='group')
fig.update_xaxes(tickmode='array', tickvals=[2,3,4],ticktext=['blessé_léger', 'blessé_grave', 'tué'])
fig.show();

grav_0_17 = age[['gravité_accident','0-17']]
grav_18_60 = age[['gravité_accident','18-60']]
grav_61_95 = age[['gravité_accident','61-95']]

labels = ['Blessés légers', 'Blessés hospitalisés', 'Tués']
fig,axs=plt.subplots(1,3,figsize=(20,8))

axs[0].pie(grav_0_17['0-17'], labels = grav_0_17['gravité_accident'],colors=
["green","orange","red"],autopct="%1.1f%%")
axs[0].set_title("0-17 ans")

axs[1].pie(grav_18_60['18-60'],labels = grav_18_60['gravité_accident'], colors=
["green","orange","red"],autopct="%1.1f%%")
axs[1].set_title("18-60 ans")

axs[2].pie(grav_61_95['61-95'],labels = grav_61_95['gravité_accident'], colors=
["green","orange","red"],autopct="%1.1f%%")
axs[2].set_title("61-95 ans")

fig.suptitle("Répartition des accidents par gravité pour les différentes tranches d'âge")
fig.legend(labels, loc='upper left')

##### Système de sécurité #####

grav = df.groupby('gravité_accident').agg({'total_sans_secu':'sum', 'total_ceinture': 'sum',
'total_casque': 'sum', 'total_secu_enfant':'sum', 'total_gilet':'sum', 'total_airbag':'sum',
'total_gants': 'sum', 'total_gants_airbag':'sum', 'total_autre': 'sum'}).reset_index()
display(grav)
fig = px.bar(grav, x='gravité_accident', y=[ 'total_sans_secu', 'total_ceinture', 'total_casque',
'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants', 'total_gants_airbag',
'total_autre'], title="Répartition du nombre d'usagers par catégorie de système de sécurité selon la
gravité", barmode='group')
fig.update_xaxes(tickmode='array', tickvals=[2,3,4],ticktext=['blessé_léger', 'blessé_grave', 'tué'])
fig.show();
```

```

##### Localisation de l'accident #####

# Carte géographique de la répartition des accidents par gravité

import geopandas as gpd
from shapely.geometry import Point

# Affichage des coordonnées de chaque accident, et transformation de 'lat', 'long' et 'dep' en type
int.
df['lat'] = df['lat'].astype('str').apply(lambda x: x.replace('\t', '.').replace(', ', '.'))
df['lat'] = df['lat'].astype('float').round(5)
df['long'] = df['long'].astype('str').apply(lambda x: x.replace('\t', '.').replace(', ', '.'))
df['long'] = df['long'].astype('float').round(5) #avec 5 décimales, précision de 1m

# Lignes dont les départements sont numériques
df_dep_num = df.loc[df['dep'].astype('str').apply(lambda x: x.isnumeric())].astype('str')

# Lignes pour lesquelles le dep est la corse
df_dep_corse = df.loc[df['dep'].astype('str').apply(lambda x: x.isnumeric() == False)].astype('str')

# Sélectionner les accidents dans les départements dont le code est inférieur à 900, c'est à dire ceux
en métropole
df_m = df[(df['dep'].isin(df_dep_num.dep.unique())) | (df['dep'].isin(df_dep_corse.dep.unique()))]

print(df_m['dep'].unique())
# Supprimer les accidents avec des coordonnées de latitude et de longitude nulles ou manquantes
df_m = df_m[(df_m['lat'] != 0.0) & (df_m['long'] != 0.0)].dropna(subset = ['lat', 'long'], axis = 0)

# Créer une géométrie Point pour chaque paire de coordonnées (longitude, latitude)
geometry = [Point(xy) for xy in zip(df_m['long'], df_m['lat'])]

# Créer un GeoDataFrame à partir des données d'accidents et de la géométrie Point
geo_df = gpd.GeoDataFrame(df_m, geometry=geometry)

# Créer une figure et des axes pour la carte
figure, ax = plt.subplots(figsize = (13, 12))

# Masquer les axes pour une apparence plus propre
plt.axis('off')

# Définir les limites des axes pour inclure toute la France
ax.set_xlim([-5.5, 9.5]) # Longitude (Ouest-Est)
ax.set_ylim([41.0, 51.5]) # Latitude (Sud-Nord)

# Tracer les accidents sur la carte en utilisant différentes couleurs pour chaque gravité
geo_df[geo_df['gravité_accident'] == 2].plot(ax = ax, markersize = 5, color = 'green', label = 'blessés
légers')
geo_df[geo_df['gravité_accident'] == 3].plot(ax = ax, markersize = 5, color = 'orange', label =
'blessés graves')
geo_df[geo_df['gravité_accident'] == 4].plot(ax = ax, markersize = 5, color = 'red', label = 'tués')
plt.title('Représentation de la gravité des accidents en France entre 2019 et 2023')
plt.legend();

##### Agglo / Hors agglo #####

# Répartition du nombre d'accidents selon la localisation agglo/ non agglo
sns.countplot(x = 'agg', hue = 'gravité_accident', data = df_filtered)
plt.title("Répartition du nombre d'accidents selon la localisation agglo/hors agglo")
plt.xticks(rotation = 90)
plt.xlabel('localisation')
plt.show();

```



```
##### Test du Chi2 #####
contingence_agg = pd.crosstab(df['gravité_accident'], df['agg'], normalize = True)
contingence_agg = pd.DataFrame(contingence_agg).apply(lambda x: round(x, 4)*100, axis = 1)
print(contingence_agg)

# Créer la heatmap avec Plotly Express
sns.heatmap(contingence_agg, annot=True, cmap='Blues')

results = chi2_contingency(contingence_agg)
print ('la statistique du test est', results[0])
print ('la-pvalue du test est', results[1])
# Appliquer le test du Chi-carré

print(p_value)
# Interprétation:

if p_value < 0.05:
    print("La localisation (agglo/hors agglo) influence significativement la gravité de l'accident (p < 0.05).")
else:
    print ("La localisation (agglo/hors agglo) n'influence pas la gravité de l'accident (p >= 0.05).")
# Le type de choc " tonneaux" impacte la gravité significativement ( blessés hospitalisés et tués )
```

```

##### Véhicules #####

##### Catégorie véhicule #####

# extraire tous df accidents où un PL est impliqué 9423 accidents
acc_avec_pl=df.loc[df["PL"]>=1]
acc_avec_pl=acc_avec_pl.groupby("gravité_accident").size()
print("proportion des gravités d'accidents impliquant au moins 1
PL",acc_avec_pl.div(acc_avec_pl.sum()))
# 13% des accidents impliquant au moins 1 PL sont mortels ; 30% avec blessés hospi

# extraire tous les accidents où un 2 ou 3 roues ou quad est impliqué 92608 accidents
acc_avec_2_3roues=df.loc[df["2roues_3roues_quad"]>=1]
acc_avec_2_3roues=acc_avec_2_3roues.groupby("gravité_accident").size()
print("proportion des gravités d'accidents impliquant au moins 1 2roues ou 3 roues ou quad
",acc_avec_2_3roues.div(acc_avec_2_3roues.sum()))

# idem pour Bus_Car
acc_avec_bus_car=df.loc[df["bus_car"]>=1]
acc_avec_bus_car=acc_avec_bus_car.groupby("gravité_accident").size()
print("proportion des gravités d'accidents impliquant au moins 1 bus ou car ",
acc_avec_bus_car.div(acc_avec_bus_car.sum()))

# idem pour VL VU 222890 accidents

acc_av_vl_vu=df.loc[df["VL_VU"]>=1]
acc_av_vl_vu=acc_av_vl_vu.groupby("gravité_accident").size()
print("proportion des gravités d'accidents impliquant au moins 1 VL
VU",acc_av_vl_vu.div(acc_av_vl_vu.sum()))

# idem pour velo trott et edp 45918 accidents
acc_av_velo_trott_edp=df.loc[df["velo_trott_edp"]>=1]
acc_av_velo_trott_edp=acc_av_velo_trott_edp.groupby("gravité_accident").size()
print("proportion des gravités d'accidents impliquant au moins 1 engin de déplacement personnel (velo,
trott ou edp)",acc_av_velo_trott_edp.div(acc_av_velo_trott_edp.sum()))

fig,axs=plt.subplots(1,5,figsize=(15,10))
labels = ['Blessés légers', 'Blessés hospitalisés', 'Tués']

axs[0].pie(acc_avec_pl,colors=["green","orange","red"],autopct="%1.1f%%")
axs[0].set_title("Avec PL")
axs[1].pie(acc_avec_2_3roues,colors=["green","orange","red"],autopct="%1.1f%%")
axs[1].set_title("Avec 2 ou 3 roues ou quad")
axs[2].pie(acc_avec_bus_car,colors=["green","orange","red"],autopct="%1.1f%%")
axs[2].set_title("Avec bus/car")
axs[3].pie(acc_av_vl_vu,colors=["green","orange","red"],autopct="%1.1f%%")
axs[3].set_title("Avec VL VU")
axs[4].pie(acc_av_velo_trott_edp,colors=["green","orange","red"],autopct="%1.1f%%")
axs[4].set_title("Avec velo, trott ou edp")

fig.suptitle("Répartition des accidents par gravité pour différents types de véhicules")
fig.legend(labels, loc='center')

# idem pour piétons 44692 accidents
acc_av_pietons=df.loc[df["place_pieton"]>=1]
acc_av_pietons=acc_av_pietons.groupby("gravité_accident").size()
print("proportion des gravités d'accidents impliquant au moins 1
piéton",acc_av_pietons.div(acc_av_pietons.sum()))

```

```

##### Matrices de corrélation #####

# Pour plus de lisibilité, on fait plusieurs heatmaps sur des sélections de colonnes

##### Matrice de corrélation usagers #####

selected_columns1 = [ 'usager_count', '0-17', '18-60', '61-95', 'gravité_accident', 'homme', 'femme',
'place_conducteur', 'pax_AV', 'pax_AR', 'pax_Milieu', 'place_pieton', 'blessé_léger',
'blessé_hospitalisé', 'tué' ]
corr_matrix_subset1 = df_filtre[selected_columns1].corr()
plt.figure(figsize=(12, 8))
sns.heatmap(corr_matrix_subset1, annot=True, cmap="coolwarm", linewidths=0.5, fmt=".2f")
plt.show()

##### Matrice de corrélation selon les conditions de la route #####

import numpy as np
df_filtre['nbv'] = df_filtre['nbv'].replace(to_replace = ['#ERREUR', '#VALEURMULTI'], value = np.nan)
selected_columns3 = [ 'gravité_accident', 'nationale_departementale_communale', 'autoroute',
'autre_route', 'sens_unique', 'vma', 'bidirectionnel', 'route_seche',
'route_mouillee_enneigee', 'etat_route_autre', 'nbv', 'blessé_léger', 'blessé_hospitalisé', 'tué' ]
corr_matrix_subset3 = df_filtre[selected_columns3].corr()
plt.figure(figsize=(12, 8))
sns.heatmap(corr_matrix_subset3, annot=True, cmap="coolwarm", linewidths=0.5, fmt=".2f" )
plt.show()

##### Matrice de corrélation: obstacles et types de chocs #####

selected_columns2 = [ 'gravité_accident', 'obstacle_fixe', 'obstacle_mobile',
'aucun_choc', 'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'blessé_léger',
'blessé_hospitalisé', 'tué' ]
corr_matrix_subset2 = df_filtre[selected_columns2].corr()
plt.figure(figsize=(12, 8))
sns.heatmap(corr_matrix_subset2, annot=True, cmap="coolwarm", linewidths=0.5, fmt=".2f")
plt.show()

```

IV.2. Modélisation

IV.2.a. Feature engineering pour l'élaboration d'un dataframe avec les variables sélectionnées , nommé df_machine learning

```

from google.colab import drive
import pandas as pd
import os
from io import StringIO
# Monter Google Drive
drive.mount('/content/drive', force_remount= True) #force_remount = True permet d'avoir un fichier mis
à jour si modifs sur google drive

file_path = '/content/drive/My
Drive/Datascientest/Projet_accidents/Dataset/2019_2023/df_total_final.csv'
df = pd.read_csv(file_path)
df.head(10)
df.info()
df.gravité_accident.value_counts()

```

```

##### Nous conservons uniquement l'heure de l'accident

df['date']=
pd.to_datetime(df['jour'].astype('str')+'/'+df['mois'].astype('str')+'/'+df['an'].astype('str')+
+'/'+df['hrmn'].astype('str'), dayfirst = True)
df['heure'] = df['date'].dt.hour

##### Modification du type de la variable gravité_accident
df['gravité_accident']= df['gravité_accident'].astype('category')

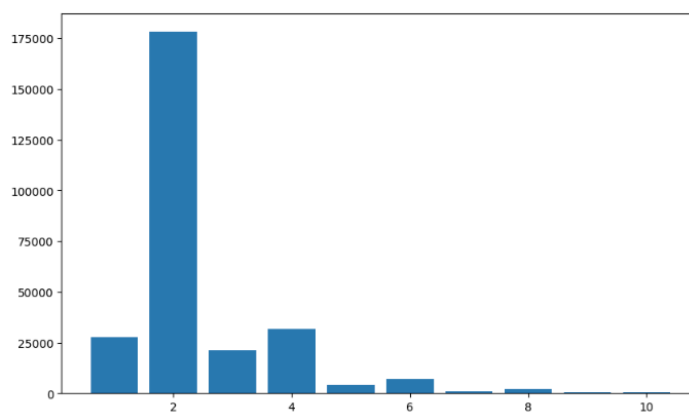
##### Nettoyage de la colonne nombre de voies

mode_value = df['nbv'].mode()[0]

# Remplacement des valeurs spécifiques
df['nbv']= df['nbv'].replace([-1, 11,12, '-1', '-1', 0], mode_value)
df['nbv'] = df['nbv'].replace(['#VALEURMULTI', '-1', '0', '#ERREUR', '11', '12'], mode_value)
df['nbv'] = df['nbv'].replace({'2': 2, '8': 8, '6': 6, '4': 4, '5': 5, '7': 7, '3':3, '1':1, '10':10,
'9':9})
df.nbv.value_counts()

import matplotlib.pyplot as plt
nbv_counts = df.nbv.value_counts()
plt.figure(figsize=(10, 6))
plt.bar(nbv_counts.index, nbv_counts.values)
plt.show();

```



```
##### Nettoyage colonne atmosphère (atm): regroupées en 4 catégories: 'Temps normal', 'temps couvert',  
'temps pluvieux', 'autre'  
df.atm.value_counts()  
import matplotlib.pyplot as plt  
atm_counts = df.atm.value_counts()  
plt.figure(figsize=(10, 6))  
plt.bar(atm_counts.index, atm_counts.values)  
plt.show();  
  
def regrouper(val):  
    if val in [1]:  
        return 'temps_normal'  
    elif val in [2, 3]:  
        return 'Temps_pluvieux'  
    elif val in [8]:  
        return 'Temps_couvert'  
    else:  
        return 'Autre'  
  
df['atm'] = df['atm'].apply(regrouper)  
df.atm.value_counts()
```

IV.2.b. Travail sur la série temporelle

```

# Préparation d'un df pour la série temporelle

from google.colab import drive
import pandas as pd
import os
from io import StringIO
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Monter Google Drive
drive.mount('/content/drive', force_remount= True) #force_remount = True permet d'avoir un fichier mis
à jour si modifs sur google drive

file_path = '/content/drive/My
Drive/Datascientest/Projet_accidents/Dataset/2019_2023/df_total_final.csv'
df_total_final=pd.read_csv("/content/drive/MyDrive/Datascientest/Projet_accidents/Dataset/2019_2023/df_
total_final.csv")
df_temp = df_total_final.copy()
df_temp['date']=
pd.to_datetime(df_temp['jour'].astype('str')+'/' +df_temp['mois'].astype('str')+'/' +df_temp['an'].astype
('str')+ '/' +df_temp['hrmn'].astype('str'), dayfirst = True)
df_temp['an_mois_jour'] = df_temp['date'].dt.strftime('%Y-%m-%d')

df_temp = df_temp[['an_mois_jour', 'blessé_léger', 'blessé_hospitalisé', 'tué','dep']]

print(df_temp.head())
# On enregistre le df_total_final dans le drive partagé
df_temp.to_csv('/content/drive/MyDrive/Datascientest/Projet_accidents/Dataset/2019_2023/df_temp.csv',
index=False)

df_temp.dep.value_counts()

#Il y a 25239 accidents dans le département 75, 13955 dans le 93, 12353 dans le 13, données suffisantes
pour effectuer des prédictions.
#Nous allons tester des prédictions par analyse temporelle, sur le département 75.
#Cette analyse peut s'appliquer à tous les départements (s'ils ont suffisamment d'accidents recensés
pour effectuer des prédictions).

# Filtrage pour le département 75

df_temp =
pd.read_csv('/content/drive/MyDrive/Datascientest/Projet_accidents/Dataset/2019_2023/df_temp.csv',
index_col = 'an_mois_jour', parse_dates=True)
df_temp_75 = df_temp[df_temp['dep'] == '75']
df_temp_75.head()

```

Mounted at /content/drive
 <ipython-input-9-dde33cbf1a44>:15: DtypeWarning: Columns (12) have mixed types. Specify dtype option on import or set low_memory=False.
 df_total_final=pd.read_csv("/content/drive/MyDrive/Datascientest/Projet_accidents/Dataset/2019_2023/df_total_final.csv")

	an_mois_jour	blessé_léger	blessé_hospitalisé	tué	dep
0	2019-11-30	2	0	0	93
1	2019-11-30	1	0	0	93
2	2019-11-28	2	0	0	92
3	2019-11-30	1	0	0	94
4	2019-11-30	1	0	0	94

count

dep

75	25239
----	-------

93	13955
----	-------

13	12353
----	-------

92	11882
----	-------

94	11820
----	-------

...	...
-----	-----

9	124
---	-----

8	91
---	----

977	65
-----	----

986	56
-----	----

975	17
-----	----

116 rows x 1 columns

dtype: int64

	blessé_léger	blessé_hospitalisé	tué	dep
an_mois_jour				
2019-10-24	1	0	0	75
2019-10-23	3	0	0	75
2019-10-23	1	0	0	75
2019-10-23	1	0	0	75
2019-10-24	1	0	0	75

```
<ipython-input-11-ec64435788fb>:5: FutureWarning: 'M' is deprecated and will be removed in a future version, please use 'ME' instead.
df_temp_75_resampled = df_temp_75.resample('D').sum().resample('M').sum()
```

	blessé_léger	blessé_hospitalisé	tué
an_mois_jour			
2019-01-31	472	29	1
2019-02-28	434	36	3
2019-03-31	495	34	0
2019-04-30	509	42	5
2019-05-31	488	24	2

```

# Etude approfondie des blessés hospitalisés en vue d'une prédiction

plt.figure(figsize = (30,15))
plt.plot('blessé_hospitalisé',data=df_temp_75_resampled)
plt.show()

from statsmodels.tsa.seasonal import seasonal_decompose

variable_hospitalisé= seasonal_decompose(df_temp_75_resampled['blessé_hospitalisé'])
variable_hospitalisé.plot()
plt.show();

# On observe des résidus avec de grandes variations, mais une saisonnalité. Testons un modèle de
régression linéaire.

from sklearn.linear_model import LinearRegression
from sklearn.metrics import root_mean_squared_error
# hospitalisé
df_temp_75_resampled['blessé_hospitalisé_lag1'] = df_temp_75_resampled['blessé_hospitalisé'].shift(1)
df_temp_75_resampled.head()

#Création de variables explicatives à partir de la série temporelle (moyenne mobile sur les 3 derniers
mois)

#Hospitalisé
df_temp_75_resampled['blessé_hospitalisé_MA_3months']=
df_temp_75_resampled['blessé_hospitalisé'].rolling(window = 3).mean()
df_temp_75_resampled.head()

#Création de variables explicatives à partir de la série temporelle (moyenne mobile sur les 3 derniers
mois)

#Hospitalisé
df_temp_75_resampled['blessé_hospitalisé_MA_3months']=
df_temp_75_resampled['blessé_hospitalisé'].rolling(window = 3).mean()
df_temp_75_resampled.head()

# Suppression des Nans
df_temp_75_resampled= df_temp_75_resampled.dropna()
df_temp_75_resampled.head()

model= LinearRegression()

# Hospitalisé
X_hospitalisé_train = X_hospitalisé.iloc[: -24]
X_hospitalisé_test = X_hospitalisé.iloc[-24:]

y_hospitalisé_train = y_hospitalisé[:-24]
y_hospitalisé_test = y_hospitalisé.iloc[-24:]

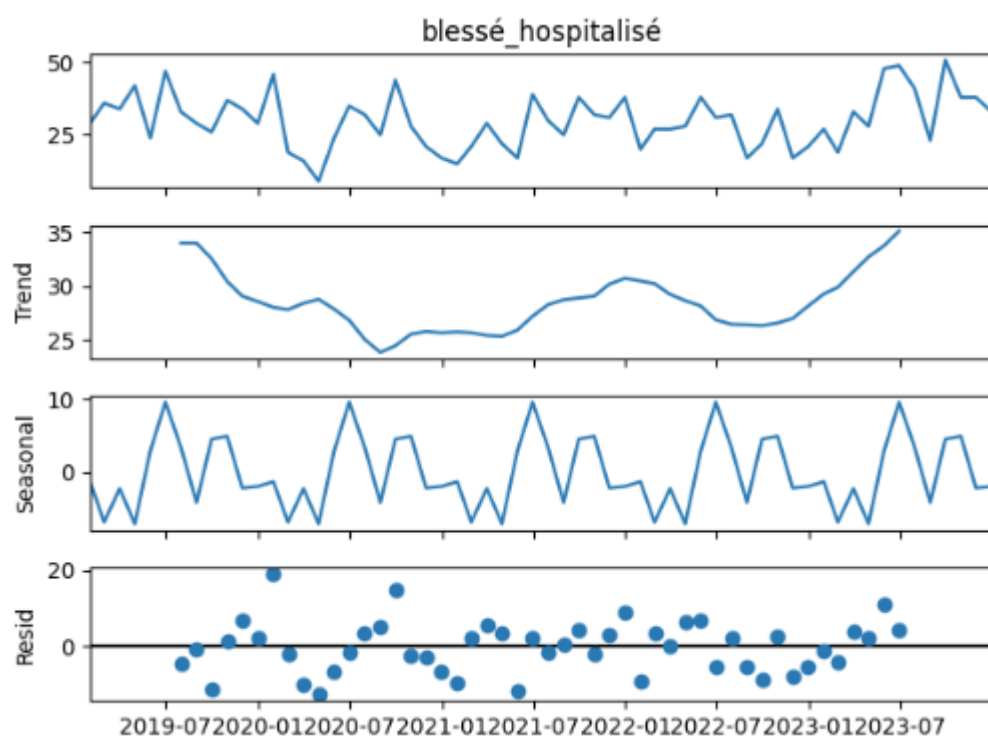
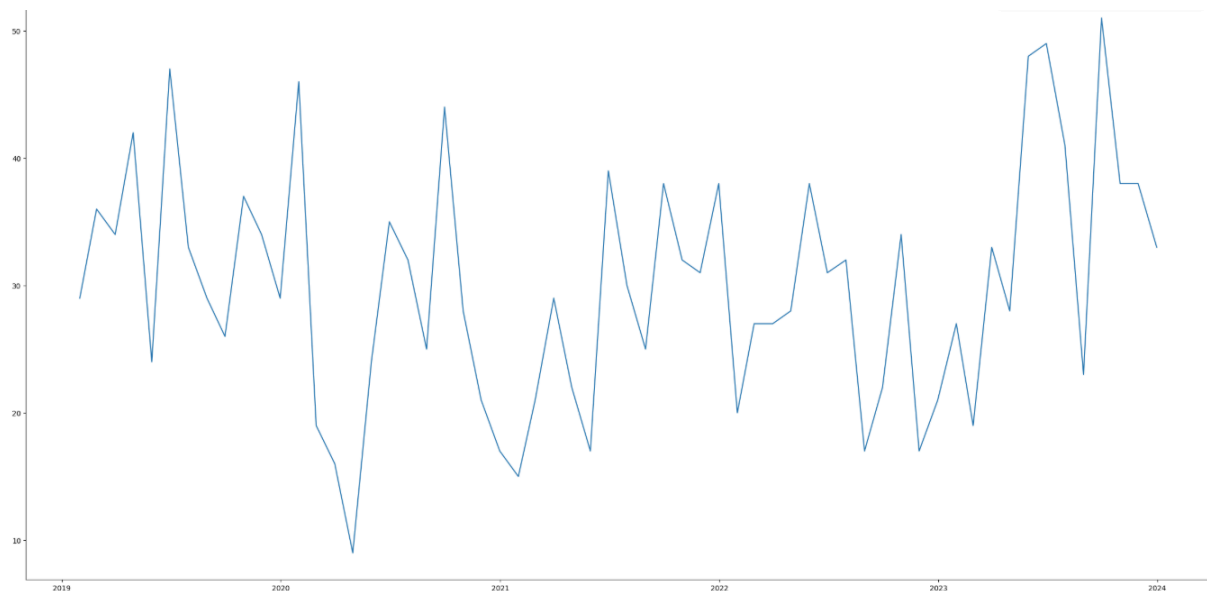
model.fit(X_hospitalisé_train, y_hospitalisé_train)

pred_test_hospitalisé = model.predict(X_hospitalisé_test)

print ('rmse', root_mean_squared_error(pred_test_hospitalisé, y_hospitalisé_test))

plt.plot(y_hospitalisé_test.index, pred_test_hospitalisé, label = 'valeurs prédites')
plt.plot(y_hospitalisé.index, y_hospitalisé, label = 'valeurs réelles')
plt.title('Prédictions du nombre de blessés hospitalisés')
plt.legend()
plt.show();

```



blessé_léger blessé_hospitalisé tué blessé_hospitalisé_lag1
 an_mois_jour

2019-01-31	472	29	1	NaN
2019-02-28	434	36	3	29.0
2019-03-31	495	34	0	36.0
2019-04-30	509	42	5	34.0
2019-05-31	488	24	2	42.0



	blessé_léger	blessé_hospitalisé	tué	blessé_hospitalisé_lag1	blessé_hospitalisé_MA_3months
an_mois_jour					
2019-03-31	495	34	0	36.0	33.000000
2019-04-30	509	42	5	34.0	37.333333
2019-05-31	488	24	2	42.0	33.333333
2019-06-30	545	47	5	24.0	37.666667
2019-07-31	520	33	5	47.0	34.666667

```

# Normalisation des données

# Hospitalisé
y_hospitalisé = df_temp_75_resampled['blessé_hospitalisé']
X_hospitalisé = df_temp_75_resampled.drop('blessé_hospitalisé', axis = 1)

model= LinearRegression()

# Hospitalisé
X_hospitalisé_train = X_hospitalisé.iloc[: -24]
X_hospitalisé_test = X_hospitalisé.iloc[-24:]

y_hospitalisé_train = y_hospitalisé[:-24]
y_hospitalisé_test = y_hospitalisé.iloc[-24:]

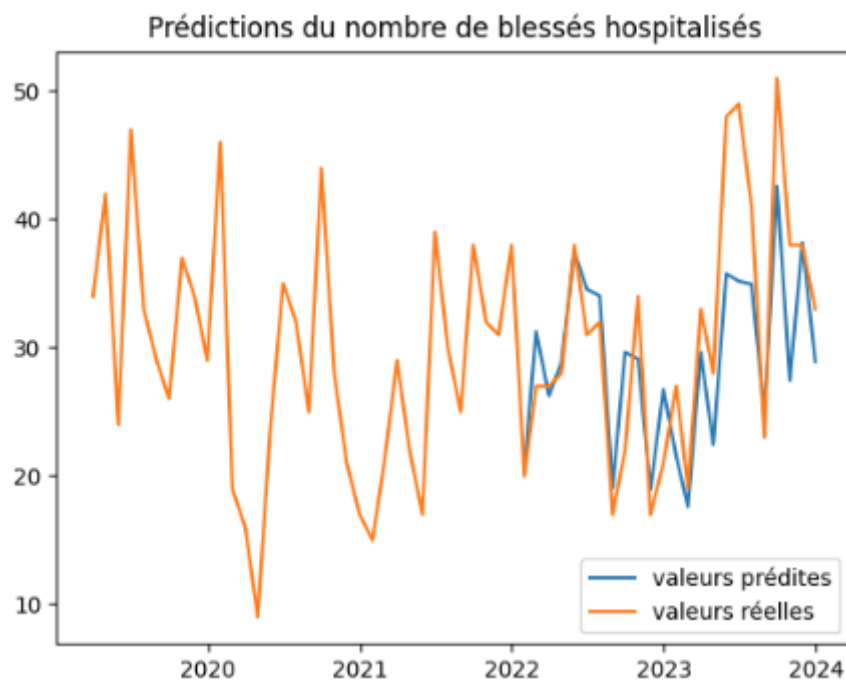
model.fit(X_hospitalisé_train, y_hospitalisé_train)

pred_test_hospitalisé = model.predict(X_hospitalisé_test)

print ('rmse', root_mean_squared_error(pred_test_hospitalisé, y_hospitalisé_test))

plt.plot(y_hospitalisé_test.index, pred_test_hospitalisé, label = 'valeurs prédites')
plt.plot(y_hospitalisé.index, y_hospitalisé, label = 'valeurs réelles')
plt.title('Prédictions du nombre de blessés hospitalisés')
plt.legend()
plt.show();

```



IV.1.2.c. Modèle 3 classes Random Forest

```
from google.colab import drive
import pandas as pd
import os
from io import StringIO
# Monter Google Drive
drive.mount('/content/drive', force_remount= True) #force_remount = True permet d'avoir un fichier mis
à jour si modifs sur google drive
df=pd.read_csv('/content/drive/MyDrive/Datascientest/Projet_accidents/Dataset/2019_2023/df_machine_learning.csv')
df['gravité_accident'] = df['gravité_accident']-2
df.info()

# 0: blessé_léger
# 1: blessé_hospitalisé
# 2: tué

#### Import des bibliothèques nécessaires
import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import matplotlib.pyplot as plt
from sklearn.pipeline import Pipeline
from imblearn.pipeline import Pipeline as imPipeline
from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import SMOTE
from sklearn.metrics import precision_score, recall_score, f1_score, confusion_matrix,
classification_report, roc_auc_score
import seaborn as sns
from sklearn.model_selection import GridSearchCV
```

```

# MODELE RANDOM FOREST SIMPLE

X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
            'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche',
                        'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
                        'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL-VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classif', RandomForestClassifier(n_estimators=100, random_state=42))])

# Entraîner le modèle
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

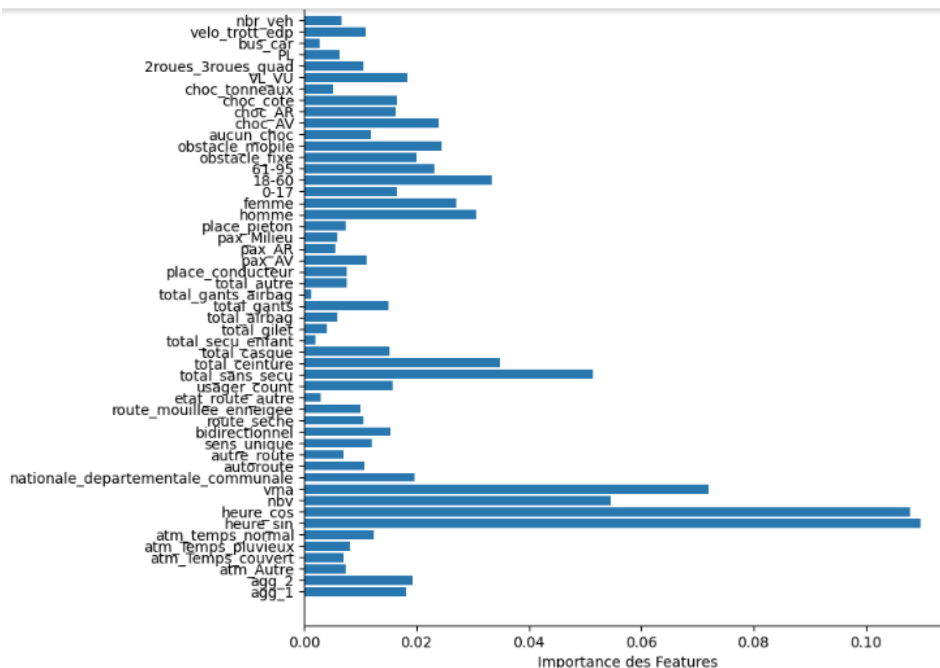
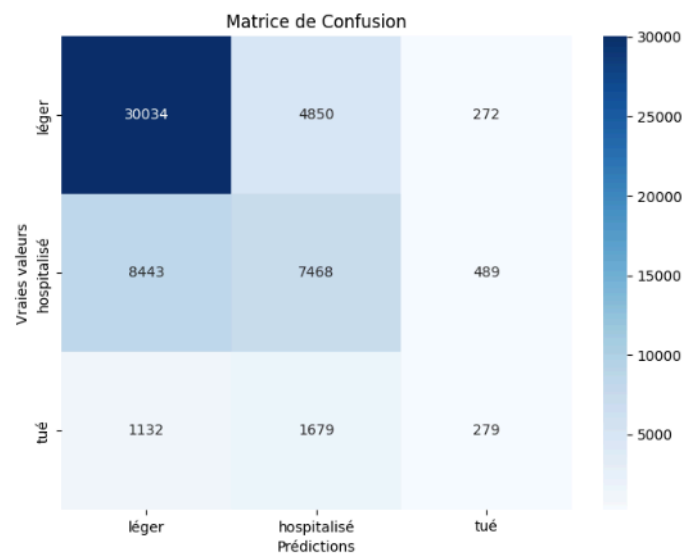
# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['léger', 'hospitalisé',
                                'tué'], yticklabels=['léger', 'hospitalisé', 'tué'])
plt.xlabel('Prédiction')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

```

Accuracy du modèle : 0.69
 Précision : 0.66
 Rappel : 0.69
 F1-score : 0.67

Rapport de Classification :

	precision	recall	f1-score	support
0	0.76	0.85	0.80	35156
1	0.53	0.46	0.49	16400
2	0.27	0.09	0.14	3090
accuracy			0.69	54646
macro avg	0.52	0.47	0.48	54646
weighted avg	0.66	0.69	0.67	54646




```

# MODELE RANDOM FOREST AVEC LES 5 VARIABLES LES PLUS IMPORTANTES

# Préparation des données
X = df.drop(['Num_Acc', 'total_ceinture', 'total_casque',
            'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
            'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
            'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
            '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
            'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
            '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh', 'autoroute', 'autre_route',
            'sens_unique', 'bidirectionnel', 'route_seche', 'atm',
            'route_mouillee_enneigee', 'etat_route_autre', 'usager_count', 'lat', 'long', 'jour', 'mois',
            'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé',
            'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale',
                        'total_sans_secu'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42))])

# Entraîner le modèle
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

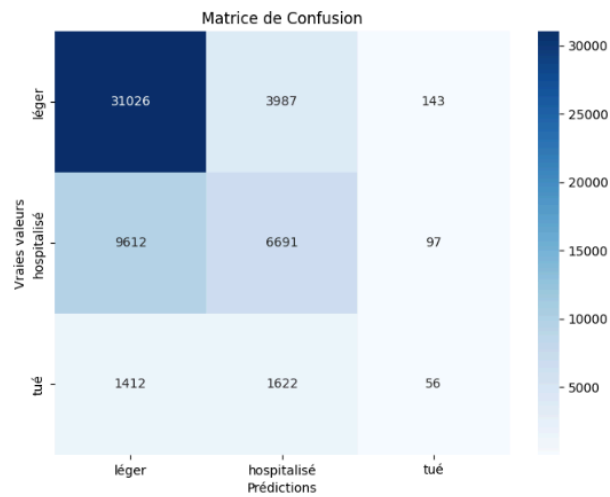
# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['léger', 'hospitalisé',
                                'tué'], yticklabels=['léger', 'hospitalisé', 'tué'])
plt.xlabel('Prédiction')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

```

Accuracy du modèle : 0.69
Précision : 0.65
Rappel : 0.69
F1-score : 0.66

Rapport de Classification :

	precision	recall	f1-score	support
0	0.74	0.88	0.80	35156
1	0.54	0.41	0.47	16400
2	0.19	0.02	0.03	3090
accuracy			0.69	54646
macro avg	0.49	0.44	0.43	54646
weighted avg	0.65	0.69	0.66	54646



```

# MODELE RANDOM FOREST AVEC OVERSAMPLING POUR GERER LE DESEQUILIBRE DES CLASSES

# Préparation des données
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
'sens_unique', 'bidirectionnel', 'route_seche',
'route_mouillee_enneigee', 'etat_route_autre', 'usager_count',
'total_sans_secu', 'total_ceinture', 'total_casque',
'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
'18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
'2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Appliquer SMOTE dans la pipeline (avant l'entraînement du modèle)
smote = SMOTE(random_state=42, sampling_strategy = 'auto', k_neighbors = 5)

# Définir la pipeline complète avec SMOTE
pipeline = imPipeline([
    ('preprocessor', preprocessor), # Transformation des variables
    ('smote', smote), # Oversampling via SMOTE
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42)) # Classificateur
])

# Entraîner le modèle avec oversampling
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle avec oversampling : {accuracy:.2f}")

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle : {accuracy:.2f}")

```

```

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['léger', 'hospitalisé', 'tué'], yticklabels=['léger', 'hospitalisé', 'tué'])
plt.xlabel('Prédictions')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

# Récupérer l'importance des features
# Récupérer le modèle du classifieur après l'entraînement
model = pipeline.named_steps['classifieur']

# Accéder aux importances des features
importances = model.feature_importances_

# Récupérer les noms des features après transformation
# Utiliser l'encodeur OneHotEncoder pour gérer les variables catégorielles encodées
# et la transformation cyclique pour obtenir les noms des features résultants.
cat_columns = pipeline.named_steps['preprocessor'].transformers_[0][1].get_feature_names_out(categorical_features)
cyclical_columns = ['heure_sin', 'heure_cos'] # Ces noms sont définis par la transformation de l'heure

# Les noms des features après transformation
features = np.concatenate([cat_columns, cyclical_columns, passthrough_features])

# Tracer l'importance des features
plt.figure(figsize=(12, 12))
plt.barh(features, importances)
plt.xlabel("Importance des Features")
plt.title("Importance des features dans le modèle Random Forest avec oversampling")
plt.show()

```

Accuracy du modèle avec oversampling : 0.68

Accuracy du modèle : 0.68

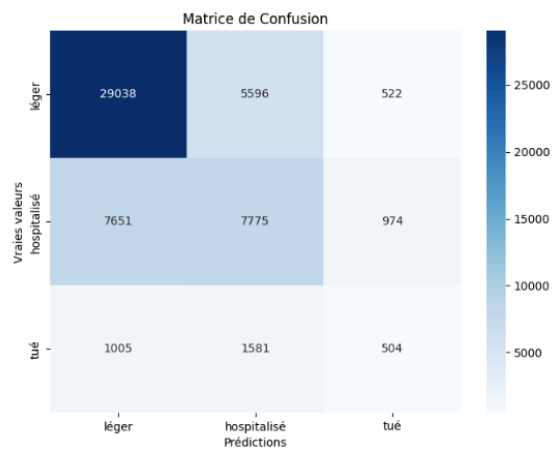
Précision : 0.67

Rappel : 0.68

F1-score : 0.67

Rapport de Classification :

	precision	recall	f1-score	support
0	0.77	0.83	0.80	35156
1	0.52	0.47	0.50	16400
2	0.25	0.16	0.20	3090
accuracy			0.68	54646
macro avg	0.51	0.49	0.50	54646
weighted avg	0.67	0.68	0.67	54646



```

# RANDOM FOREST AVEC UNDERSAMPLING

# Préparation des données
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
'sens_unique', 'bidirectionnel', 'route_seche',
'route_mouillee_enneigee', 'etat_route_autre', 'usager_count',
'total_sans_secu', 'total_ceinture', 'total_casque',
'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
'18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL-VU',
'2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Appliquer RandomUnderSampler dans la pipeline (avant l'entraînement du modèle)
under_sampler = RandomUnderSampler(random_state=42)

# Définir la pipeline complète avec undersampling
pipeline = imPipeline([
    ('preprocessor', preprocessor), # Transformation des variables
    ('under_sampler', under_sampler), # Undersampling via RandomUnderSampler
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42)) # Classificateur
])

# Entraîner le modèle avec undersampling
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle avec undersampling : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

```

```

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['léger', 'hospitalisé', 'tué'], yticklabels=['léger', 'hospitalisé', 'tué'])
plt.xlabel('Prédictions')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

# Récupérer l'importance des features
# Récupérer le modèle du classifieur après l'entraînement
model = pipeline.named_steps['classifieur']

# Accéder aux importances des features
importances = model.feature_importances_

# Récupérer les noms des features après transformation
# Utiliser l'encodeur OneHotEncoder pour gérer les variables catégorielles encodées
# et la transformation cyclique pour obtenir les noms des features résultants.
cat_columns = pipeline.named_steps['preprocessor'].transformers_[0]
[1].get_feature_names_out(categorical_features)
cyclical_columns = ['heure_sin', 'heure_cos'] # Ces noms sont définis par la transformation de l'heure

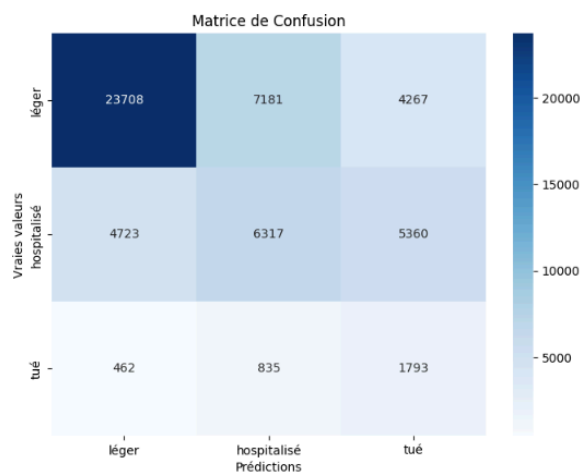
# Les noms des features après transformation
features = np.concatenate([cat_columns, cyclical_columns, passthrough_features])

# Tracer l'importance des features
plt.figure(figsize=(12, 12))
plt.barh(features, importances)
plt.xlabel("Importance des Features")
plt.title("Importance des features dans le modèle Random Forest avec undersampling")
plt.show()

```

Accuracy du modèle avec undersampling : 0.58
 Précision : 0.67
 Rappel : 0.58
 F1-score : 0.61

Rapport de Classification :				
	precision	recall	f1-score	support
0	0.82	0.67	0.74	35156
1	0.44	0.39	0.41	16400
2	0.16	0.58	0.25	3090
accuracy			0.58	54646
macro avg	0.47	0.55	0.47	54646
weighted avg	0.67	0.58	0.61	54646




```

# Grid search pour optimiser paramètres du modèle avec toutes les variables. Utilisation de
class_weight='balanced' pour gérer le déséquilibre des classes

X = df.drop(['lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
'sens_unique', 'bidirectionnel', 'route_seche',
'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
'total_sans_secu', 'total_ceinture', 'total_casque',
'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
'18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
'2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# ♦ **Pipeline de preprocessing**
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)
], remainder='passthrough')

# ♦ **Modèle de classification**
classifieur = RandomForestClassifier(random_state=42, class_weight='balanced')

# ♦ **Pipeline principal avec imblearn Pipeline**
pipeline = Pipeline([
    ('preprocessor', preprocessor), # Étape de transformation
    ('classifieur', classifieur) # Modèle final
])

# ♦ **Définition de la grille de recherche**
param_grid = {
    'classifieur__n_estimators': [50, 100, 200], # Nombre d'arbres
    'classifieur__max_depth': [10, 20, None], # Profondeur maximale
    'classifieur__class_weight': ['balanced', None] # Tester 'balanced' et None
}

# ♦ **Lancer la GridSearchCV**
grid_search = GridSearchCV(pipeline, param_grid, cv=3, scoring='f1_weighted', verbose=2, n_jobs=-1)
grid_search.fit(X_train, y_train)

# ♦ **Afficher les meilleurs paramètres**
print("Meilleurs paramètres:", grid_search.best_params_)

# ♦ **Évaluation du modèle optimal**
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

```

```

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

print("\n📊 **Performance du modèle optimal**")
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1-score: {f1:.2f}")
print("\n📋 Rapport de Classification:")
print(classification_report(y_test, y_pred))

# 🚀 **Matrice de confusion**
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y),
yticklabels=np.unique(y))
plt.xlabel("Prédictions")
plt.ylabel("Vraies valeurs")
plt.title("Matrice de Confusion")
plt.show()

```

Meilleurs paramètres: {'classifier__class_weight': None, 'classifier__max_depth': 20, 'classifier__n_estimators': 100}

📊 **Performance du modèle optimal**

Accuracy: 0.72

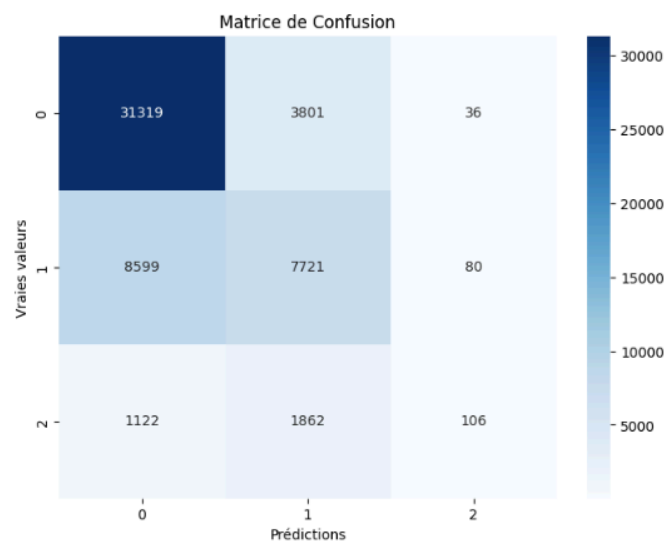
Precision: 0.69

Recall: 0.72

F1-score: 0.69

Rapport de Classification:

	precision	recall	f1-score	support
0	0.76	0.89	0.82	35156
1	0.58	0.47	0.52	16400
2	0.48	0.03	0.06	3090
accuracy			0.72	54646
macro avg	0.61	0.47	0.47	54646
weighted avg	0.69	0.72	0.69	54646



```

## Random Forest avec définition d'une métrique qui donne plus de poids à la classe 'blessé
hospitalisé' pour le F1_score, en utilisant les meilleurs paramètres retenus précédemment lors de la
gridsearch.
# Meilleurs paramètres: {'classifier__max_depth': 10, 'classifier__n_estimators': 50}

# ON va créer une métrique personnalisée qui donne plus de poids à la prédiction '3': blessé
hospitalisé

from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import f1_score
import numpy as np
from sklearn.base import BaseEstimator, TransformerMixin

# Préparation des données
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
'sens_unique', 'bidirectionnel', 'route_seche',
'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
'total_sans_secu', 'total_ceinture', 'total_casque',
'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
'18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
'2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Préparation des données
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
'tué', 'blessé_léger',
'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```

```

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche', 'route_mouillee_enneigee',
                        'etat_route_autre',
                        'usager_count', 'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant',
                        'total_gilet', 'total_airbag', 'total_gants', 'total_gants_airbag',
                        'total_autre', 'place_conducteur',
                        'pax_AV', 'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95',
                        'obstacle_fixe', 'obstacle_mobile', 'aucun_choc', 'choc_AV', 'choc_AR',
                        'choc_cote', 'choc_tonneaux',
                        'VL_VU', '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] #
À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)],
    remainder='passthrough') # Laisser les autres variables inchangées

# Définir les différentes pondérations à tester
class_weights_list = [
    {0: 1, 1: 2, 2: 1},
    {0: 1, 1: 10, 2: 5},
    {0: 1, 1: 10, 2: 10},
    {0: 1, 1: 10, 2: 5},
]

# Définir une fonction pour la précision pondérée
def weighted_f1_score(y_true, y_pred, class_weights):
    # Calculer la précision pour chaque classe
    f1_per_class = f1_score(y_true, y_pred, average=None)

    # Associer les poids aux classes dans le même ordre que dans precision_per_class
    weights = [class_weights.get(i, 1) for i in range(len(f1_per_class))]

    # Calculer la précision pondérée en fonction des poids
    weighted_f1 = np.dot(f1_per_class, weights) / sum(weights)
    return weighted_f1

# Définir la pipeline complète
pipeline = Pipeline([
    ('preprocessor', preprocessor), # Transformation des variables
    ('classifier', RandomForestClassifier(n_estimators=50, max_depth=10, random_state=42)) #
Classificateur
])

# Pour chaque configuration de poids, effectuer une validation croisée et calculer la précision
pondérée
results = {}

for class_weights in class_weights_list:
    # Utiliser cross_val_score pour évaluer la performance avec les poids de classes
    f1_scores = cross_val_score(pipeline, X_train, y_train, cv=5, scoring=lambda est, X, y:
    weighted_f1_score(y, est.predict(X), class_weights)
    )

    # Enregistrer la moyenne des scores de précision pondérée
    results[str(class_weights)] = np.mean(f1_scores)

# Afficher les résultats pour chaque configuration de pondération
for class_weights, score in results.items():
    print(f"Configuration des poids {class_weights} -> Précision pondérée : {score}")

# Trouver la configuration qui donne la meilleure précision pondérée
best_class_weights = max(results, key=results.get)
print(f"\nMeilleure configuration de pondération : {best_class_weights}")

# Créer la pipeline en utilisant les poids optimaux dans le classificateur RandomForest
pipeline_with_optimal_weights = Pipeline([
    ('preprocessor', preprocessor), # Transformation des variables
    ('classifier', RandomForestClassifier(
        n_estimators=50,
        max_depth=10,
        random_state=42,
        class_weight=eval(best_class_weights) # Appliquer les poids ici
    ))
])

# Entraîner le modèle avec les poids optimaux
pipeline_with_optimal_weights.fit(X_train, y_train)

# Faire des prédictions sur l'ensemble de test
y_pred = pipeline_with_optimal_weights.predict(X_test)

```

```

# Evaluation de la performance du modèle

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1_optimal = f1_score(y_test, y_pred, average='weighted')

print("\n📊 **Performance du modèle optimal**")
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1-score pondéré avec les poids optimaux: {f1_optimal}")
print("\nRapport de Classification:")
print(classification_report(y_test, y_pred))

# 📌 **Matrice de confusion**
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y),
            yticklabels=np.unique(y))
plt.xlabel("Prédictions")
plt.ylabel("Vraies valeurs")
plt.title("Matrice de Confusion")
plt.show()

```

```

Configuration des poids {0: 1, 1: 2, 2: 1} -> Précision pondérée : 0.44670245271593345
Configuration des poids {0: 1, 1: 10, 2: 5} -> Précision pondérée : 0.3539732055397752
Configuration des poids {0: 1, 1: 10, 2: 10} -> Précision pondérée : 0.27334463537080333

```

```

Meilleure configuration de pondération : {0: 1, 1: 2, 2: 1}

```

```

📊 **Performance du modèle optimal**
Accuracy: 0.70
Precision: 0.70
Recall: 0.70
F1-score pondéré avec les poids optimaux: 0.6825473378851216

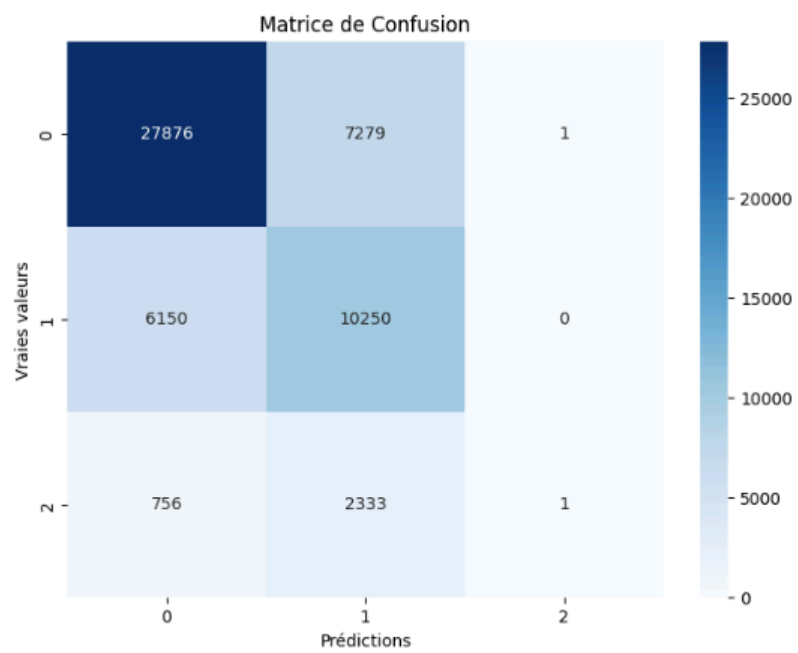
```

```

Rapport de Classification:

```

	precision	recall	f1-score	support
0	0.80	0.79	0.80	35156
1	0.52	0.62	0.57	16400
2	0.50	0.00	0.00	3090
accuracy			0.70	54646
macro avg	0.61	0.47	0.45	54646
weighted avg	0.70	0.70	0.68	54646



Random Forest avec définition d'une métrique qui donne plus de poids à la classe 'blessé hospitalisé' pour le Recall, en utilisant les meilleurs paramètres retenus précédemment lors de la gridsearch.

```
from sklearn.metrics import recall_score
```

```
# Définir une fonction pour le rappel pondéré
```

```
def weighted_recall(y_true, y_pred, class_weights):
```

```
    # Calculer le rappel pour chaque classe
```

```
    recall_per_class = recall_score(y_true, y_pred, average=None)
```

```
    # Associer les poids aux classes dans le même ordre que dans recall_per_class
```

```
    weights = [class_weights.get(i, 1) for i in range(len(recall_per_class))]
```

```
    # Calculer le rappel pondéré en fonction des poids
```

```
    weighted_recall_score = np.dot(recall_per_class, weights) / sum(weights)
```

```
    return weighted_recall_score
```

```
# Définir les différentes pondérations à tester (déjà définies dans ton code précédent)
```

```
class_weights_list = [
```

```
    {0: 1, 1: 2, 2: 1},
```

```
    {0: 1, 1: 10, 2: 5},
```

```
    {0: 1, 1: 10, 2: 10},
```

```
    {0: 1, 1: 10, 2: 5},
```

```
]
```

```
# Initialiser un dictionnaire pour stocker les résultats du rappel pondéré
```

```
results_recall = {}
```

```
# Pour chaque configuration de poids, effectuer une validation croisée et calculer le rappel pondéré
```

```
for class_weights in class_weights_list:
```

```
    # Utiliser cross_val_score pour évaluer la performance avec les poids de classes
```

```
    recall_scores = cross_val_score(
```

```
        pipeline, X_train, y_train, cv=5,
```

```
        scoring=lambda est, X, y: weighted_recall(y, est.predict(X), class_weights)
```

```
    )
```

```
    # Enregistrer la moyenne des scores de rappel pondéré
```

```
    results_recall[str(class_weights)] = np.mean(recall_scores)
```

```
# Afficher les résultats pour chaque configuration de pondération
```

```
for class_weights, score in results_recall.items():
```

```
    print(f"Configuration des poids {class_weights} -> Rappel pondéré : {score}")
```

```
# Trouver la configuration qui donne le meilleur rappel pondéré
```

```
best_class_weights_recall = max(results_recall, key=results_recall.get)
```

```
print(f"\nMeilleure configuration de pondération (Rappel) : {best_class_weights_recall}")
```

```
# Créer la pipeline en utilisant les poids optimaux dans le classificateur RandomForest
```

```
pipeline_with_optimal_weights = Pipeline([
```

```
    ('preprocessor', preprocessor), # Transformation des variables
```

```
    ('classifier', RandomForestClassifier(
```

```
        n_estimators=50,
```

```
        max_depth=10,
```

```
        random_state=42,
```

```
        class_weight=eval(best_class_weights) # Appliquer les poids ici
```

```
    ))
```

```
])
```

```
# Entraîner le modèle avec les poids optimaux
```

```
pipeline_with_optimal_weights.fit(X_train, y_train)
```

```
# Faire des prédictions sur l'ensemble de test
```

```
y_pred = pipeline_with_optimal_weights.predict(X_test)
```

```
# Evaluation de la performance du modèle
```

```
accuracy = accuracy_score(y_test, y_pred)
```

```
precision = precision_score(y_test, y_pred, average='weighted')
```

```
recall_optimal = recall_score(y_test, y_pred, average='weighted')
```

```
f1_score = f1_score(y_test, y_pred, average='weighted')
```

```
print("\n📊 **Performance du modèle optimal**")
```

```
print(f"Accuracy: {accuracy:.2f}")
```

```
print(f"Precision: {precision:.2f}")
```

```
print(f"Recall_optimal: {recall:.2f}")
```

```
print(f"F1-score: {f1_score:.2f}")
```

```
print("\nRapport de Classification:")
```

```
print(classification_report(y_test, y_pred))
```

```
# 📌 **Matrice de confusion**
```

```
conf_matrix = confusion_matrix(y_test, y_pred)
```

```
plt.figure(figsize=(8, 6))
```

```
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y),
```

```
yticklabels=np.unique(y))
```

```
plt.xlabel("Prédictions")
```

```
plt.ylabel("Vraies valeurs")
```

```
plt.title("Matrice de Confusion")
```

```
plt.show()
```

Configuration des poids {0: 1, 1: 2, 2: 1} -> Rappel pondéré : 0.43160848418020964
 Configuration des poids {0: 1, 1: 10, 2: 5} -> Rappel pondéré : 0.3113245375793031
 Configuration des poids {0: 1, 1: 10, 2: 10} -> Rappel pondéré : 0.23904561744646738

Meilleure configuration de pondération (Rappel) : {0: 1, 1: 2, 2: 1}

 ****Performance du modèle optimal****

Accuracy: 0.70

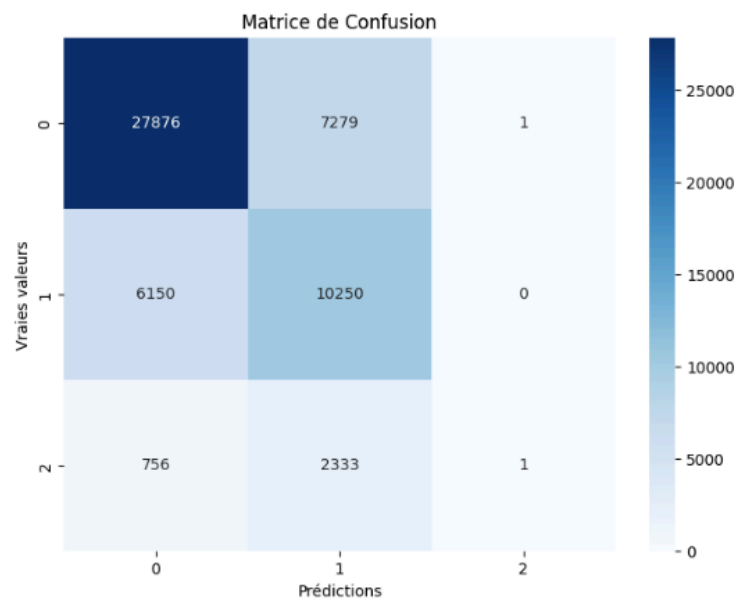
Precision: 0.70

Recall_optimal: 0.70

F1-score: 0.68

Rapport de Classification:

	precision	recall	f1-score	support
0	0.80	0.79	0.80	35156
1	0.52	0.62	0.57	16400
2	0.50	0.00	0.00	3090
accuracy			0.70	54646
macro avg	0.61	0.47	0.45	54646
weighted avg	0.70	0.70	0.68	54646



IV.1.2.d. Modèle 3 classes Régression Logistique


```

# REGRESSION LOGISTIQUE
# Préparation des données (Exemple de df à adapter)
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
            'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche',
                        'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
                        'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète avec la régression logistique
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(max_iter=1000, random_state=42))] # Régression Logistique avec
itérations supplémentaires

# Entraîner le modèle
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

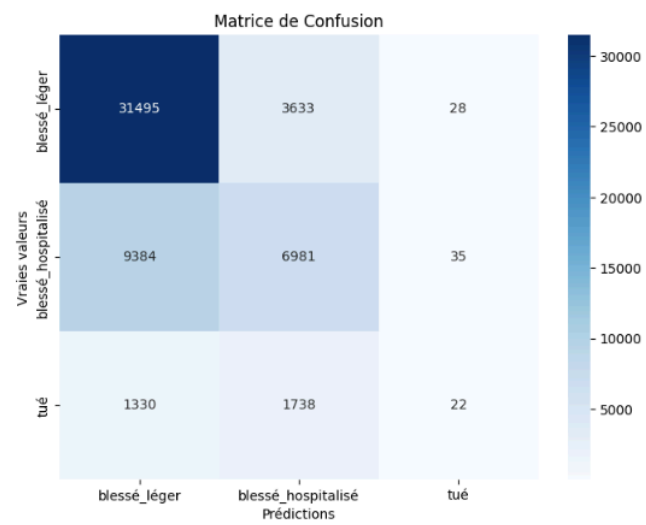
# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=
['blessé_léger', 'blessé_hospitalisé', 'tué'], yticklabels=['blessé_léger', 'blessé_hospitalisé', 'tué'])
plt.xlabel('Prédictions')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

```

Accuracy du modèle : 0.70
Précision : 0.66
Rappel : 0.70
F1-score : 0.67

Rapport de Classification :

	precision	recall	f1-score	support
0	0.75	0.90	0.81	35156
1	0.57	0.43	0.49	16400
2	0.26	0.01	0.01	3090
accuracy			0.70	54646
macro avg	0.52	0.44	0.44	54646
weighted avg	0.66	0.70	0.67	54646



```

# Logistic Régression avec oversampling
# Préparation des données (Exemple de df à adapter)
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche',
                        'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
                        'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète avec SMOTE (oversampling) et la régression logistique
pipeline = imPipeline([
    ('preprocessor', preprocessor), # Prétraitement des données
    ('smote', SMOTE(random_state=42)), # Oversampling avec SMOTE
    ('classifieur', LogisticRegression(max_iter=1000, random_state=42)) # Régression logistique
])

# Entraîner le modèle
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle avec oversampling : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

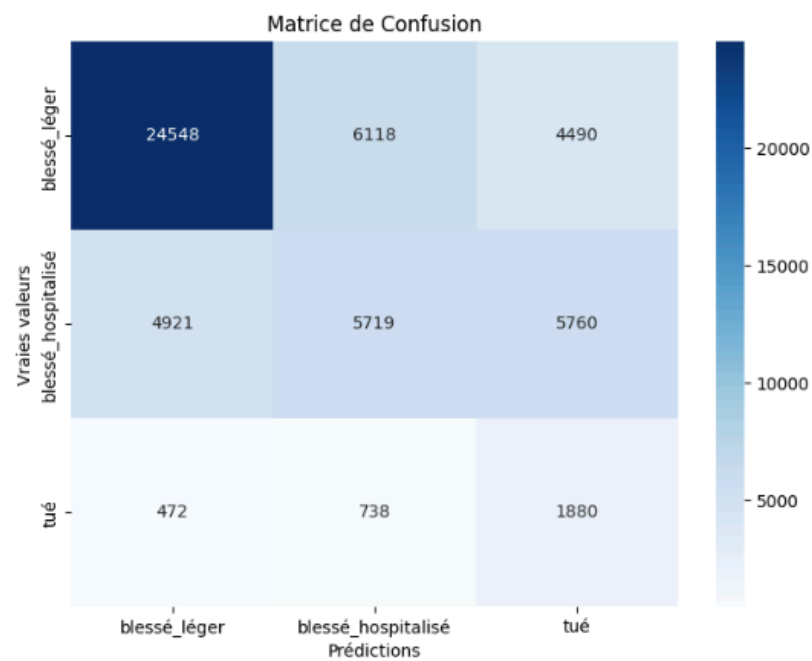
# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=
            ['blessé_léger', 'blessé_hospitalisé', 'tué'], yticklabels=['blessé_léger', 'blessé_hospitalisé', 'tué'])
plt.xlabel('Prédictions')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

```

Accuracy du modèle avec oversampling : 0.59
Précision : 0.67
Rappel : 0.59
F1-score : 0.62

Rapport de Classification :

	precision	recall	f1-score	support
0	0.82	0.70	0.75	35156
1	0.45	0.35	0.39	16400
2	0.15	0.61	0.25	3090
accuracy			0.59	54646
macro avg	0.48	0.55	0.47	54646
weighted avg	0.67	0.59	0.62	54646



```

# Logistic Régression avec UnderSampling
# Préparation des données (Exemple de df à adapter)
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche',
                        'route_mouillee_enneigee', 'etat_route_autre', 'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
                        'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète avec RandomUnderSampler (undersampling) et la régression logistique
pipeline = imPipeline([
    ('preprocessor', preprocessor), # Prétraitement des données
    ('undersample', RandomUnderSampler(random_state=42)), # Undersampling avec RandomUnderSampler
    ('classifier', LogisticRegression(max_iter=1000, random_state=42)) # Régression logistique
])

# Entraîner le modèle
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle avec undersampling : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

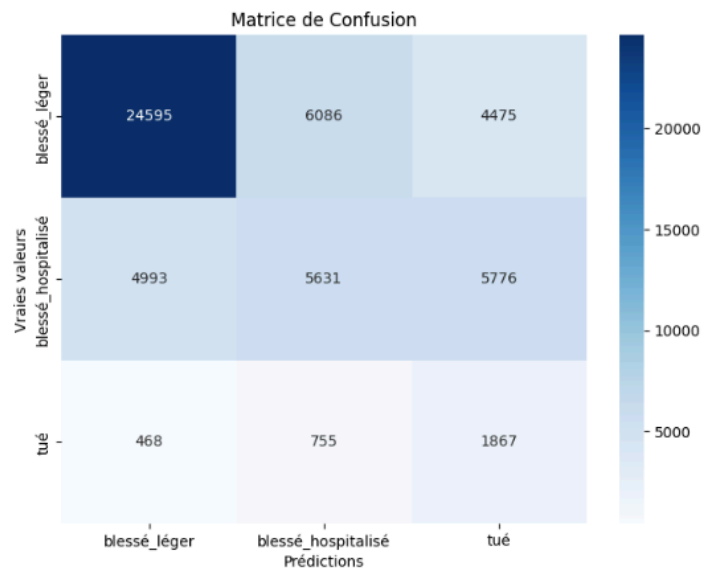
# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=
            ['blessé_léger', 'blessé_hospitalisé', 'tué'], yticklabels=
            ['blessé_léger', 'blessé_hospitalisé', 'tué'])
plt.xlabel('Prédictions')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

```

Accuracy du modèle avec undersampling : 0.59
Précision : 0.67
Rappel : 0.59
F1-score : 0.62

Rapport de Classification :

	precision	recall	f1-score	support
0	0.82	0.70	0.75	35156
1	0.45	0.34	0.39	16400
2	0.15	0.60	0.25	3090
accuracy			0.59	54646
macro avg	0.47	0.55	0.46	54646
weighted avg	0.67	0.59	0.62	54646



```

# Gridsearch pour optimiser les paramètres du modèle, avec class_weight = 'balanced' pour gérer le
déséquilibre des classes

# Préparation des données (Exemple de df à adapter)
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
'sens_unique', 'bidirectionnel', 'route_seche',
'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
'total_sans_secu', 'total_ceinture', 'total_casque',
'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
'18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
'2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète avec la régression logistique
pipeline = Pipeline([
    ('preprocessor', preprocessor), # Prétraitement des données
    ('classifier', LogisticRegression(max_iter=1000, random_state=42, class_weight = 'balanced')) #
Régression logistique
])

# Paramètres à tester dans GridSearch
param_grid = {
    'classifier__C': [0.01, 0.1, 1, 10, 100], # Paramètre C de la régression logistique
    'classifier__solver': ['liblinear', 'saga'], # Différents solveurs pour la régression logistique
}

# Définir GridSearchCV pour tester les différentes configurations
grid_search = GridSearchCV(
    pipeline, param_grid, cv=5, scoring='accuracy', n_jobs=-1, verbose=1)

# Entraîner le modèle avec GridSearch
grid_search.fit(X_train, y_train)

# Afficher les meilleurs paramètres trouvés
print("Meilleurs paramètres :")
print(grid_search.best_params_)

# Prédictions
y_pred = grid_search.predict(X_test)

# ♦ **Évaluation du modèle optimal**
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

print("\n📊 **Performance du modèle optimal**")
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall: {recall:.2f}")
print(f"F1-score: {f1:.2f}")
print("\nRapport de Classification:")
print(classification_report(y_test, y_pred))

```

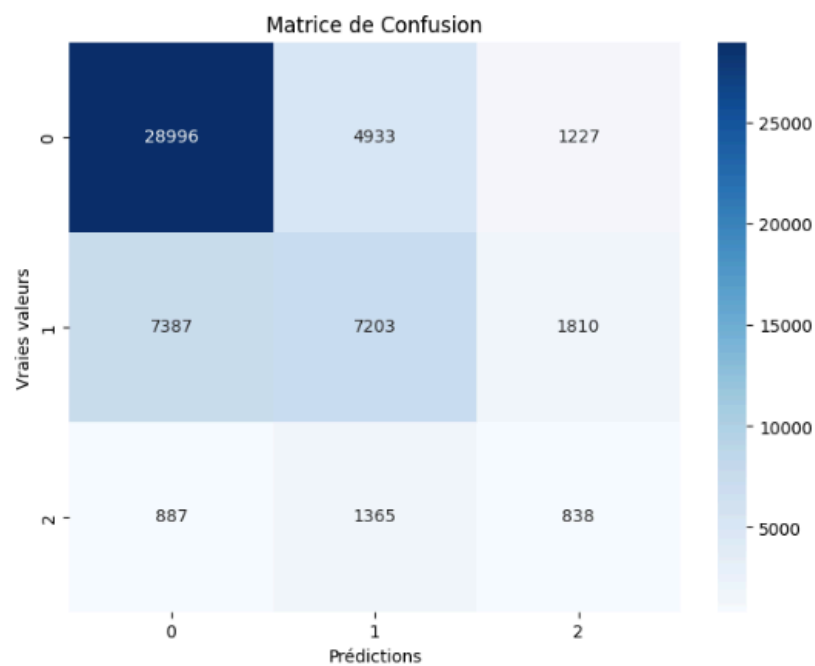
```
Meilleurs paramètres :
{'classifier__C': 0.01, 'classifier__solver': 'liblinear'}
```

```
 **Performance du modèle optimal**
```

```
Accuracy: 0.68
Precision: 0.67
Recall: 0.68
F1-score: 0.67
```

```
Rapport de Classification:
```

	precision	recall	f1-score	support
0	0.78	0.82	0.80	35156
1	0.53	0.44	0.48	16400
2	0.22	0.27	0.24	3090
accuracy			0.68	54646
macro avg	0.51	0.51	0.51	54646
weighted avg	0.67	0.68	0.67	54646



IV.1.2.e. Modèle 3 classes KNN


```

# Modèle KNN avec Gridsearch

# Importation du df_machine_learning

import numpy as np
import pandas as pd
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix,
classification_report
import matplotlib.pyplot as plt
import seaborn as sns

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float)
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos]

# Chargement des données (Assurez-vous que df est défini)
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
            'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Séparation des données en ensemble d'entraînement et de test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Définition des caractéristiques
categorical_features = ['agg', 'atm']
cyclical_features = ['heure']
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche', 'route_mouillee_enneigee', 'etat_route_autre',
                        'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque', 'total_secu_enfant', 'total_gilet',
                        'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV', 'pax_AR', 'pax_Milieu',
                        'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc', 'choc_AV', 'choc_AR',
                        'choc_cote', 'choc_tonneaux', 'VL_VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh']

# Transformation des données
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features),
    ('num', StandardScaler(), passthrough_features)
])

# Définition du modèle KNN avec GridSearchCV
knn = KNeighborsClassifier()
param_grid = {'classifier__n_neighbors': [3, 5, 7, 9], 'classifier__weights': ['uniform', 'distance']}

# Pipeline avec le classificateur KNN
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', knn)
])

# GridSearch pour trouver les meilleurs hyperparamètres
grid_search = GridSearchCV(pipeline, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid_search.fit(X_train, y_train)

# Meilleur modèle trouvé
best_model = grid_search.best_estimator_
print(f"Meilleurs paramètres : {grid_search.best_params_}")

# Prédictions
y_pred = best_model.predict(X_test)

```

```

# Évaluation du modèle
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')
conf_matrix = confusion_matrix(y_test, y_pred)
class_report = classification_report(y_test, y_pred)

print(f"Accuracy : {accuracy:.2f}")
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

# Affichage de la matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['léger', 'hospitalisé', 'tué'], yticklabels=['léger', 'hospitalisé', 'tué'])
plt.xlabel('Prédictions')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

```

```

Meilleurs paramètres : {'classifier__n_neighbors': 9, 'classifier__weights': 'uniform'}
Accuracy : 0.69
Précision : 0.66
Rappel : 0.69
F1-score : 0.66

```

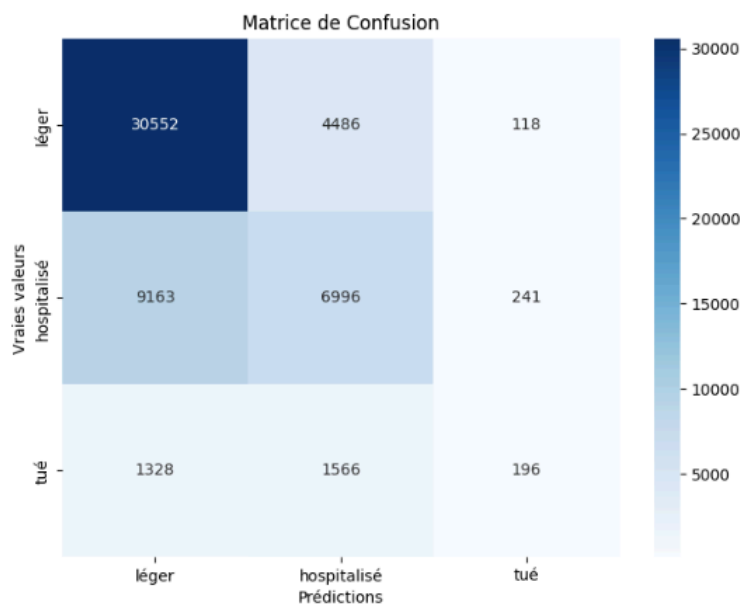
```

Rapport de Classification :
              precision    recall  f1-score   support

      0       0.74       0.87       0.80       35156
      1       0.54       0.43       0.48       16400
      2       0.35       0.06       0.11        3090

 accuracy          0.69
  macro avg       0.54       0.45       0.46
 weighted avg     0.66       0.69       0.66

```



```

#KNN avec une métrique make_scorer pour améliorer le F1 score et Gridsearch
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import f1_score, make_scorer, confusion_matrix, classification_report
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.neighbors import KNeighborsClassifier

# =====
# Transformation des Données =====
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float)
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Chargement des données (remplace par ton DataFrame réel)
df = pd.read_csv("ton_fichier.csv") # Remplace par le vrai fichier

# Séparation des features et du target
X = df.drop(['lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger',
            'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Décalage des labels (classes 2,3,4 deviennent 0,1,2)
y = y - 2

# Séparation en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Variables catégorielles et cycliques
categorical_features = ['agg', 'atm']
cyclical_features = ['heure']
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche', 'route_mouillée_enneigée',
                        'etat_route_autre',
                        'usager_count', 'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant',
                        'total_gilet', 'total_airbag', 'total_gants', 'total_gants_airbag',
                        'total_autre',
                        'place_conducteur', 'pax_AV', 'pax_AR', 'pax_Milieu', 'place_pieton', 'homme',
                        'femme',
                        '0-17', '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV',
                        'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU', '2roues_3roues_quad', 'PL',
                        'bus_car',
                        'velo_trott_edp', 'nbr_veh']

# Transformation des données
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features),
    ('scaler', StandardScaler(), passthrough_features) # Normalisation pour KNN
])

X_train_transformed = preprocessor.fit_transform(X_train)
X_test_transformed = preprocessor.transform(X_test)

```

```

# Application de SMOTE pour équilibrer les classes
# =====
smote = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = smote.fit_resample(X_train_transformed, y_train)

# Définition de la Métrique Personnalisée (F1-score pour une classe cible)
# =====
def f1_class_specific(y_true, y_pred, target_class=1):
    """
    Fonction pour calculer le F1-score d'une classe spécifique.
    """
    y_pred = (y_pred == target_class).astype(int)
    y_true = (y_true == target_class).astype(int)
    return f1_score(y_true, y_pred)

# Scorer pour GridSearchCV
f1_scorer = make_scorer(f1_class_specific, greater_is_better=True, target_class=1)

# Entraînement du Modèle KNN avec GridSearchCV
# =====
knn_model = KNeighborsClassifier()

param_grid = {
    'n_neighbors': [3, 5, 7, 9],
    'weights': ['uniform', 'distance'],
    'metric': ['euclidean', 'manhattan', 'minkowski']
}

grid_search = GridSearchCV(
    knn_model, param_grid, scoring=f1_scorer,
    cv=3, n_jobs=-1, verbose=1
)

grid_search.fit(X_train_balanced, y_train_balanced)

# Meilleurs hyperparamètres
best_params = grid_search.best_params_
print("Meilleurs hyperparamètres :", best_params)

# Évaluation sur l'Ensemble de Test
# =====
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test_transformed)

# Calcul du F1-score pour la classe cible
f1_test = f1_score(y_test == 1, y_pred == 1)
print(f"F1-score de la classe cible (accidents graves) sur test : {f1_test:.4f}")

# Affichage de la Matrice de Confusion
# =====
conf_matrix = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(6,6))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", xticklabels=[0,1,2], yticklabels=[0,1,2])
plt.xlabel("Prédiction")
plt.ylabel("Vraies Classes")
plt.title("Matrice de Confusion - KNN")
plt.show()

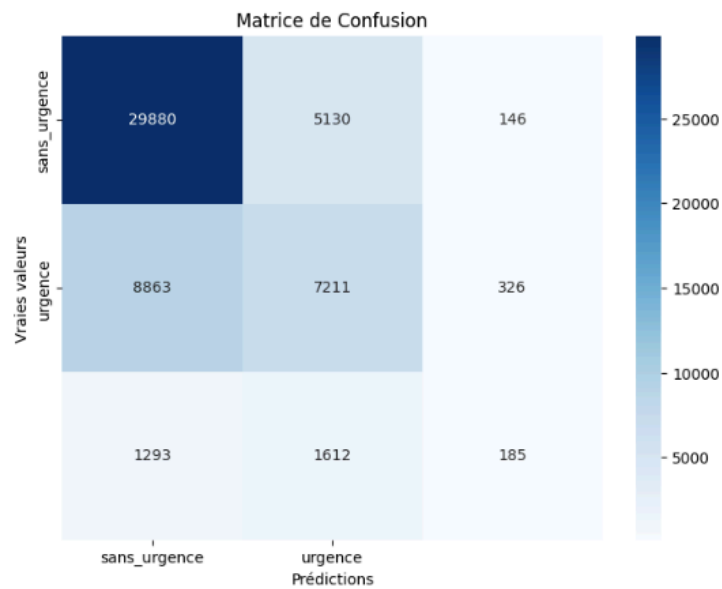
# Rapport de classification complet
print(classification_report(y_test, y_pred, target_names=["léger", "hospitalisé", "tué"]))

```

F1-score moyen pour la classe 1 : 0.4686
 F1-score de la classe 1 sur l'ensemble de test : 0.4751

Rapport de Classification :

	precision	recall	f1-score	support
0	0.75	0.85	0.79	35156
1	0.52	0.44	0.48	16400
2	0.28	0.06	0.10	3090
accuracy			0.68	54646
macro avg	0.51	0.45	0.46	54646
weighted avg	0.65	0.68	0.66	54646



IV.1.2.f. Modèle 3 classes XGBoost

```

# XGBoost avec oversampling
from imblearn.over_sampling import SMOTE
from sklearn.pipeline import Pipeline

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparation des variables d'entrée et de sortie
X = df.drop(['lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger',
            'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Décalage des labels pour correspondre à l'attente d'XGBoost ([2,3,4] -> [0,1,2])
y = y - 2

# Séparation en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Variables à encoder
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche', 'route_mouillee_enneigee',
                        'etat_route_autre',
                        'usager_count', 'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant',
                        'total_gilet', 'total_airbag', 'total_gants', 'total_gants_airbag',
                        'total_autre',
                        'place_conducteur', 'pax_AV', 'pax_AR', 'pax_Milieu', 'place_pieton', 'homme',
                        'femme',
                        '0-17', '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV',
                        'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU', '2roues_3roues_quad', 'PL',
                        'bus_car',
                        'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer pour transformer les variables
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)
], remainder='passthrough')

# Transformation des données d'entraînement
X_train_transformed = preprocessor.fit_transform(X_train)
X_test_transformed = preprocessor.transform(X_test)

# Application de SMOTE pour équilibrer les classes
smote = SMOTE(sampling_strategy='auto', random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train_transformed, y_train)

# Création et entraînement du modèle XGBoost
model = XGBClassifier(
    objective='multi:softmax', # Classification multiclasses
    num_class=3, # Nombre de classes
    eval_metric='mlogloss', # Log loss
    use_label_encoder=False, # Suppression d'un avertissement lié à XGBoost
    random_state=42
)
model.fit(X_train_resampled, y_train_resampled)

# Prédiction (on reconvertit les classes en [2,3,4])
y_pred = model.predict(X_test_transformed) + 2

# Évaluation du modèle
print(f"Accuracy du modèle : {accuracy_score(y_test + 2, y_pred):.2f}")
print("\nRapport de classification :")
print(classification_report(y_test + 2, y_pred))

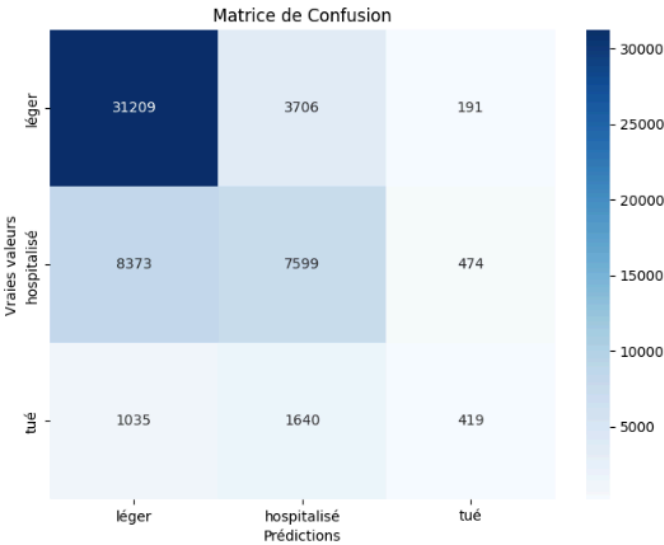
# Visualisation de la matrice de confusion
conf_matrix = confusion_matrix(y_test + 2, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['léger', 'hospitalisé', 'tué'],
            yticklabels=['léger', 'hospitalisé', 'tué'])
plt.xlabel('Prédictions')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

```

Accuracy du modèle : 0.72

Rapport de classification :

	precision	recall	f1-score	support
2	0.77	0.89	0.82	35106
3	0.59	0.46	0.52	16446
4	0.39	0.14	0.20	3094
accuracy			0.72	54646
macro avg	0.58	0.50	0.51	54646
weighted avg	0.69	0.72	0.70	54646



```

# XGBOOST avec l'argument classweight pour gérer le déséquilibre des classes
# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparation des variables d'entrée et de sortie
X = df.drop(['lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger',
            'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Vérification des classes uniques
print("Classes uniques avant modification:", np.unique(y))

# Décalage des labels pour correspondre à l'attente d'XGBoost ([2,3,4] -> [0,1,2])
y = y - 2

# Vérification après transformation
print("Classes uniques après transformation:", np.unique(y))

# Séparation en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Variables à encoder
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche', 'route_mouillee_enneigee',
                        'etat_route_autre',
                        'usager_count', 'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant',
                        'total_gilet', 'total_airbag', 'total_gants', 'total_gants_airbag',
                        'total_autre',
                        'place_conducteur', 'pax_AV', 'pax_AR', 'pax_Milieu', 'place_pieton', 'homme',
                        'femme',
                        '0-17', '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV',
                        'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU', '2roues_3roues_quad', 'PL',
                        'bus_car',
                        'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer pour transformer les variables
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)
], remainder='passthrough')

# Calculer les poids des classes (pour gérer le déséquilibre)
class_weights = compute_sample_weight(class_weight='balanced', y=y_train)
print("Poids des classes calculés avec 'balanced' :", class_weights)

# Créer et entraîner le modèle XGBoost
model = XGBClassifier(
    objective='multi:softmax', # Classification multiclasses
    num_class=3, # Nombre de classes
    eval_metric='mlogloss', # Log loss
    use_label_encoder=False, # Suppression d'un avertissement lié à XGBoost
    random_state=42
)

# Appliquer la transformation de prétraitement sur les données d'entraînement
X_train_transformed = preprocessor.fit_transform(X_train)
X_test_transformed = preprocessor.transform(X_test)

# Entraîner le modèle avec les poids de classes
model.fit(X_train_transformed, y_train, sample_weight=class_weights)

# Prédiction (on reconvertit les classes en [2,3,4])
y_pred = model.predict(X_test_transformed) + 2

# Évaluation du modèle
accuracy = accuracy_score(y_test + 2, y_pred) # On ajoute 2 à y_test pour correspondre aux vraies classes
print(f"Accuracy du modèle : {accuracy:.2f}")

# Rapport de classification détaillé
print("\nRapport de classification :")
print(classification_report(y_test + 2, y_pred))

```



```

# Optimisation des hyperparamètres et classe weight

from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import classification_report, confusion_matrix
from xgboost import XGBClassifier
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.pipeline import Pipeline
from sklearn.base import BaseEstimator, TransformerMixin

# Définition de la transformation cyclique pour les variables horaires
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float)
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos]

# Séparation des variables d'entrée et de sortie
X = df.drop(['lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger',
            'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Décalage des labels pour correspondre à l'attente d'XGBoost ([2,3,4] -> [0,1,2])
y = y - 2

# Séparation en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Liste des variables catégorielles et numériques
categorical_features = ['agg', 'atm']
cyclical_features = ['heure']
numerical_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                    'sens_unique', 'bidirectionnel', 'route_seche', 'route_mouillee_enneigee',
                    'etat_route_autre',
                    'usager_count', 'total_sans_secu', 'total_ceinture', 'total_casque',
                    'total_secu_enfant',
                    'total_gilet', 'total_airbag', 'total_gants', 'total_gants_airbag',
                    'total_autre',
                    'place_conducteur', 'pax_AV', 'pax_AR', 'pax_Milieu', 'place_pieton', 'homme',
                    'femme',
                    '0-17', '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                    'choc_AV',
                    'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU', '2roues_3roues_quad', 'PL',
                    'bus_car',
                    'velo_trott_edp', 'nbr_veh']

# Pipeline de transformation
preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
        ('cyclical', CyclicalFeatures(), cyclical_features),
        ('num', StandardScaler(), numerical_features)
    ],
    remainder='passthrough' # Conserve les autres variables sans modification
)

# Construction du pipeline avec XGBoost
pipeline = Pipeline(steps=[
    ('preprocessor', preprocessor),
    ('classifier', XGBClassifier(objective='multi:softmax', num_class=3, eval_metric='mlogloss',
                                random_state=42))
])

# Définition de la grille d'hyperparamètres pour GridSearchCV
param_grid = {
    'classifier__n_estimators': [100, 200, 300],
    'classifier__max_depth': [3, 6, 10],
    'classifier__scale_pos_weight': [1, 2, 5] # Gestion du déséquilibre des classes
}

# Recherche des meilleurs paramètres avec GridSearchCV
grid_search = GridSearchCV(pipeline, param_grid, cv=3, scoring='accuracy', n_jobs=-1, verbose=1)

# Entraîner le modèle avec la recherche en grille
grid_search.fit(X_train, y_train)

```



```
# Meilleurs paramètres et précision
print("Meilleurs paramètres trouvés : ", grid_search.best_params_)
print("Meilleure précision sur l'ensemble d'entraînement : ", grid_search.best_score_)

# Prédiction sur l'ensemble de test
y_pred = grid_search.predict(X_test)

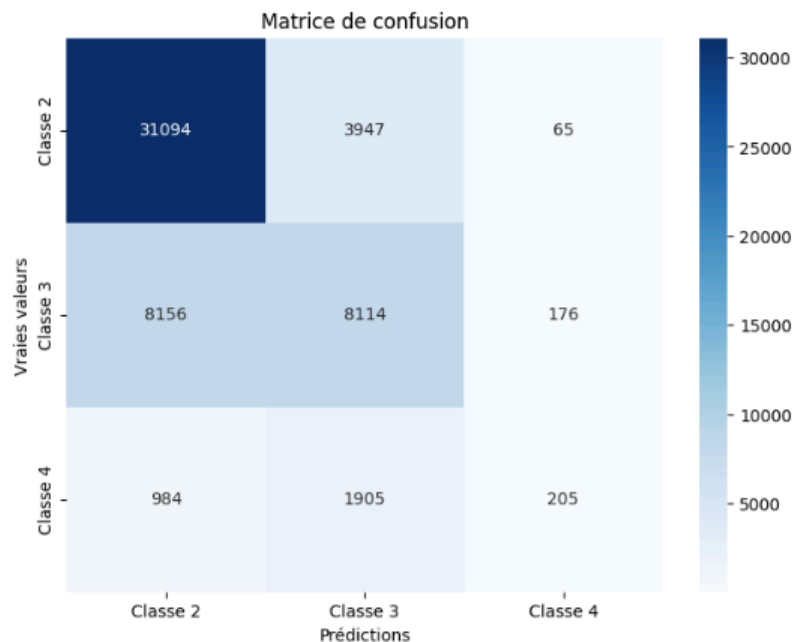
# Rapport de classification
print("\nRapport de classification :")
print(classification_report(y_test, y_pred))

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['Classe 2', 'Classe 3', 'Classe 4'], yticklabels=['Classe 2', 'Classe 3', 'Classe 4'])
plt.xlabel('Prédictions')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de confusion')
plt.show()
```

Meilleurs paramètres trouvés : {'classifier__max_depth': 6, 'classifier__n_estimators': 100, 'classifier__scale_pos_weight': 1}
Meilleure précision sur l'ensemble d'entraînement : 0.7177006130478544

Rapport de classification :

	precision	recall	f1-score	support
0	0.77	0.89	0.83	35106
1	0.58	0.49	0.53	16446
2	0.46	0.07	0.12	3094
accuracy			0.72	54646
macro avg	0.60	0.48	0.49	54646
weighted avg	0.70	0.72	0.70	54646



```

# XGB00ST Avec scoring personnalisé pour optimiser le F1_score de la classe 1
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import xgboost as xgb
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import f1_score, make_scorer, confusion_matrix, classification_report
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.base import BaseEstimator, TransformerMixin
from xgboost import XGBClassifier

# =====
# Transformation des Données
# =====
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float)
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparation des features et du target
X = df.drop(['lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger',
            'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Décalage des labels (classes 2,3,4 deviennent 0,1,2)
y = y - 2

# Séparation en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Variables catégorielles et cycliques
categorical_features = ['agg', 'atm']
cyclical_features = ['heure']
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche', 'route_mouvillée_enneigée',
                        'etat_route_autre',
                        'usager_count', 'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant',
                        'total_gilet', 'total_airbag', 'total_gants', 'total_gants_airbag',
                        'total_autre',
                        'place_conducteur', 'pax_AV', 'pax_AR', 'pax_Milieu', 'place_pieton', 'homme',
                        'femme',
                        '0-17', '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV',
                        'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU', '2roues_3roues_quad', 'PL',
                        'bus_car',
                        'velo_trott_edp', 'nbr_veh']

# Transformation des données
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)
], remainder='passthrough')

X_train_transformed = preprocessor.fit_transform(X_train)
X_test_transformed = preprocessor.transform(X_test)

# =====
# Équilibrage des Classes avec SMOTE
# =====
smote = SMOTE(random_state=42)
X_train_balanced, y_train_balanced = smote.fit_resample(X_train_transformed, y_train)

# =====
# Définition de la Métrique Personnalisée (F1-score pour une classe cible)
# =====
def f1_xgb(preds, dtrain, target_class=1):
    labels = dtrain.get_label()
    preds = np.argmax(preds.reshape(len(labels), -1), axis=1)
    f1 = f1_score(labels == target_class, preds == target_class)
    return f'f1_class_{target_class}', f1

```

```

# Fonction pour GridSearchCV
def f1_class_specific(y_true, y_pred, target_class=1):
    y_pred = (y_pred == target_class).astype(int)
    y_true = (y_true == target_class).astype(int)
    return f1_score(y_true, y_pred)

f1_scorer = make_scorer(f1_class_specific, greater_is_better=True, target_class=1)

# =====
# 4 Entraînement du Modèle XGBoost avec GridSearchCV
# =====
xgb_model = XGBClassifier(
    objective="multi:softmax",
    num_class=3,
    eval_metric="mlogloss",
    use_label_encoder=False
)

param_grid = {
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.1, 0.2],
    'n_estimators': [50, 100, 200]
}

grid_search = GridSearchCV(
    xgb_model, param_grid, scoring=f1_scorer,
    cv=3, n_jobs=-1, verbose=1
)

grid_search.fit(X_train_balanced, y_train_balanced)

# Meilleurs hyperparamètres
best_params = grid_search.best_params_
print("Meilleurs hyperparamètres :", best_params)

# =====
# 5 Évaluation sur l'Ensemble de Test
# =====
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test_transformed)

# Calcul du F1-score pour la classe cible
f1_test = f1_score(y_test == 1, y_pred == 1)
print(f"F1-score de la classe cible (accidents graves) sur test : {f1_test:.4f}")

# =====
# 6 Matrice de Confusion et Rapport de Classification
# =====
conf_matrix = confusion_matrix(y_test, y_pred)

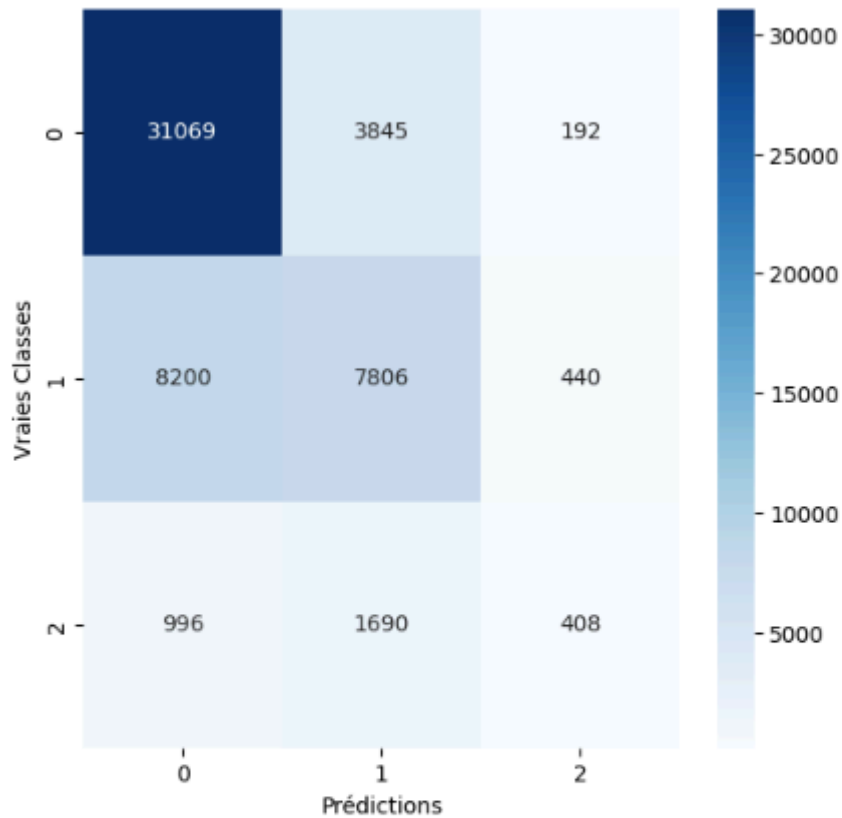
plt.figure(figsize=(6,6))
sns.heatmap(conf_matrix, annot=True, fmt="d", cmap="Blues", xticklabels=[0,1,2], yticklabels=[0,1,2])
plt.xlabel("Prédictions")
plt.ylabel("Vraies Classes")
plt.title("Matrice de Confusion - XGBoost avec GridSearchCV")
plt.show()

# Rapport de classification
print("\nRapport de Classification :")
print(classification_report(y_test, y_pred, target_names=["léger", "hospitalisé", "tué"]))

```

Meilleurs hyperparamètres : {'learning_rate': 0.2, 'max_depth': 7, 'n_estimators': 200}
F1-score de la classe cible (accidents graves) sur test : 0.5241

Matrice de Confusion - XGBoost avec GridSearchCV



Rapport de Classification :

	precision	recall	f1-score	support
léger	0.77	0.89	0.82	35106
hospitalisé	0.59	0.47	0.52	16446
tué	0.39	0.13	0.20	3094
accuracy			0.72	54646
macro avg	0.58	0.50	0.52	54646
weighted avg	0.69	0.72	0.70	54646

IV.1.2.g. Modèle 2 classes Random Forest

```

# Random Forest simple
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
            'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche',
                        'route_mouillee_enneigee', 'etat_route_autre', 'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
                        'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42))])

# Entraîner le modèle
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['Classe 0', 'Classe 1'],
            yticklabels=['Classe 0', 'Classe 1'])
plt.xlabel('Prédiction')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

```



```
# Récupérer l'importance des features
# Récupérer le modèle du classifieur après l'entraînement
model = pipeline.named_steps['classifieur']

# Accéder aux importances des features
importances = model.feature_importances_

# Récupérer les noms des features après transformation
# Utiliser l'encodeur OneHotEncoder pour gérer les variables catégorielles encodées
# et la transformation cyclique pour obtenir les noms des features résultants.
cat_columns = pipeline.named_steps['preprocessor'].transformers_[0]
[1].get_feature_names_out(categorical_features)
cyclical_columns = ['heure_sin', 'heure_cos'] # Ces noms sont définis par la transformation de l'heure

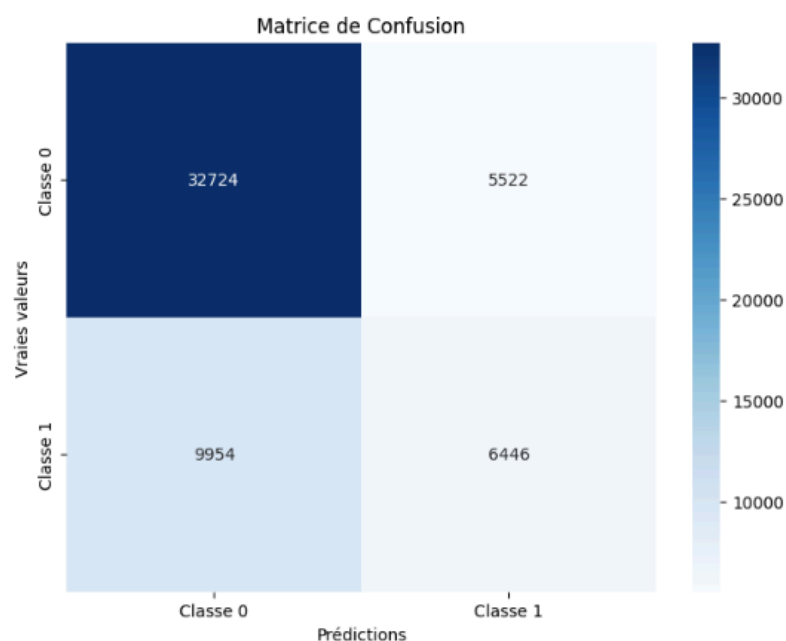
# Les noms des features après transformation
features = np.concatenate([cat_columns, cyclical_columns, passthrough_features])

# Tracer l'importance des features
plt.figure(figsize=(12, 12))
plt.barh(features, importances)
plt.xlabel("Importance des Features")
plt.title("Importance des features dans le modèle Random Forest avec oversampling")
plt.show()
```

Accuracy du modèle : 0.72
Précision : 0.70
Rappel : 0.72
F1-score : 0.70

Rapport de Classification :

	precision	recall	f1-score	support
sans_urgence	0.77	0.86	0.81	38246
urgence	0.54	0.39	0.45	16400
accuracy			0.72	54646
macro avg	0.65	0.62	0.63	54646
weighted avg	0.70	0.72	0.70	54646



```

# Random Forest avec oversampling
# Préparation des données
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
'sens_unique', 'bidirectionnel', 'route_seche',
'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
'total_sans_secu', 'total_ceinture', 'total_casque',
'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
'18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
'2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Appliquer SMOTE dans la pipeline (avant l'entraînement du modèle)
smote = SMOTE(random_state=42, sampling_strategy = 'auto', k_neighbors = 5)

# Définir la pipeline complète avec SMOTE
pipeline = imPipeline([
    ('preprocessor', preprocessor), # Transformation des variables
    ('smote', smote), # Oversampling via SMOTE
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42)) # Classificateur
])

# Entraîner le modèle avec oversampling
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle avec oversampling : {accuracy:.2f}")

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle : {accuracy:.2f}")

```



```

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['Classe 0', 'Classe 1'],
            yticklabels=['Classe 0', 'Classe 1'])
plt.xlabel('Prédictions')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

# Récupérer l'importance des features
# Récupérer le modèle du classifieur après l'entraînement
model = pipeline.named_steps['classifieur']

# Accéder aux importances des features
importances = model.feature_importances_

# Récupérer les noms des features après transformation
# Utiliser l'encodeur OneHotEncoder pour gérer les variables catégorielles encodées
# et la transformation cyclique pour obtenir les noms des features résultants.
cat_columns = pipeline.named_steps['preprocessor'].transformers_[0]
[1].get_feature_names_out(categorical_features)
cyclical_columns = ['heure_sin', 'heure_cos'] # Ces noms sont définis par la transformation de l'heure

# Les noms des features après transformation
features = np.concatenate([cat_columns, cyclical_columns, passthrough_features])

# Tracer l'importance des features
plt.figure(figsize=(12, 12))
plt.barh(features, importances)
plt.xlabel("Importance des Features")
plt.title("Importance des features dans le modèle Random Forest avec oversampling")
plt.show()

```

```

Accuracy du modèle avec oversampling : 0.71
Accuracy du modèle : 0.71
Précision : 0.70
Rappel : 0.71
F1-score : 0.71

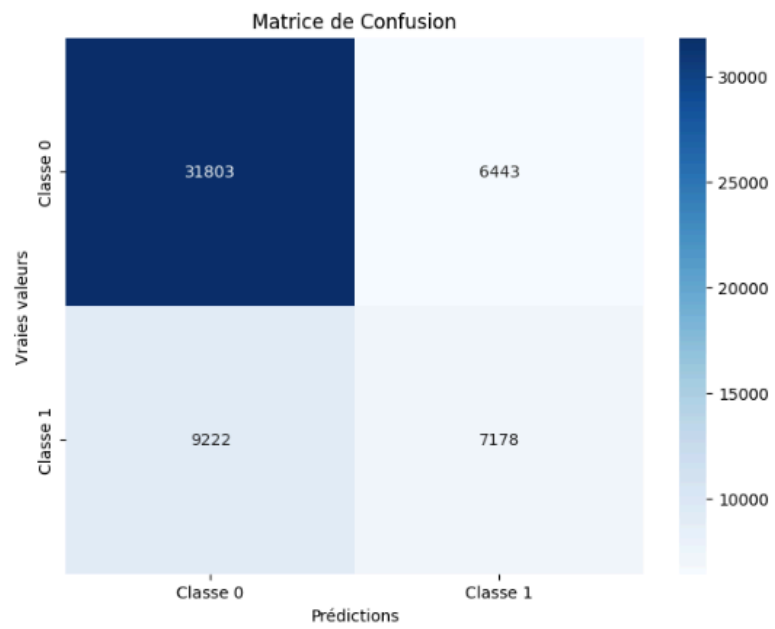
```

```

Rapport de Classification :

```

	precision	recall	f1-score	support
sans_urgence	0.78	0.83	0.80	38246
urgence	0.53	0.44	0.48	16400
accuracy			0.71	54646
macro avg	0.65	0.63	0.64	54646
weighted avg	0.70	0.71	0.71	54646



```

# Random Forest avec Undersampling
# Préparation des données
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
'sens_unique', 'bidirectionnel', 'route_seche',
'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
'total_sans_secu', 'total_ceinture', 'total_casque',
'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
'18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
'2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Appliquer RandomUnderSampler dans la pipeline (avant l'entraînement du modèle)
under_sampler = RandomUnderSampler(random_state=42)

# Définir la pipeline complète avec undersampling
pipeline = imPipeline([
    ('preprocessor', preprocessor), # Transformation des variables
    ('under_sampler', under_sampler), # Undersampling via RandomUnderSampler
    ('classifier', RandomForestClassifier(n_estimators=100, random_state=42)) # Classificateur
])

# Entraîner le modèle avec undersampling
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle avec undersampling : {accuracy:.2f}")

```

```

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['Classe 0', 'Classe 1'],
            yticklabels=['Classe 0', 'Classe 1'])
plt.xlabel('Prédictions')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

# Récupérer l'importance des features
# Récupérer le modèle du classifieur après l'entraînement
model = pipeline.named_steps['classifieur']

# Accéder aux importances des features
importances = model.feature_importances_

# Récupérer les noms des features après transformation
# Utiliser l'encodeur OneHotEncoder pour gérer les variables catégorielles encodées
# et la transformation cyclique pour obtenir les noms des features résultants.
cat_columns = pipeline.named_steps['preprocessor'].transformers_[0]
[1].get_feature_names_out(categorical_features)
cyclical_columns = ['heure_sin', 'heure_cos'] # Ces noms sont définis par la transformation de l'heure

# Les noms des features après transformation
features = np.concatenate([cat_columns, cyclical_columns, passthrough_features])

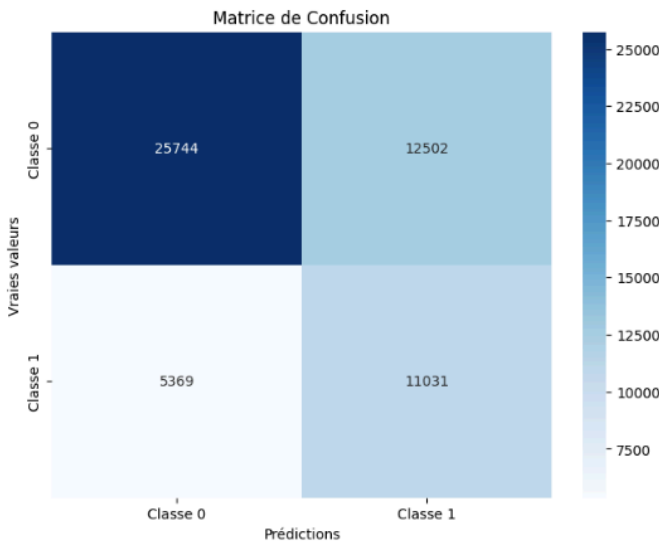
# Tracer l'importance des features
plt.figure(figsize=(12, 12))
plt.barh(features, importances)
plt.xlabel("Importance des Features")
plt.title("Importance des features dans le modèle Random Forest avec undersampling")

```

Accuracy du modèle avec undersampling : 0.67
Précision : 0.72
Rappel : 0.67
F1-score : 0.69

Rapport de Classification :

	precision	recall	f1-score	support
sans_urgence	0.83	0.67	0.74	38246
urgence	0.47	0.67	0.55	16400
accuracy			0.67	54646
macro avg	0.65	0.67	0.65	54646
weighted avg	0.72	0.67	0.69	54646



```

# Grid search pour optimiser paramètres du modèle avec toutes les variables. Utilisation de
class_weight='balanced' pour gérer le déséquilibre des classes
X = df.drop(['lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
'sens_unique', 'bidirectionnel', 'route_seche',
'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
'total_sans_secu', 'total_ceinture', 'total_casque',
'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
'18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
'2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# ♦ **Pipeline de preprocessing**
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)
], remainder='passthrough')

# ♦ **Modèle de classification**
classifier = RandomForestClassifier(random_state=42, class_weight='balanced')

# ♦ **Pipeline principal avec imblearn Pipeline**
pipeline = Pipeline([
    ('preprocessor', preprocessor), # Étape de transformation
    ('classifier', classifier) # Modèle final
])

# ♦ **Définition de la grille de recherche**
param_grid = {
    'classifier__n_estimators': [50, 100, 200], # Nombre d'arbres
    'classifier__max_depth': [10, 20, None], # Profondeur maximale
    'classifier__class_weight': ['balanced', None] # Tester 'balanced' et None
}

# ♦ **Lancer la GridSearchCV**
grid_search = GridSearchCV(pipeline, param_grid, cv=3, scoring='f1_weighted', verbose=2, n_jobs=-1)
grid_search.fit(X_train, y_train)

# ♦ **Afficher les meilleurs paramètres**
print("Meilleurs paramètres:", grid_search.best_params_)

# ♦ **Évaluation du modèle optimal**
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# ♦ **Matrice de confusion**
conf_matrix = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=np.unique(y),
yticklabels=np.unique(y))
plt.xlabel("Prédictions")
plt.ylabel("Vraies valeurs")
plt.title("Matrice de Confusion")
plt.show()

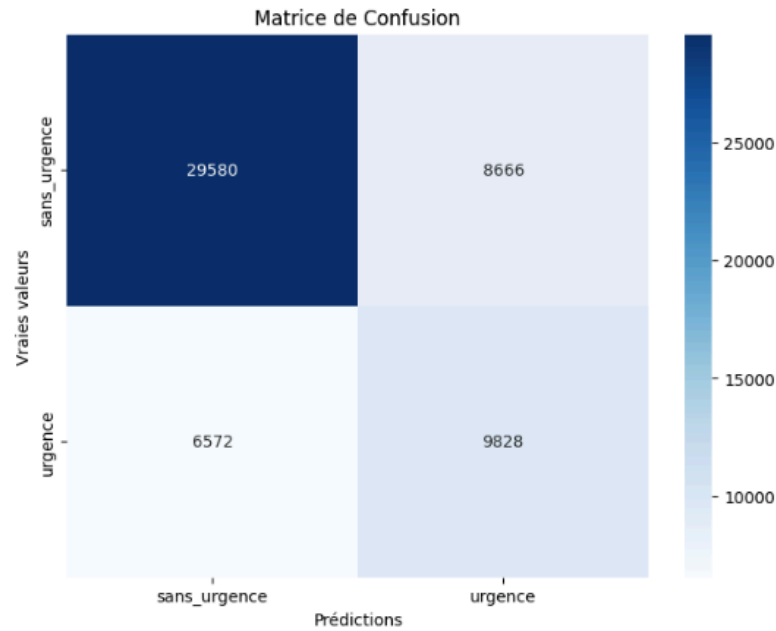
```

Meilleurs paramètres: {'classifier__class_weight': 'balanced', 'classifier__max_depth': 20, 'classifier__n_estimators': 200}

 **Performance du modèle optimal**
Accuracy: 0.72
Precision: 0.73
Recall: 0.72
F1-score: 0.73

Rapport de Classification:

	precision	recall	f1-score	support
sans_urgence	0.82	0.77	0.80	38246
urgence	0.53	0.60	0.56	16400
accuracy			0.72	54646
macro avg	0.67	0.69	0.68	54646
weighted avg	0.73	0.72	0.73	54646



```

# Random Forest avec métrique pondérée pour optimiser le Recall
from sklearn.metrics import recall_score

# Préparation des données
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
'sens_unique', 'bidirectionnel', 'route_seche',
'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
'total_sans_secu', 'total_ceinture', 'total_casque',
'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
'18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
'2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Préparation des données
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
'tué', 'blessé_léger',
'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

from sklearn.preprocessing import LabelEncoder

# Encoder les labels en entiers
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(df['gravité_accident'])

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

```



```

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche', 'route_mouillee_enneigee',
                        'etat_route_autre',
                        'usager_count', 'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant',
                        'total_gilet', 'total_airbag', 'total_gants', 'total_gants_airbag',
                        'total_autre', 'place_conducteur',
                        'pax_AV', 'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95',
                        'obstacle_fixe', 'obstacle_mobile', 'aucun_choc', 'choc_AV', 'choc_AR',
                        'choc_cote', 'choc_tonneaux',
                        'VL_VU', '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] #
À laisser inchangées

# Définir les différentes pondérations à tester (déjà définies dans ton code précédent)
class_weights_list = [{0: 1, 1: 2}, {0: 1, 1: 3}, {0: 1, 1: 5}, {0: 1, 1: 10},]

# Classe 0: sans_urgence
# Classe 1: urgence
# Définir une fonction pour la précision pondérée
def weighted_f1_score(y_true, y_pred, class_weights):
    # Calculer la précision pour chaque classe
    f1_per_class = f1_score(y_true, y_pred, average=None)

    # Associer les poids aux classes dans le même ordre que dans precision_per_class
    weights = [class_weights.get(i, 1) for i in range(len(f1_per_class))]

    # Calculer la précision pondérée en fonction des poids
    weighted_f1 = np.dot(f1_per_class, weights) / sum(weights)
    return weighted_f1

# Définir la pipeline complète
pipeline = Pipeline([
    ('preprocessor', preprocessor), # Transformation des variables
    ('classifier', RandomForestClassifier(n_estimators=50, max_depth=10, random_state=42)) #
    Classificateur
])

# Pour chaque configuration de poids, effectuer une validation croisée et calculer la précision
# pondérée
results = {}

for class_weights in class_weights_list:
    # Utiliser cross_val_score pour évaluer la performance avec les poids de classes
    f1_scores = cross_val_score(pipeline, X_train, y_train, cv=5, scoring=lambda est, X, y:
    weighted_f1_score(y, est.predict(X), class_weights) )

    # Enregistrer la moyenne des scores de précision pondérée
    results[str(class_weights)] = np.mean(f1_scores)

# Afficher les résultats pour chaque configuration de pondération
for class_weights, score in results.items():
    print(f"Configuration des poids {class_weights} -> Précision pondérée : {score}")

# Trouver la configuration qui donne la meilleure précision pondérée
best_class_weights = max(results, key=results.get)
print(f"\nMeilleure configuration de pondération : {best_class_weights}")

# Créer la pipeline en utilisant les poids optimaux dans le classificateur RandomForest
pipeline_with_optimal_weights = Pipeline([
    ('preprocessor', preprocessor), # Transformation des variables
    ('classifier', RandomForestClassifier(n_estimators=50, max_depth=10, random_state=42,
    class_weight=eval(best_class_weights) # Appliquer les poids ici )))

# Entraîner le modèle avec les poids optimaux
pipeline_with_optimal_weights.fit(X_train, y_train)

# Faire des prédictions sur l'ensemble de test
y_pred = pipeline_with_optimal_weights.predict(X_test)

# Evaluation de la performance du modèle

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall_optimal = recall_score(y_test, y_pred, average='weighted')
f1_score = f1_score(y_test, y_pred, average='weighted')

print("\n **Performance du modèle optimal**")
print(f"Accuracy: {accuracy:.2f}")
print(f"Precision: {precision:.2f}")
print(f"Recall_optimal: {recall:.2f}")
print(f"F1-score: {f1_score:.2f}")
print("\nRapport de Classification:")
print(classification_report(y_test, y_pred))

```

Configuration des poids {0: 1, 1: 2} -> Précision pondérée : 0.5588953910759638
 Configuration des poids {0: 1, 1: 3} -> Précision pondérée : 0.5254302035131241
 Configuration des poids {0: 1, 1: 5} -> Précision pondérée : 0.4919650159502845
 Configuration des poids {0: 1, 1: 10} -> Précision pondérée : 0.4615421181658849

Meilleure configuration de pondération : {0: 1, 1: 2}

 ****Performance du modèle optimal****

Accuracy: 0.72

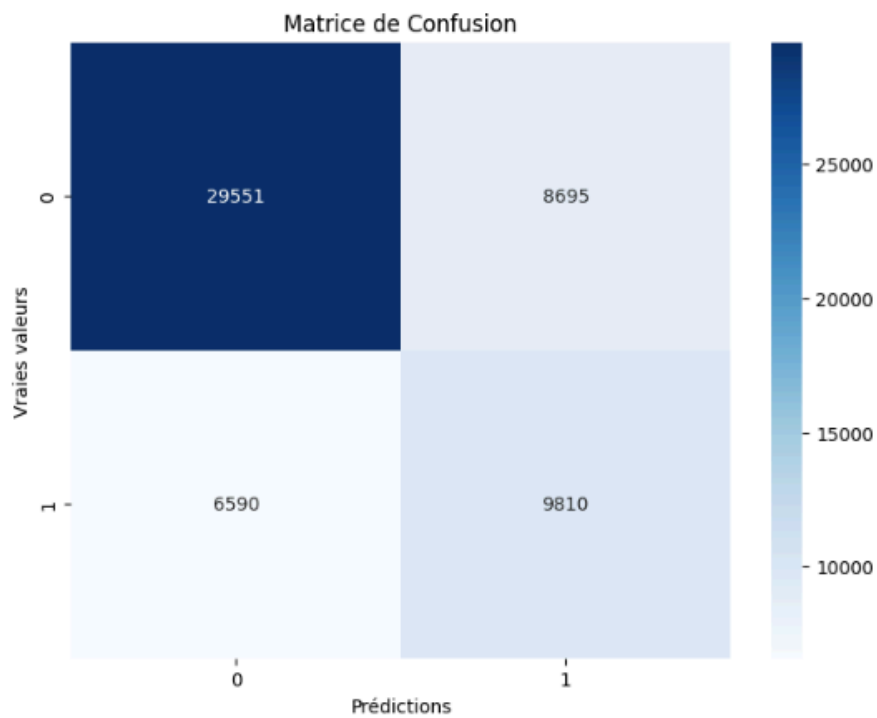
Precision: 0.73

Recall_optimal: 0.72

F1-score: 0.72

Rapport de Classification:

	precision	recall	f1-score	support
0	0.82	0.77	0.79	38246
1	0.53	0.60	0.56	16400
accuracy			0.72	54646
macro avg	0.67	0.69	0.68	54646
weighted avg	0.73	0.72	0.72	54646



```

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer X et y
X = df.drop(['lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche',
                        'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
                        'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# ♦ **Pipeline de preprocessing**
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)
], remainder='passthrough')

# ♦ **Définir un scorer personnalisé pour optimiser le F1-score de la classe 1 (hospitalisé)**
def class_specific_f1(y_true, y_pred, class_index=1):
    return f1_score(y_true, y_pred, labels=[class_index], average='micro')

# Crée un scorer personnalisé basé sur le F1-score pour la classe 1
custom_f1_scorer = make_scorer(class_specific_f1, class_index=1)

# ♦ **Modèle de classification**
classifier = RandomForestClassifier(random_state=42, class_weight='balanced')

# ♦ **Pipeline principal avec imblearn Pipeline**
pipeline = Pipeline([
    ('preprocessor', preprocessor), # Étape de transformation
    ('classifier', classifier) # Modèle final
])

# ♦ **Définition de la grille de recherche**
param_grid = {
    'classifier__n_estimators': [50, 100, 200], # Nombre d'arbres
    'classifier__max_depth': [10, 20, None], # Profondeur maximale
    'classifier__class_weight': ['balanced', None] # Tester 'balanced' et None
}

# ♦ **Lancer la GridSearchCV avec le scorer personnalisé**
grid_search = GridSearchCV(pipeline, param_grid, cv=3, scoring=custom_f1_scorer, verbose=2, n_jobs=-1)
grid_search.fit(X_train, y_train)

# ♦ **Afficher les meilleurs paramètres**
print("Meilleurs paramètres:", grid_search.best_params_)

# ♦ **Évaluation du modèle optimal**
best_model = grid_search.best_estimator_
y_pred = best_model.predict(X_test)

# Calcul des métriques
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

```

Meilleurs paramètres: {'classifier__class_weight': 'balanced', 'classifier__max_depth': 10, 'classifier__n_estimators': 50}

 ****Performance du modèle optimal****

Accuracy: 0.70

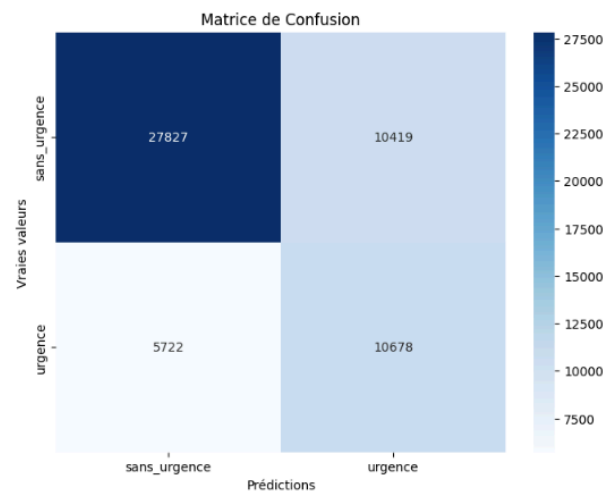
Precision: 0.73

Recall: 0.70

F1-score: 0.71

Rapport de Classification:

	precision	recall	f1-score	support
sans_urgence	0.83	0.73	0.78	38246
urgence	0.51	0.65	0.57	16400
accuracy			0.70	54646
macro avg	0.67	0.69	0.67	54646
weighted avg	0.73	0.70	0.71	54646



IV.1.2.h. Modèle 2 classes Régression Logistique

```

# Régression logistique simple
# Préparation des données (Exemple de df à adapter)
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
            'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche',
                        'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
                        'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL-VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète avec la régression logistique
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', LogisticRegression(max_iter=1000, random_state=42))] # Régression Logistique avec
itérations supplémentaires

# Entraîner le modèle
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['sans_urgence', 'urgence'],
            yticklabels=['sans_urgence', 'urgence'])
plt.xlabel('Prédictions')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

```

Accuracy du modèle : 0.73
Précision : 0.71
Rappel : 0.73
F1-score : 0.70

Rapport de Classification :

	precision	recall	f1-score	support
sans_urgence	0.76	0.91	0.83	38246
urgence	0.60	0.32	0.41	16400
accuracy			0.73	54646
macro avg	0.68	0.61	0.62	54646
weighted avg	0.71	0.73	0.70	54646

```

# Régression logistique avec oversampling
# Préparation des données (Exemple de df à adapter)
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Préparation des données (Exemple de df à adapter)
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche',
                        'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
                        'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète avec RandomUnderSampler (undersampling) et la régression logistique
pipeline = imPipeline([
    ('preprocessor', preprocessor), # Prétraitement des données
    ('undersample', RandomUnderSampler(random_state=42)), # Undersampling avec RandomUnderSampler
    ('classifier', LogisticRegression(max_iter=1000, random_state=42)) # Régression logistique
])

# Entraîner le modèle
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle avec undersampling : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

```

```

# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=[
    'blessé_léger', 'blessé_hospitalisé', 'tué'], yticklabels=['blessé_léger', 'blessé_hospitalisé', 'tué'])
plt.xlabel('Prédictions')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
    'sens_unique', 'bidirectionnel', 'route_seche',
    'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
    'total_sans_secu', 'total_ceinture', 'total_casque',
    'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
    'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
    'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
    '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
    'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
    '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète avec SMOTE (oversampling) et la régression logistique
pipeline = imPipeline([
    ('preprocessor', preprocessor), # Prétraitement des données
    ('smote', SMOTE(random_state=42)), # Oversampling avec SMOTE
    ('classif', LogisticRegression(max_iter=1000, random_state=42)) # Régression logistique
])

# Entraîner le modèle
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle avec oversampling : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

```


Accuracy du modèle avec oversampling : 0.68
Précision : 0.72
Rappel : 0.68
F1-score : 0.69

Rapport de Classification :

	precision	recall	f1-score	support
0	0.83	0.68	0.75	38246
1	0.47	0.68	0.56	16400
accuracy			0.68	54646
macro avg	0.65	0.68	0.65	54646
weighted avg	0.72	0.68	0.69	54646

```

# Régression logistique avec undersampling
# Préparation des données (Exemple de df à adapter)
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Préparation des données (Exemple de df à adapter)
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche',
                        'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
                        'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète avec RandomUnderSampler (undersampling) et la régression logistique
pipeline = imPipeline([
    ('preprocessor', preprocessor), # Prétraitement des données
    ('undersample', RandomUnderSampler(random_state=42)), # Undersampling avec RandomUnderSampler
    ('classifier', LogisticRegression(max_iter=1000, random_state=42)) # Régression logistique
])

# Entraîner le modèle
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle avec undersampling : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

```

```

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche',
                        'route_mouillee_enneigee', 'etat_route_autre', 'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
                        'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète avec RandomUnderSampler (undersampling) et la régression logistique
pipeline = imPipeline([
    ('preprocessor', preprocessor), # Prétraitement des données
    ('undersample', RandomUnderSampler(random_state=42)), # Undersampling avec RandomUnderSampler
    ('classifier', LogisticRegression(max_iter=1000, random_state=42)) # Régression logistique
])

# Entraîner le modèle
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle avec undersampling : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

```

Accuracy du modèle avec undersampling : 0.68
Précision : 0.72
Rappel : 0.68
F1-score : 0.69

Rapport de Classification :

	precision	recall	f1-score	support
sans_urgence	0.83	0.68	0.75	38246
urgence	0.48	0.67	0.56	16400
accuracy			0.68	54646
macro avg	0.65	0.68	0.65	54646
weighted avg	0.72	0.68	0.69	54646

IV.1.2.i. Modèle 2 classes KNN

```

# KNN avec scoring pour optimiser recall
# Préparation des données (Exemple de df à adapter)
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
            'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche',
                        'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
                        'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir la pipeline complète avec KNN
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', KNeighborsClassifier(n_neighbors=5))] # KNN avec 5 voisins

# Créer le scorer pour optimiser le recall
recall_scorer = make_scorer(recall_score, average='weighted')

# Recherche des hyperparamètres avec GridSearchCV
param_grid = {'classifier__n_neighbors': [3, 5, 7, 9, 11]} # Exemple de grid de voisins

# GridSearchCV pour optimiser le recall
grid_search = GridSearchCV(pipeline, param_grid, cv=5, scoring=recall_scorer)
grid_search.fit(X_train, y_train)

# Meilleurs hyperparamètres et meilleure score
print(f"Meilleurs paramètres : {grid_search.best_params}")
print(f"Meilleur recall : {grid_search.best_score:.2f}")

# Prédiction sur l'ensemble de test
y_pred = grid_search.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

```

```
Meilleurs paramètres : {'classifier__n_neighbors': 3}
Meilleur recall : nan
Accuracy du modèle : 0.70
Précision : 0.68
Rappel : 0.70
F1-score : 0.69
```

```
Rapport de Classification :
      precision    recall  f1-score   support

sans_urgence      0.77      0.81      0.79      38246
urgence           0.49      0.42      0.45      16400

   accuracy      0.70      0.70      0.70      54646
  macro avg      0.63      0.62      0.62      54646
weighted avg      0.68      0.70      0.69      54646
```

IV.1.2.i. Modèle 2 classes XGBoost

```

# XGBOOST avec l'argument classweight pour gérer le déséquilibre des classes
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler, LabelEncoder
from sklearn.compose import ColumnTransformer
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils.class_weight import compute_sample_weight
from xgboost import XGBClassifier
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparation des variables d'entrée et de sortie
X = df.drop(['lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger',
            'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

# Encoder les classes texte en entiers
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y) # Transforme 'sans_urgence' et 'urgence' en 0 et 1

# Séparation en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Variables à encoder
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche', 'route_mouillee_enneigee',
                        'etat_route_autre',
                        'usager_count', 'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant',
                        'total_gilet', 'total_airbag', 'total_gants', 'total_gants_airbag',
                        'total_autre',
                        'place_conducteur', 'pax_AV', 'pax_AR', 'pax_Milieu', 'place_pieton', 'homme',
                        'femme',
                        '0-17', '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV',
                        'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU', '2roues_3roues_quad', 'PL',
                        'bus_car',
                        'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Transformation des données
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features),
    ('scaler', StandardScaler(), passthrough_features) # Normalisation pour XGBoost
])

# Calcul des poids des classes
class_weights = compute_sample_weight(class_weight='balanced', y=y_train)

# Créer et entraîner le modèle XGBoost
model = XGBClassifier(
    objective='multi:softmax', # Classification multiclasses
    num_class=3, # Nombre de classes
    eval_metric='mlogloss', # Log loss
    use_label_encoder=False, # Suppression d'un avertissement lié à XGBoost
    random_state=42
)

# Appliquer la transformation de prétraitement sur les données d'entraînement
X_train_transformed = preprocessor.fit_transform(X_train)
X_test_transformed = preprocessor.transform(X_test)

# Entraîner le modèle avec les poids de classes
model.fit(X_train_transformed, y_train, sample_weight=class_weights)

# Prédictions
y_pred = model.predict(X_test_transformed)

```



```
# Évaluation du modèle
accuracy = accuracy_score(y_test, y_pred) # Les labels sont déjà sous forme numérique
print(f"Accuracy du modèle : {accuracy:.2f}")

# Rapport de classification détaillé
print("\nRapport de classification :")
print(classification_report(y_test, y_pred))

# Visualisation de la matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)
```

Accuracy du modèle : 0.70

Rapport de classification :

	precision	recall	f1-score	support
0	0.84	0.70	0.77	38200
1	0.50	0.70	0.59	16446
accuracy			0.70	54646
macro avg	0.67	0.70	0.68	54646
weighted avg	0.74	0.70	0.71	54646


```

# XGBOOST avec oversampling
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from imblearn.over_sampling import SMOTE
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix,
classification_report
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.base import BaseEstimator, TransformerMixin
from xgboost import XGBClassifier

# Préparation des données
X = df.drop(['Num_Acc', 'lat', 'long', 'jour', 'mois', 'date', 'an', 'hrmn', 'dep', 'int', 'col',
            'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé', 'gravité_accident'], axis=1)
y = df['gravité_accident']

from sklearn.preprocessing import LabelEncoder

# Encoder les labels en entiers
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(df['gravité_accident']) # Convertit 'sans_urgence' -> 0 et 'urgence' -> 1

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma', 'nationale_departementale_communale', 'autoroute', 'autre_route',
                        'sens_unique', 'bidirectionnel', 'route_seche',
                        'route_mouillee_enneigee', 'etat_route_autre', 'usager_count',
                        'total_sans_secu', 'total_ceinture', 'total_casque',
                        'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
                        'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
                        'pax_AR', 'pax_Milieu', 'place_pieton', 'homme', 'femme', '0-17',
                        '18-60', '61-95', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
                        'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux', 'VL_VU',
                        '2roues_3roues_quad', 'PL', 'bus_car', 'velo_trott_edp', 'nbr_veh'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Appliquer SMOTE dans la pipeline (avant l'entraînement du modèle)
smote = SMOTE(random_state=42, sampling_strategy = 'auto', k_neighbors = 5)

```

```

# Définir la pipeline complète avec SMOTE
pipeline = imPipeline([
    ('preprocessor', preprocessor), # Transformation des variables
    ('smote', smote), # Oversampling via SMOTE
    ('classifier', XGBClassifier(objective='multi:softmax', num_class=3, eval_metric='mlogloss',
random_state=42)) # XGBoost
])

# Entraîner le modèle avec oversampling
pipeline.fit(X_train, y_train)

# Prédiction
y_pred = pipeline.predict(X_test)

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle avec oversampling : {accuracy:.2f}")

# Évaluation
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy du modèle : {accuracy:.2f}")

# Précision, Rappel, F1-score
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1 = f1_score(y_test, y_pred, average='weighted')

# Matrice de confusion
conf_matrix = confusion_matrix(y_test, y_pred)

# Rapport de classification
class_report = classification_report(y_test, y_pred)

# Afficher les résultats détaillés
print(f"Précision : {precision:.2f}")
print(f"Rappel : {recall:.2f}")
print(f"F1-score : {f1:.2f}")
print("\nRapport de Classification :")
print(class_report)

# Matrice de confusion
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues', xticklabels=['Classe 0', 'Classe 1'],
yticklabels=['Classe 0', 'Classe 1'])
plt.xlabel('Prédiction')
plt.ylabel('Vraies valeurs')
plt.title('Matrice de Confusion')
plt.show()

# Récupérer l'importance des features
# Récupérer le modèle du classifieur après l'entraînement
model = pipeline.named_steps['classifier']

# Accéder aux importances des features
importances = model.feature_importances_

# Récupérer les noms des features après transformation
# Utiliser l'encodeur OneHotEncoder pour gérer les variables catégorielles encodées
# et la transformation cyclique pour obtenir les noms des features résultants.
cat_columns = pipeline.named_steps['preprocessor'].transformers_[0]
[1].get_feature_names_out(categorical_features)
cyclical_columns = ['heure_sin', 'heure_cos'] # Ces noms sont définis par la transformation de l'heure

# Les noms des features après transformation
features = np.concatenate([cat_columns, cyclical_columns, passthrough_features])

# Tracer l'importance des features
plt.figure(figsize=(12, 12))
plt.barh(features, importances)
plt.xlabel("Importance des Features")
plt.title("Importance des features dans le modèle Random Forest avec oversampling")
plt.show()

```

Accuracy du modèle avec oversampling : 0.74
 Accuracy du modèle : 0.74
 Précision : 0.73
 Rappel : 0.74
 F1-score : 0.73

Rapport de Classification :

	precision	recall	f1-score	support
0	0.79	0.85	0.82	38246
1	0.58	0.47	0.52	16400
accuracy			0.74	54646
macro avg	0.68	0.66	0.67	54646
weighted avg	0.73	0.74	0.73	54646

IV.1.2.j .Essai des deux meilleurs modèles , Random Forest à 3 classes et XGBoost à 2 classes en sélectionnant les variables facilement accessibles par un boitier situé dans les voitures accidentées

```

#Random Forest

# Préparation des données
X = df.drop(['Num_Acc', 'lat', 'long', 'date', 'jour', 'mois', 'an', 'hrmn', 'dep', 'int', 'col',
'tué', 'blessé_léger', 'indemne', 'blessé_hospitalisé',
'gravité_accident', 'nationale_departementale_communale', 'autoroute', 'autre_route',
'sens_unique', 'bidirectionnel', 'route_seche',
'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
'total_sans_secu', 'total_ceinture', 'total_casque',
'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
'pax_AR', 'pax_Milieu', 'place_pieton', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux'], axis=1)
y = df['gravité_accident']

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparer en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Liste des variables catégorielles et cycliques

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma',
'homme', 'femme', '0-17',
'18-60', '61-95'] # À laisser inchangées

# Appliquer le ColumnTransformer avec `remainder='passthrough'`
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features)], remainder='passthrough')

# Définir les pondérations des classes
class_weights = {0: 1, 1: 2, 2: 1}

# Définir la pipeline complète
pipeline = Pipeline([
    ('preprocessor', preprocessor),
    ('classifier', RandomForestClassifier(n_estimators=50, class_weight=class_weights, max_depth=10,
random_state=42))])

# Entraîner le modèle avec les poids optimaux
model_boitier_random_forest_3_classes = pipeline.fit(X_train, y_train)

# Faire des prédictions sur l'ensemble de test
y_pred = model_boitier_random_forest_3_classes.predict(X_test)

# Evaluation de la performance du modèle

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
f1_optimal = f1_score(y_test, y_pred, average='weighted')

```

Accuracy: 0.67
Precision: 0.65
Recall: 0.67
F1-score pondéré avec les poids optimaux: 0.657698764109621

Rapport de Classification:

	precision	recall	f1-score	support
0	0.78	0.77	0.77	35156
1	0.48	0.59	0.53	16400
2	0.00	0.00	0.00	3090
accuracy			0.67	54646
macro avg	0.42	0.45	0.44	54646
weighted avg	0.65	0.67	0.66	54646

```

# XGBoost

# Définition de la classe pour la transformation de l'heure
class CyclicalFeatures(BaseEstimator, TransformerMixin):
    """Transforme une colonne de type heure en variables cycliques sin et cos."""
    def __init__(self, period=24):
        self.period = period

    def fit(self, X, y=None):
        return self

    def transform(self, X):
        X = X.astype(float) # Assurer que les valeurs sont numériques
        X_sin = np.sin(2 * np.pi * X / self.period)
        X_cos = np.cos(2 * np.pi * X / self.period)
        return np.c_[X_sin, X_cos] # Retourne un array 2D avec sin et cos

# Séparation des variables d'entrée et de sortie
X = df.drop(['lat', 'long', 'date', 'an', 'jour', 'mois', 'hrmn', 'dep', 'int', 'col', 'tué',
            'blessé_léger',
            'indemne', 'blessé_hospitalisé', 'gravité_accident', 'nationale_departementale_communale',
            'autoroute', 'autre_route',
            'sens_unique', 'bidirectionnel', 'route_seche',
            'route_mouillée_enneigée', 'etat_route_autre', 'usager_count',
            'total_sans_secu', 'total_ceinture', 'total_casque',
            'total_secu_enfant', 'total_gilet', 'total_airbag', 'total_gants',
            'total_gants_airbag', 'total_autre', 'place_conducteur', 'pax_AV',
            'pax_AR', 'pax_Milieu', 'place_pieton', 'obstacle_fixe', 'obstacle_mobile', 'aucun_choc',
            'choc_AV', 'choc_AR', 'choc_cote', 'choc_tonneaux'], axis=1)
y = df['gravité_accident']

print(X.columns)
# Encoder les classes texte en entiers
label_encoder = LabelEncoder()
y = label_encoder.fit_transform(y) # Transforme 'sans_urgence' et 'urgence' en 0 et 1

# Séparation en train/test
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

# Liste des variables catégorielles et cycliques
categorical_features = ['agg', 'atm'] # Encodage
cyclical_features = ['heure'] # Transformation cyclique
passthrough_features = ['nbv', 'vma',
                        'homme', 'femme',
                        'PL', 'bus_car', 'VL_VU', '2roues_3roues_quad', '0-17',
                        '18-60', '61-95'] # À laisser inchangées

# Transformation des données
preprocessor = ColumnTransformer([
    ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
    ('cyclical', CyclicalFeatures(), cyclical_features),
    ('scaler', StandardScaler(), passthrough_features) # Normalisation pour XGBoost
])

# Calcul des poids des classes
class_weights = compute_sample_weight(class_weight='balanced', y=y_train)

# Créer et entraîner le modèle XGBoost
model = XGBClassifier(
    objective='multi:softmax', # Classification multiclass
    num_class=3, # Nombre de classes
    eval_metric='mlogloss', # Log loss
    use_label_encoder=False, # Suppression d'un avertissement lié à XGBoost
    random_state=42
)

# Appliquer la transformation de prétraitement sur les données d'entraînement
X_train_transformed = preprocessor.fit_transform(X_train)
X_test_transformed = preprocessor.transform(X_test)

# Entraîner le modèle avec les poids de classes
model.fit(X_train_transformed, y_train, sample_weight=class_weights)

# Prédiction
y_pred = model.predict(X_test_transformed)

# Évaluation du modèle
accuracy = accuracy_score(y_test, y_pred) # Les labels sont déjà sous forme numérique
print(f"Accuracy du modèle : {accuracy:.2f}")

# Rapport de classification détaillé
print("\nRapport de classification :")
print(classification_report(y_test, y_pred))

```

Accuracy du modèle : 0.67

Rapport de classification :

	precision	recall	f1-score	support
0	0.82	0.67	0.74	38200
1	0.46	0.65	0.54	16446
accuracy			0.67	54646
macro avg	0.64	0.66	0.64	54646
weighted avg	0.71	0.67	0.68	54646